Project History — Global Inference on Boundary-Head Candidates

For the same runs as a table rather than a narrative, see [[EXPERIMENT_CATALOG]] — newest first, with research line, outcome and cost.

> ⚠ **2026-08-18: every joint_ie / 137k number in this document predates a data repair and is superseded.** 45 corpora shipped overlapping train/val/test, and the scaling configs additionally paired regenerated train files against frozen val slices (252 train-in-val, 22 val-in-test). All were repaired, the slices rebuilt, and the four scaling points now gate CLEAN. The curve is being re-run from scratch — see `JOINT_IE_SCALING.md` and [[lambda-137k-curve-restart]]. Numbers below are kept as the record of what was measured and believed at the time; do not compare new results against them.

A chronological record of what was decided, why, and what later proved wrong. The other working papers state conclusions; this one states the *path*, including the reversals, because several of the most useful results in this program are corrections to earlier results in this program.

This is the project's single history. A working document whose purpose is finished is retired into it rather than left in the paper set: its narrative goes into the phase that covers it, anything still load-bearing graduates into the paper that needs it, and the file is removed. The full original text of every retired document remains in git — this carries the story, not the archive. Retired documents are listed at the end with what they concluded and what replaced them.

The papers state what is true now; this states how it was found and what was believed before. Dates are commit dates. Numbers are measured unless marked as an estimate.

Companion documents: [[RESEARCH_PROGRAM]] (thesis + paper map), [[EKF_MHT_DESIGN]] (tracker line), [[JOINT_IE_SCALING]] (curve line), [[BOUNDARY_ARCHITECTURE]] (the head), [[PAPER_0_FOUNDATION]] (foundation draft).

Origin — September 2024

The idea does not start in the commit log, and a journal built only from `git log` would miss it by two years.

September 2024 — Weaver's "*Exposing the Power of the Kalman Filter*" (Towards Data Science, 7 Nov 2023) is read. It walks the predict/update cycle from first principles and ends by motivating the **Extended** Kalman filter for nonlinear systems. The question it raises — could a Kalman filter track a real-world quantity reported across a stream of documents? — is the one this whole programme exists to answer.

September 2024 – August 2026 — turned over, not built. The first commit of this line lands 2026-08-07, roughly two years later.

The question, in the form it was actually asked: *can a Kalman filter track real-world events reported in text, and **diarize** them into separate streams?* Two halves — track, and decide which stream each observation belongs to. The second is not a downstream detail; it is half the question, and it is the half that has failed in every real-event test so far.

That ordering is worth recording because it is not recoverable from the code, and it explains the shape of everything below: **the filter came first and the extraction second.** Document-level event extraction, the boundary head, the counting layer and the entire joint_ie line are not the goal — they are what the original question turned out to require before it could be asked properly. It is also why [[EKF_MHT_DESIGN]] §1 scopes its claim so narrowly (*streaming, quantitatively-evolving events*, explicitly NOT "EKF helps event extraction") — that scope is the original question, kept honest.

Phase 0 — inherited baseline (to mid-July 2026)

The repository begins 2025-07-07 as GLiNER2 proper: a span-architecture extractor with entity, relation, classification and JSON-structure heads driven by query markers. Nearly all of that predates this research program. The program starts where the foundation paper starts.

Phase 1 — foundation, and the head-init finding (16 Jul – 5 Aug)

Threshold calibration became a first-class step (16 Jul). A post-training sweep picks the operating point rather than assuming 0.5. This choice pays off, badly and twice, much later — see the matched-threshold entries below.

Document-level extraction was scoped (17 Jul): `DOCUMENT_EXTRACTION_PLAN.md`, a global event decoder, and an opt-in `global_decode` path. The idea that a greedy per-query decode leaves something on the table is the seed of the whole joint_ie line.

Infrastructure, deliberately, before results (18–23 Jul): spot-resume, "fail loud on missing `metric_for_best`", event converters (ChFinAnn, DocFEE, DuEE, Mendeley), the extraction viewer, HF push tooling. The `metric_for_best` guard is worth noting — it was added defensively and later caught a real failure.

Decision: report fair span diagnostics (16–17 Jul), following Ortmann 2022, instead of a single micro-F1. This is the first instance of a habit that recurs throughout: prefer the measurement that can embarrass you.

The head-init finding (31 Jul, §10.6): training *from an encoder* with fresh heads behaves very differently from warm-starting them. Follow-ups on 3 Aug recorded a **negative** A/B and a sanity run showing fresh heads cannot bootstrap from synthetic data alone.

The mmBERT head-init scaling curve (4–5 Aug): 10K / 40K / 100K base points, RAMS argument-strict F1 **0.050 → 0.115 → 0.158**. Knee between 10K and 40K, still climbing at 100K. Recorded then as a data-volume law about mmBERT.

> **Later overturned.** See 9 Aug: the same curve on the *boundary* head is nearly flat (0.177 → 0.202). The deficit was a property of the **span head**, not of mmBERT or of data volume. The original reading was not wrong about its own numbers; it was wrong about what they generalised to.
>
> **And 15 Aug: "nearly flat" is really "flat within noise".** A control re-run of the published 137K recipe scored +0.023 above it, so single-run variance is ≥±0.02 and the whole boundary spread (0.177–0.215) fits inside it. The span-head conclusion survives and is strengthened — span climbs 0.108, five times the variance — but no shape should be read off the boundary row.

Phase 2 — the architecture pivot (24 Jul – 6 Aug)

joint_ie decoding lands (24 Jul), then **the boundary architecture** (27–28 Jul) — a head with no span-width cap, replacing the span head's 20-width lattice.

Decision: adopt boundary as the architecture for this program (JOINT_IE_SCALING decision 1). Rationale: dense document-level relations are exactly where a 20-width cap bites, and the whole thesis is about output that does not fit a per-query greedy decode.

Merge of origin/main (5 Aug) brought the boundary rewrite in as the baseline, with our features re-ported on top over seven commits. A parallel decision from the same week (21 Jul) kept our DDP implementation over main's while adopting main's tests.

Counting layer work (6 Aug) with a documented 19-instance cap.

Phase 3 — the EKF/MHT line, built in a day (7 Aug)

Seventeen commits, and the shape of the argument matters more than the count.

Decision: synthetic streams before real news. A parametric disaster-stream generator with regimes, so the tracker could be developed against ground truth that actually exists. Cost: $0.

The sequence was deliberately adversarial to its own thesis:

1. tracker + baselines harness
2. **MoE gate (Reading B) — "beats both baselines and the EKF"**, recorded as such
3. harder-regime ablation + an estimate-scaled `R` fix, where the EKF earns its keep
4. learned gate, cross-regime transfer, union gate
5. text → observation extraction and normalization
6. Sonnet-5 realizer (real prose from synthetic streams) + a one-sided innovation gate
7. model extraction arm; **confidence-as-soft-R tested and recorded as a negative**
8. the `missing`-role probe: the weak role was a **confidence-cut selection bias**, an extractor problem, not a normalization problem

9. fine-tune on 250 realized streams — end-to-end EKF **0.291 → 0.165**, `missing` **0.458 → 0.122**

Decisions with reasons, recorded as negatives: a symmetric 3σ gate is catastrophic on rising tolls (use the one-sided, dynamics-aware gate); confidence-as-soft-R does not beat a hard cut, because zero-shot errors are gross false positives rather than graded noise.

Venezuela 2026 scaffolding was built the same day as the intended *blind* test — an event after the model cutoff. It remains the only genuinely blind test available and is still unrun.

Phase 4 — joint_ie design, and a blocker taken seriously (7–8 Aug)

The boundary→joint_ie adapters were built and unit-tested: mentions, relation pairs, then `joint_decode` end to end.

Decision 4b, which blocked Phase A (8 Aug): the beam modelled only nodes (mentions) and edges (relations) — there was no record/instance concept, so events would bypass the beam entirely and decode greedily. With RAMS in the warm-start, events were on the evaluated path. Rather than proceed and quietly compare a beam arm against a partly-greedy arm, the blocker was declared and the design extended (events as trigger node + role edges; anchorless structures).

Decision: every scaling point carries relation data (8 Aug) at the pool's own 73/27 ratio, "removes a real confound" — otherwise volume and *whether the relation head was warmed at all* would move together.

Decision: drop SciERC on licensing (8 Aug), not leakage. It contributed 265 of 37,237 relation records (0.19%) and alone forced every model card to "research use only". A negligible data cost for a materially better license posture, taken because the models are intended to be public.

Decision: exclude DocRED because Re-DocRED re-annotates the same documents — leakage, checked rather than assumed (a 3,000-sentence `sentence_rex` sample had zero verbatim overlap with Re-DocRED's 3.2M characters).

Phase 5 — the bug week (8 Aug)

A dense run of defects, each found by trying to run the thing rather than by reading it.

- **Boundary layout omitted event-role queries** — `[V]` was missing from the extractive markers. Fixed, with a test asserting layout/marker parity for all five task types.
- **Evaluation aborted on any unalignable entity surface.** `error_policy` (malformed records) and `on_missing_surface` (alignment) are different knobs; the latter was never forwarded, so it sat at "raise" and killed runs at the first epoch-end eval.
- **FA2 raised on every inference call** — weights load fp32 by design and training wraps forward in autocast, but inference never did.

- **Blind test loaded boundary checkpoints as span**, dying on `config.max_width` *after* training had fully completed and saved.
- **Pushed models had no model card at all.**
- **NaN root cause** (8 Aug): single-variable matrix on one H100, 60 steps, deterministic — sdpa+bf16 goes non-finite at step 15 at 2.0 samples/s; **FA2+bf16 is clean at 22.0**. Either variable alone fixes it, so it is the interaction. Never DDP, never the data, never the head.

Decision: `finalize_run.py` — replay the post-training tail (blind test + card) against an existing checkpoint, so a job whose tail crashes does not require retraining.

Phase 6 — the curve runs (9–10 Aug)

Events emitted nothing, and had never emitted anything (9 Aug). Every event metric read 0.0000 at every threshold down to 0.01. The boundary engine skipped non-entity queries assuming the record head would decode them — but the record head is **inert for events** (an events schema produces no `record_metadata`, so `compile_record_specs` returns `{}`). The models had learned trigger and role spans all along; nothing assembled them. Two lessons recorded: a metric that cannot leave 0.0 is indistinguishable from a model that learned nothing, and `metric_for_best` had been pointing at exactly such a metric.

The blind test contained no events despite 73% event training data (9 Aug). No event corpus declared a `test:` key — and fixing that was not enough, because `_event_split` filters on `Path(p).is_file()` and the slices had never been copied to the box. A missing-path filter that drops silently is indistinguishable from "this corpus has no test data".

First boundary point: 10K on the boundary head beat the span curve's 100K on arguments.

The matched-threshold rule, first use (9 Aug): warm starts *appeared* to regress with more base data. They had not; the points were calibrated at different thresholds. A cross-model warning was added to `model_card.py`.

The curve completed, 12/12 arms, ~23h on one H100 (10 Aug). Event-argument F1, all points threshold-matched at 0.3:

	10K	40K	100K	137K
mmbert (cold base)	0.010	0.039	0.079	**0.098**
rams (warm start)	0.177	0.191	**0.202**	0.192

Finding, then same-day retraction. It was written up as "warm-starting and base-scaling saturate at different points". A support check run afterwards showed the mmbert arms straddle the 9 Aug blind-test fix: 10k/40k/100k scored on 3,527 argument instances, 137K on 20,845. So the cold-base row is not a curve, and the cross-row comparison never shared a test set. **Only the RAMS row survives** (support 2,016 throughout): it saturates from 100K, and the dip is saturation not decline — one seed, ~5% relative.

The failure is the matched-threshold lesson one level up. Thresholds were checked because they had already overturned a result; **support** was not, and it is the same class of error. `compare_capabilities.py` now prints support beside every metric and flags a change.

The matched-threshold rule, second use. Re-DocRED looked erratic (relation 0.176 / 0.136 / 0.207 / 0.176). Its thresholds *alternate* 0.1/0.3/0.1/0.3; split by threshold, both halves rise monotonically. Not noise — artefact. Two process fixes recorded: `metric_sweep: true` is right for shipping one model and wrong for a curve, and `test_metrics.json` does not record its threshold, so comparability cannot be checked from the metrics alone.

All 12 arms are on HF (private), plus 26M of training logs pulled before the box was destroyed.

Phase 7 — real events (9–10 Aug)

Turkiye–Syria 2023

Decision: pre-register before running, including predictions, because the outcome was already known to the model driving the pipeline (Feb 2023 predates the cutoff). Ground truth was sourced from one Al Jazeera tracker page sampled through the Wayback Machine, one archive URL per point — **search-result summaries were rejected outright** after they contradicted each other on dates (31,643 for 9 Feb *and* 17,134 for 10 Feb).

Decision: truncate the series at 21 Feb. The page froze at 41,000 and reported it into April while the real toll reached 53,537 — staleness of the source, provable because Al Jazeera itself published 44,218 on 24 Feb. Left in, a tracker that simply stopped updating would have scored perfectly for six weeks.

Result: a negative one. Extraction read the trajectory nearly point-for-point; `est_last_value` **beat** the EKF (0.136 vs 0.208); the 1999 Izmit toll of 17,500, quoted in the article's history section, was tracked as a 2023 figure in every configuration; and Syria was never recovered, because the association key was computed per *document*.

A pre-registered prediction failed, and the reason outlived the experiment: range-normalized RMSE never noticed that 12 of 20 readings came from a different earthquake, because 17,500 sits mid-range of a 1,014 → 41,000 trajectory. **A badly wrong observation scored well.** Only carrying the baseline exposed it.

Then attribution was solved — by framing, not by machinery. Record extraction has an inverted-U response to context volume: below ~750 characters the record head does not fire; at 1,000–1,500 it emits exactly two records and binds **both** countries 16/16; above ~2,500 it fires normally but binds the wrong pairs. Asking the record head for the `location` it already knew, at the right window, gave turkey 0.107 and syria 0.075.

Correction recorded the same day: an earlier claim that the encoder "physically cannot see the whole document" past 512 tokens was **wrong** — DeBERTa-v3 uses relative attention only, GLiNER2 does no chunking, and entities were demonstrably found at character 6,038. The mechanism is degradation past the trained length, not truncation.

And the benchmark cannot score the filter. With clean attribution `last_value` hits 0.000 — necessarily, since the ground truth was read from the same sentence the extractor reads. The baseline is an oracle by construction. Turkiye tests extraction and attribution; it cannot test the tracker.

`extract_long` supersedes the lead-window fix: GLiNER2 already ships overlapping-chunk long-document extraction, and at `chunk_size=200` it gets 16/16 on both countries over the *whole* article, with no dependence on where a publication puts its historical section.

Hurricane Helene 2024

Decision: split the sources. Ground truth from Wikipedia's per-state casualty table (31 dated snapshots, 7 state streams + Total, with genuine downward revisions), feed from AP wire prose — so `last_value` is no longer an oracle.

Result: the Turkiye-solved configuration does not generalise. It fragments one event into **18 association keys** spanning every geographic granularity, and the event type varies per article. Turkiye had two clean country-level places; Helene has a hierarchy.

The required fix is the opposite of Turkiye's. There the pipeline had to *split*; here AP reports the national total and rarely breaks it down, so it must *pool*. The extracted values in time order are the national total trajectory. **Neither "always split" nor "always pool" is right — correct granularity is a property of what the source reports**, and nothing infers it.

Scored as one national stream: EKF 2.042 vs `last_value` 2.125. The EKF edges a non-oracle baseline for the first time, but both are worse than predicting a constant, so the win is not claimed. Recall is the limit: 25 observations from 70 articles.

Standing position: three real-world attempts, zero clean wins for the tracker. Turkiye could not test it; Helene cannot yet reach it.

Phase 8 — the warm-start run (10 Aug)

Diagnosis first: `mmbert-137k` cannot do `[C]` record extraction at all — `None` at every threshold down to 0.01, with `enable_records: true` and a real `record_decoder` present. The cause is the mixture. It contains a corpus **named `text2json` whose 7,817 records all supervise `entities`, not structures**, so the record head was never taught the task. The same audit explains the weak NER: those records were the *only* entity supervision, 5.7% of a 137K mix.

Decision: warm-start with replay at 70/30 (user's ratio, from experience). Replay is sampled across both old task families at the pool's own 73/27 ratio, and the mixture is pre-shuffled at the example level so every batch is a mixture rather than a block. NER is split evenly between `pile_ner_def` and `nuner_full` rather than by pool size, because pool size reflects how big a dataset someone published, not how useful it is.

Decision: heterogeneous field types. The casualty corpus gains a string `location` beside its numeric roles, because the previous fine-tune saw only numeric fields and collapsed to "emit a digit" — asked for a `location` it returns the number.

Two false starts, recorded because the second correction reverses the first:

1. The run trained correctly but at **4.6 samples/s against the curve's 22**, ETA 14h. It was killed after ~$6 rather than paying 4.6× an estimate.
2. **Profiling blamed document length.** It found a real effect — cost is superlinear in sequence length, and `text2json` again (30.6% over 4,000 chars, max 102,068) sits in the *replay* list. A 2,000-char cap gives 1.87× for no loss in record count.
3. **But that was a secondary factor.** The dominant cause was **FA2 silently not loading**. `transformers` was unpinned, a fresh box resolved 5.13 against a `kernels` pin matching 5.6, the ranges are disjoint, and the encoder fell back to sdpa — which is both ~11× slower and numerically fatal on bf16 ModernBERT. With FA2 restored: **56.9 samples/s, zero non-finite losses, ~70 minutes**.

The profiling was run on CPU, where FA2 does not exist, so it was structurally incapable of finding the real cause. A secondary factor was presented as the answer.

Fixes: `transformers` pinned alongside `kernels` with a note that the ranges move together; `GLINER2_STRICT_ATTN=1` turns the first fallback into a load-time error so a run that requires FA2 fails before spending GPU hours.

Phase 9 — the record head finally learns (10 Aug, late)

The warm start was rerun as a two-arm A/B after the first attempt produced correct multi-instance records with `location: None` every time.

Diagnosis, confirmed. The corpus emitted no `record_metadata`, so `compile_record_specs` returned nothing and the record head was never supervised. Declaring `mode=` in `build_multievent_corpus.py` fixed it: **`location` fills 6/9 (4/9 correct)** against 0/9 in every prior configuration.

`anchorless` learns nothing (1/9 instances). The earlier evidence had been explicitly discounted -- a model never trained on anchorless failing to decode it says nothing about the mode -- so it was run rather than assumed. Training on it did not rescue it.

The two arms explain each other. `natural` costs relation -0.037; `anchorless` costs -0.002 and is flat everywhere. One arm learned a task and displaced capacity; the other learned nothing to displace anything with. That reframes the regression as a **price**, not a defect.

Two self-inflicted costs worth recording. A loader change made earlier the same day was verified by LOADING a checkpoint but never by running a forward -- the hub FA2 repo form is CUDA-only and raised at first forward on CPU, which is worse than the fallback it replaced. And the 2xH100 box ran ~40 minutes past the point every artifact was local, roughly $5-6, because analysis was treated as the task and the machine as background. Second cost lapse of the day; the first was launching a run whose throughput had never been sanity-checked.

Phase 10 — the beam arm was never runnable (10 Aug, close)

A question about the mention-key format ("if there truly are duplicate spans, can we dedup them?") turned out to be aimed at a real defect and at an imprecise description of it.

The description was wrong first. The note said the collision came from `role_name` being `'head'` / `'tail'`. Relation field names are arbitrary and binding is positional, so that was incidental. The defect is that the **relation type is dropped**: `spec["task_name"]` exists at `model.py:1260` and `query_types` is built from `field_name` alone at `engine.py:516`.

Dedup was the wrong fix, and the probe is why. On `deaths_in {head,tail}` + `injured_in {head,tail}`: 512 mentions, 271 unique keys, 241 colliding — and **0 of 241 collisions had matching logits**. Each relation type is its own schema group with its own query embeddings, so each scores the span independently. These were duplicate *keys*, not duplicate *mentions*; merging them would have scored one relation's role edges against another relation's mention evidence. Running-but-wrong, which is worse than crashing.

Two things the reproduction found that reasoning had not. The raise is *threshold-dependent* — no raise at 0.5, raises at 0.05 and 0.01 — so a schema looks fine until the model gets confident. And `TypedEndpoints` had the same root cause: with bare role names every relation declares `("head",)/("tail",)`, so endpoint typing was **vacuous across relation types**. The plan for number-to-place attachment depends on exactly that constraint discriminating, so dedup would have quietly forfeited the thing it was meant to enable.

Fixed by qualifying the node type per query (`qualified_query_type`, `models/base.py`) at all three sites at once — `query_types`, `_query_type` for the pair endpoint keys, and the `TypedEndpoints` construction — because a one-sided fix does not raise, it drops every edge through the `keep_ids` filter and returns empty, which reads as "the model found nothing". That is the §3 empty-layout failure a second time.

So "no raise" was refused as the acceptance test. Measured after the fix: 0 collisions, **256/256 edges resolving to nodes**, 18 surviving threshold 0.05. The regression tests were then checked by reverting the fix and confirming both fail.

Consequence for the papers. The beam arm has never run on a schema with two relation types. Every number in the 12-arm curve is the greedy arm. The papers call the comparison "unmeasured"; until today it was *unrunnable*.

Phase 11 — Phase A finally runs, and answers a different question (10 Aug, close)

With the collision fixed, the beam arm ran for the first time: Re-DocRED, 96 relation types, one trained model, eval-time `decode_mode` switch.

The headline number is not the finding. Joint beat greedy 0.1803 to 0.0740 on relation F1 at threshold 0.5 — and §4b's own table already had greedy at 0.176 at threshold 0.3. Threshold 0.5 is near greedy's worst operating point, so the "win" was mostly an operating point. **Third time the matched-threshold rule**

has overturned a conclusion here, which is enough times that it stops being a lesson and becomes a standing rule.

Beam width should be 1, and that is the real result. Seven widths, monotone decline (0.2406 at W=1 to 0.2058 at W=64), entity metrics byte-identical throughout. Widening drops 40 predictions of which 18 were correct — better precision, worse F1. The beam maximizes the objective better as it widens (it even keeps greedy as a floor, so score is monotone), and the objective is not F1. **A better search on a mis-specified objective is worse output.** So the gain over greedy is the *formulation* — constrained joint selection — not the search, and Phase A's "greedy vs beam" framing is mis-specified.

Checking a remembered fact paid. The recollection was that OneIE used beam width 3; the paper says θ=10, the released package defaults to 5. But the instinct that OneIE used a *small* beam was right, and the reason is structural — their β=2 caps branching at 2 per step, so θ=10 is a wide search relative to their space. Worth recording that the correction and the intuition were both useful: the number was wrong and the direction was right.

Then the arm exposed the actual bug. Joint recall barely moved across thresholds (0.1498 → 0.1591) while greedy's moved 9× (0.0461 → 0.4134). Cause: `joint_decode` never passed `decision_threshold`, so it sat at 0.5 while utilities were centered there — edge selection ignored `--threshold` entirely. I had described that flatness as "threshold insensitivity, a useful property" one message earlier. It was a plumbing bug. `JOINT_IE_SCALING` had *predicted* this exact issue in its arm-comparability caveat and called it "moot for a single-threshold eval". It was the dominant confound.

A rejected design, recorded because the reasoning generalizes. A OneIE-style β label cap was considered and dropped: β is a pruner, and the joint arm sits at P=0.61 / R=0.15, so it targets the axis already being won. Span-dimension caps already exist, and compute was never binding. On events it would bite only on list roles, where the known failure is under-generation. *Match the knob to the failure mode, not to the paper it came from.*

Phase 12 — a summarizer, tested before it was built (10 Aug, close)

Proposal: run a purpose-built summarizer to split text into self-contained bullets so number-to-place binding becomes local, with a verbatim-number guard against fabrication.

Finding the real cases killed my own framing first. I had been asserting Helene's hard case was an intra-sentential aggregate ("120 in NC, 17 in TN, 227 total"). Searching the feed found **5** multi-number casualty sentences and none of that shape — the example came from a probe, not the corpus. What is actually there: a 140-mile distance, a 30-year career, a town's 6,000 population, 30.5 centimetres of rain, the year 2004, and four deaths belonging to **Hurricane Ivan**. The failure modes are non-casualty numbers and cross-event leakage, not aggregate splitting.

The premise test said no. Raw text 3/5; free bullets 2/5 with 2 fabrications; extractive bullets 3/5 with 0. Restructuring is neutral at best. The test also caught a flaw in itself mid-run — the first version joined the bullets back into one string, rebuilding the very ambiguity the split existed to remove.

The guard and the summarizer are in direct tension, which is the durable finding. The summarizer's highest-value act is normalizing implicit prose into digits — "they died together" → "2 people died" — and that is precisely what a verbatim guard rejects. Constraining it to extractive-only resolves the conflict and costs real recall: a firefighter whose death is never quantified becomes unreachable.

And the result redirected the work. Raw extraction's one error on that sample was a *scope* error, not an attachment error — the national 225 filed under South Carolina. Second independent signal pointing at aggregate scope.

Also caught: I reported the wrong arm of my own experiment. A `vector_state_test` run looked like a new catastrophic result until the delta (+1.3379) exactly matched the number the design doc had already recorded for the *isotropic* case. The default is isotropic; the doc's table uses `--q-prop 0.15`. The doc was right and the run was mis-flagged.

Phase 13 — the attachment blocker, mostly solved (10 Aug, close)

Number-to-place attachment had been the top open item for a day, and the fix turned out to be neither a better extractor nor a bigger model.

Counting first changed the target twice. I proposed routing *unlocated* totals to `__aggregate__` and argued it should go first because it had no bootstrap dependency. It touches **4 of 106** observations. Then dumping the state streams showed what was actually in them: contamination that is **always upward**, never once downward — Florida (truth 26) receiving 300, North Carolina (truth 96) receiving 250. A one-directional error with a 2–10x separation is gateable; that is the whole reason this worked.

Judge against a larger scope, and classify three ways. Gating a stream against its own running scale fails on its early history, where a toll legitimately jumps 6 → 25 faster than any ratio tolerates. And a two-way keep/reroute split destroyed the national stream (0.402 → 2.110), because North Carolina's **1400** is not a national total — it is not a casualty count at all. Three outcomes were needed: keep, reroute, drop.

On Helene: per-state 5.247 → 0.591, and the aggregate improved too. Flat across ratios 1.5–2.5, so not a knife-edge. The control that made me believe it: removing the same 25 observations *at random* gives 4.427, so the gate is selecting rather than thinning a filter into looking better.

Then held-out testing took some of it back. On Turkiye-Syria the gate as specified **cannot run** — there is no `__aggregate__` stream to judge against. Generalizing the reference to a global maximum makes it run: Syria, 65% contaminated with Turkey's tolls, improves 3.7x; Turkey, already clean, **degrades 2.3x**, because the global max is dominated by Turkey's own values and Turkey ends up judged against a reference it defines itself. It rerouted 1,014 at t=12.5h — Turkey's true value at that moment.

The finding is sharper than the win: **the gate needs a declared scope hierarchy, not just a magnitude.** Turkey's 41,000 filed under Syria and Turkey's 41,000 filed under Turkey are identical to any ratio test. Helene has the hierarchy declared in `rollup.json`; Turkey does not. Both numbers belong in the writeup — the Helene one alone is the misleading half.

Phase 14 — the day measurement corrected four of my own claims (11 Aug)

No new capability shipped. What happened instead is that five things I had asserted were checked, and four were wrong.

"**Recall is the limit: 25 observations from 70 articles.**" That count came from the superseded `tracked_lead` run. `extract_long` had already taken the same feed to 106 observations across 44 of 70 articles — **4.2x** — and the feed contains only ~86 sentences carrying both casualty language and a digit, so the pipeline *over*-extracts. I had repeated the stale figure in PIPELINES.md that morning and built a priority list on it.

"**North Carolina's 1400 is a non-casualty number.**" It is **Hurricane Katrina's** toll quoted inside a Helene article; the 250 is a Typhoon and the 230 is Milton. That relabel has teeth: the scope gate removes them *because they are large*, not because they belong to another event, so it silently keeps Bosnia's 16 and Mexico's 2.

"**0 false positives.**" Measured on 83 Helene observations. On 250 gold training positives the same rule false-rejects **3.6%**. Small-sample luck.

"**A model cannot referee a boundary it does not know.**" The self-guide scores **82.5%** against a 25% chance baseline, on a *harder* boundary than the one that defeated it.

MHT was closed by an oracle rather than an argument. Perfect association is worth **+0.055**, and the gate already beats a perfect two-way assignment on two states because it has a third option the oracle lacks: *drop*. A subsystem competing for a 9% residual is not the next thing to build.

What survived. Type energies solve the unit-error half cleanly — 4/4 at 0/83 — and the competing-type SET is the whole design: `quantity`, described as "a count of things that are not people", is semantically adjacent rather than incompatible and takes false positives from 0% to 21.7%. *Compete against what a value cannot be, not what it resembles.* Cross-event resisted all three signals tried, because the type is right there and only the event is wrong.

Then the same discipline caught three scoping errors in the GIST work before any compute was spent: gold cannot be the guide (0.23% coverage); same-record rivals must never be vetoed because gold is authoritative there (22.2% of material); and uniform rival sampling returns noise, since a hard negative is a *near* neighbour and 12 draws from 17,131 types never contain one.

And the tests caught two silent failures in code I had written an hour earlier — swapped axes that vetoed nothing rather than erroring, and a shadowed `floor` local that disabled the abstention check. Both are pinned by guard tests now. Neither would have raised.

Nothing is wired into `model.py`; the day's output is inputs, measurements and four corrections.

Phase 15 — the metrics were lying, twice (12–14 Aug)

A phase about the instrument rather than the model. Two defects in checkpoint selection and one in scoring, all of which had been silently shaping results.

`metric_for_best` fell back to `eval_loss` when its key was absent (`c0ab89c`). The fallback swapped both the quantity *and its direction*: a run configured to maximize an F1 maximized loss instead, selecting the worst checkpoint. This is what pinned MAVEN to epoch 1 of 10. It now raises. The sibling defect in `make_sweeping_compute_metrics` — a missing key defaulting to 0.0, which scored every threshold identically and kept the first grid point — was the real cause of 15 `[eval sweep]` lines reading 0.0000 (`3d21eba`).

Fair evaluation moved to the reference tool's weights (`d6debaf`), the paper's Eq. 6/7 rather than Eq. 5. Boundary errors now earn 0.5 TP, so the weights move F1 rather than only P and R. See [[PAPER_0_FOUNDATION]] §8.

Evaluation dropped `entity_descriptions` — and the first fix was a no-op. Corpora that name their types `e_0` / `e_1` and put the meaning in a parallel map were scored by asking the model to find "e_0" with an empty description. The tell was a baseline that could not be true: pristine `fastino/gliner2-base-v1` reading 0.1351 strict entity F1 on `pile_ner_def` at recall 0.085.

The first fix (`bbacce6`) put the map under `schema["entities"]` as the values. That type-checks and reads as correct, and changes nothing the model sees — those values are label targets, while the prompt is built from `schema["entity_descriptions"]`. It was caught only because a with/without control returned **identical numbers to four decimal places**. Real fix is `7586411`; on 100 `pile_ner_def` val records, strict F1 0.0174 → **0.5381**.

Worse, the test shipped with `bbacce6` asserted the dict shape I had assumed, so it locked the defect in. Replaced with one that pushes the schema through `processor._infer_from_json` and asserts the description text reaches the prompt — *assert against the consumer, not against your model of the consumer*.

The blast radius was measured rather than assumed, and it is small. Selection was never affected: every config but the preservation one selects on `eval_loss`, which the trainer computes from the forward pass. Blind-test reach is 49.3% for `mmbert-base` and ≤0.5% everywhere else; the record-mode A/B stands at 0.2%, because `mix_natural.test.jsonl` is empty and its 35.5% description share is a val-split property. No paper number needed redoing — which is the point of measuring instead of assuming.

The synthetic-corpus arm, and a trade measured in both directions. Fine-tuning base-v1 on a 5K Haiku-generated multi-task corpus produced large in-distribution gains — event strict **0.0083 → 0.5467**, a schema the base model essentially cannot do — against a **−0.118 strict F1 (−22% relative)** loss on general-domain NER (0.5320 → 0.4136 on 6,016 records), each arm at its own swept threshold, so the gap is not a threshold artifact. At a fixed 0.5 it reads −0.121; sweeping moves it by 0.003. The check was worth running because the fine-tune's own sweep had picked 0.7 on synthetic val, so neither arm was at its optimum.

The FairEval decomposition says *what* was lost, and it is not the span machinery: boundary errors went **down** (BES 3398→2706, BEL 1725→1272) while label errors rose (LE +1302, LBE +579) and FN rose 8,977. The encoder and span head held; the labeling head reorganized onto the synthetic type vocabulary. That is catastrophic forgetting of a label distribution, not of a representation — which predicts a cheap fix (5–10% replay) rather than an expensive one. Expected in direction, and now quantified.

Also caught, before it cost a GPU run: `uv pip install -e .` resolves transformers 5.13, because the `transformers>=5.6,<5.7` pin lives in the `[local]` extra while `kernels>=0.12,<0.13` is a core

dependency. The two ranges are disjoint; kernels never hooks, mmBERT falls back to sdpa, and bf16 goes non-finite around step 50 after running 11x slow, with one `RuntimeWarning` as the only signal. This is documented in `pyproject.toml` *because it already cost the 2026-08-10 run*, and it recurred anyway on a fresh box. Install `.[local]`, and set `GLINER2_STRICT_ATTN=1` so a fallback raises.

Recurring lessons

1. **Report the baseline every time.** Run B of Turkiye reads as a success at 0.208 without `est_last_value` beside it.
2. **A silent filter is worse than a loud failure.** Three separate incidents: `is_file()` dropping event test slices, a cached fetch failure making a transient 429 permanent, and an FA2 fallback that only warned.
3. **Check the threshold before reading a curve.** It has now changed a conclusion THREE times. Promoted from lesson to standing rule: no arm or curve comparison is readable until every arm sits at its own swept threshold.
4. **Stale state reads as current state.** A frozen tracker page, a cached failure, a leftover `FAILED_1` marker firing a waiter early.
5. **A misleadingly-named corpus cost two separate diagnoses.** `text2json` supervises entities and holds the longest documents in the mix.
6. **Profile where the phenomenon lives.** CPU profiling cannot find a GPU-kernel problem.
7. **Verify the operation you care about, not the one that is easy.** A checkpoint that LOADS is not a checkpoint that runs a forward; a metric that moves is not a metric on the same test set; a schema that is valid is not a schema that can decode.
8. **Terminate the box before analysing.** Twice in one day the machine outlived its usefulness because the interesting part was what came next.
9. **A better search on a mis-specified objective is worse output.** Beam width hurt F1 monotonically while improving the score it was built to maximize.
10. **Match the knob to the failure mode, not to the paper it came from.** A OneIE β cap is a pruner; the arm that needed help was short on recall, not precision.
11. **Find the real failure cases before designing for them.** Two designs today were aimed at an example sentence that does not occur in the corpus.
12. **Check which arm you ran.** A "new" catastrophic result was the untuned default of an experiment whose tuned table was already written down.
13. **Count before designing.** Two fixes proposed as "do this first" were no-ops at 4/106 and at 3.8% of the data. One dump of the actual values redirected a day of work.
14. **A control that removes the same amount at random.** Gating improved nRMSE 9x; random removal of the same count improved it 1.2x. Without that comparison the result was indistinguishable from thinning the filter until it stopped moving.
15. **Re-check the provenance of any number you are about to build on.** "25 observations" came from a superseded run and survived into three documents before anyone asked.

16. **Compete against what a value cannot be, not what it resembles.** A semantically adjacent competitor took false positives from 0% to 21.7%; physically incompatible ones left them at 0%.
17. **A measurement that contradicts your labels is usually right.** Twice the probe sorted cases my hand labels had mis-assigned, on both the "230"s and the "two"s.
18. **Assert against the consumer, not against your model of the consumer.** The test that shipped with `bbacce6` asserted the dict shape the fix produced, so it passed while the fix did nothing. The replacement pushes the schema through the processor and checks the text reaches the prompt.
19. **Identical output from a with/without control is a result, not a coincidence.** Two eval runs agreeing to four decimal places is what exposed a fix as a no-op. Run the control even when you are confident, and *especially* when the change is one you just wrote.
20. **A hazard documented in the source is not a hazard prevented.** The transformers/kernels pin drift is commented at length in `pyproject.toml` because it cost a GPU run — and it recurred on the next fresh box, because `-e .` does not install the extra that carries the pin. Prevention needs a check that runs, not a comment that explains.

Phase 16 — the blind test was never blind (15 Aug)

Started as "separate the event loss signal", ended by invalidating the reference every other number is quoted against. Both halves are worth keeping.

The loss half. The flat `task_loss_weights` dose sweep (0.5/1.0/2.0/4.0) was null on every metric. The temptation was to conclude loss balance is not a lever here. Bucketing the loss by task instead — `probe_task_losses.py`, reconciling to ~1e-7 with a residual of exactly 0 — showed why it *had* to be null: `query_weights` reaches only start/end/pair, **18.5%** of the loss, so `w=4` moved events from 6.6% to 10.6% of the gradient while three quarters of it sat untouched. A null result from a lever that cannot reach is not evidence about the hypothesis.

I got the first version of that measurement wrong in a way worth recording: I bucketed four terms, found "events 1.6%, entities 17.2%, task-blind 76.4%", and reported it. The 76.4% was not task-blind, it was **unmeasured** — bucketing the other five terms moved events to 6.6% and entities to **77.2%**. The correction changed the recommendation, not just the decimals: the imbalance is entities dominating, not events starving.

Regime beats dose. Event positive fraction is 0.562 at convergence but **0.052** at cold-start init. `pos_weight` fixes negative-dominated imbalance; at warm start that imbalance is already gone, so `k>1` creates the opposite one. The mechanism belongs in the cold-start rebuild, which is not where I was about to spend the GPU.

The contamination half. Chasing a structure-data gap led into `convert_text2json.py`, which emits the flat key->value shape — `{"tournament_code": "ROL-2024", "winner": "Sofia Petrova"}` — as *entities*. That is a record. 97% of the corpus. Two consequences, both measured: the record head got **zero** supervision from a corpus named text2json (`json_structures` was 0.0% of the cold-start gradient), and the entity head learned **6,203** pseudo types, 731 of them appearing exactly once, from what the mix audit calls its *only* entity supervision.

Then the splits. `SplitWriter._route` drew one random **per row**, so a document emitted more than once scattered across train/val/test. text2json's val was **99.0%** contained in its train. Aggregated over a whole config, **1,080 documents — 7.03% of the blind test — were in train**. Every number anchored to the 137k reference was measured that way.

Fixes, in the order they mattered: group splits by document (and make it the **default**, because opt-in meant 20 of 21 converter write sites silently didn't); gate a config's aggregated splits with `check_leakage.py --config`; run the same gate inside `train.py` before a single step, with test authoritative and never modified. Blind-test contamination fell **2,942 -> 21** documents on the cold-start config, 299 -> 4 on warmstart.

Two of my own fixes were wrong on the first try and caught only by re-measuring: the group key wasn't normalized the way the checkers normalize (case/whitespace variants scattered anyway), and `build_warmstart_mix.py` carved its val slice positionally, generating 27 contaminated documents itself.

Lessons.

21. **A null result from a lever you have not measured the reach of is not a result.** Measure what fraction of the objective a knob can touch before concluding the knob does nothing.
22. "**Unmeasured**" **and "not attributable" look identical in a partial decomposition.** Bucketing four of nine terms produced a confident, wrong story about where the gradient goes.
23. **Correctness invariants must be defaults, not options.** Grouped splits existed as an opt-in argument for one commit; 20 of 21 call sites didn't use it.
24. **Verify the fix with the same instrument that found the bug.** Both follow-up defects were invisible to inspection and obvious to a re-run of the checker.

Same day, second half — the reach was the whole story, and then the control was

Re-running the null sweep with the reach fixed (`task_loss_weight_scope: all`, 94.3% of the loss instead of 18.5%) gave **+0.013 event strict on both doses**, above the measured floor. Entities went *up* rather than down, so it is the plan's "free win" branch, not the predicted trade; the one consistent cost is **event_type −0.019** at the higher dose, monotone in the dose. First positive result on this line, and it only appeared because the null was diagnosed rather than believed.

Then a RAMS experiment — does an intermediate `mix_natural` stage help the event downstream? — produced a wash (arguments span 0.005 across three arms) and, incidentally, **the most consequential number of the day**. Its control was the *published* 137K recipe re-run unchanged: it scored **0.2151** against the published **0.192**. +0.023 from a re-run alone.

That single number retired a claim I had made two hours earlier from the same data. I had read the 137K point (0.192, below 100K's 0.202) as "the head-init curve turns at 100K". It does not turn. −0.010 is half of one run's variance. And since every point on that curve is one seed with no measured floor, **no point-to-point difference on it is interpretable** — including the 40K→100K rise I had been quoting as a trend.

What survives is what was never marginal: boundary beats span 3.5× at 10K (0.177 vs 0.050), and the *span* curve climbs 0.108 across its range, five times the variance. The thesis — head-init scaling is a property of

the span head, not of mmBERT — is strengthened, because the boundary head turns out not to climb *at all* within measurement.

25. **A baseline you did not produce is not a baseline.** Three times today a delta dissolved once the control was run on the same code, the same data, the same day. The cost of the control is always less than the cost of the wrong conclusion.
26. **One seed is a measurement of one seed.** Every curve in this project has one point per configuration. That was fine while the effects were 3.5×; it is not fine now that they are 0.01, and the shape of a one-seed curve is not evidence of anything.

Open

- **Clean re-baseline + `scope: all` arms running** as of 15 Aug on one 2xH100: two control seeds (new noise floor on the rebuilt `mix_natural`) then `evwide2` / `evwide4`, which test whether magnitude matters once the weight's reach is fixed at 94.3%.
- **Four converters still split row-wise** (`docee`, `docfee`, `cmnee`, `mendeley_ed`) — they do not use `SplitWriter`. 21 documents of residual contamination, removed by the trainer gate every run but not yet fixed at the source.
- **`data/scaling_joint/` val files are frozen from 8 Aug**, built from the pre-fix corpora. Rebuilding them would also rebuild the j10k/j40k/j100k slices the scaling curve rests on, so it is deferred rather than done.
- **Structures are never scored by the blind test** — `_schema_from_gold` builds no schema for `json_structures`, so structure-only records are skipped (35.1% of `mix_natural`'s val).
- Replay mix for the synthetic fine-tune — 5–10% of the original labeled data, to hold the general-NER label distribution while keeping the event gain. Predicted cheap by the Phase 15 error decomposition; unrun.
- `mmbert-base`'s blind-test entity row is understated (49.3% of its test set was scored without descriptions). No live claim depends on it; recompute if one ever does.
- Venezuela 2026 — the only genuinely blind test, still unrun.
- Helene needs administrative rollup + `extract_long` before it is a usable instrument.
- The 12 HF models carry `attn_implementation: flash_attention_2`, which silently falls back to sdpa: harmless for fp32 inference, a trap for bf16 fine-tuning from them.
- Phase B joint training, still gated on Phase A being positive. Phase A ran (Phase 11) but its headline is confounded by threshold; best-vs-best is the deciding run.
- `joint_beam_width` default is still 16; the measurement says 1.
- `RequiredRoles` fill-vs-reject trap, recorded in the registry and deferred.

Phase 17 — two builds lost, and the reference set turned out to be a ghost (19–20 Aug)

Four corrections, three of them to claims made earlier in the same two days.

MHT was priced wrong, in our favour. The `+0.055` that rejected MHT came from a *two-way* oracle — each observation goes to its own place or to the national total — which has no home for a figure belonging to no scope in the event, and therefore scores Katrina's 1,400 as badly as the shipped gate does. The tell had been sitting in the results and was read as a curiosity: the gate *beats* the perfect oracle on two states, because it can *drop*. Track birth/death **is** a null hypothesis, so with a reject option the ceiling moves to **+0.111 (18.8%)** — double.

So we built the cheapest piece that delivers one, and it lost. M5 track birth by innovation gating, tracks advanced jointly in time order, no ground truth: best 0.608 against the magnitude ratio's 0.591 — and the per-place number alone was the flattering half. With the Total column restored it is 2.115 against 0.316, a 6.7× degradation of the one stream the project calls honest. It had reproduced a bug `gate()`'s three-outcome design was written to prevent. Two causes, both measured: judging a stream against its own track is **circular** (removing the self-reference is worth more than every other knob), and the innovation is uninformative about scope on a rising toll. The circularity is the same one that killed the implied-maximum reference on Türkiye — two independent mechanisms, one cause. It is also the first evidence *for* deferred assignment, which addresses it directly.

The data-side route was built, trained, and beaten by a threshold. `casualty_loc_muted` withholds an interfering event's records while keeping its text. Two arms, four epochs, one A10, ~$2.35. The suppression is real — precision up, recall down, 15 of 20 large Helene false positives removed, ungated error 46.844 → 19.822. Then a **declared per-event plausibility ceiling**, one threshold with no model behind it, recovered and exceeded the whole gain: at a ceiling of 2,000 the *control* wins both ungated (5.853 vs 6.194) and gated (3.336 vs 3.729), carrying 81 more observations. The ceiling removes only junk; muting removed real signal too.

And it fixed the class it was not built for. Chasing a 94,000 that had wrecked Tennessee found Asheville's population — emitted as `dead` *and* `injured` *and* `missing` at confidence 1.0. None of the large values was a Helene toll: populations, **FEMA flood-insurance policies** (129,933), wellness checks, power crews, troops, churches, and years read as tolls. The sharpest is a 15,000 whose sentence exists to warn against that exact error. Both genuine cross-event tolls survive muting *and* the ceiling. A correction inside the correction: troops and crews were first filed as "counts of non-people" — they are people, just not casualties, and that distinction is the point, because entity typing reaches insurance policies and stops dead at power crews.

The reference set behind all of it cannot be regenerated. `tracked_rollup.json`, written 2026-08-10 and the source of every published Helene number, reproduces from no committed state: the `--rollup` flag did not exist in any commit before the file was written, and the rollup file was not in the tree either. Both were uncommitted working-tree state, committed later in a form that does not reproduce it. Comparisons among the published figures stand — one frozen artifact — but no new model can be placed on their scale, which is what blocked the muting arm's pre-registered guard. `run_pipeline.py` now records its full

invocation and a `-dirty` git marker in every output. Second time provenance has stopped this line; Türkiye is stalled the same way.

What is actually open, unchanged by two builds: cross-event contamination. Katrina's 1,400 and Maria's 3,000 survive every mechanism tried. A ceiling low enough to catch them is just the magnitude gate again, rejecting a figure for being large rather than for belonging to another storm.

Phase 18 — the critical path turned out to be the extractor (20 Aug)

Three association mechanisms had to be built and lost before this was visible.

A router proposal that finally builds a representation. Take `min(start)..max(end)` over an event's own trigger and arguments, embed that block, match against live filters. It fixes the objection that sank clustering — it *produces* a representation instead of assuming one — and it is per-event and local, which is the discourse attachment that proximity and type both failed at. Verified on the hard case: in the Katrina passage the only named event found is `Hurricane Katrina`, so the block is Katrina-local.

It could not be run. The span architecture emits a bag of triggers all sharing one role; no threshold works, the Katrina block being either the bare name without its own 1,400 or a sweep that swallows Helene. The boundary base yields nothing above threshold 0.3 on English disaster copy and nonsense at 0.1.

The cause, and a correction to a claim made an hour earlier. A first pass read the model card and said 68% of its event supervision was Chinese. Wrong, and the truth was worse: DocEE, ChFinAnn and DocFEE are not events at all — `entities` + `classifications` — so counting them flattered both sides. Corpora that actually bind arguments to triggers give **798 English rows against 20,884 Chinese**, a 3.7% English share, with CASIE the entire English side and MAVEN and Mendeley trigger-only. An argument F1 of 0.506 is a Chinese-only number. *(0.506 is a RELAXED own-test figure — see Phase 19 for the category error this nearly caused, and for the rebuild's outcome.)*

So the front end is being rebuilt — cold start, 189,284 records, English trigger→argument 798 → ~39,800, Chinese kept because it is why the head works at all. The risk is declared up front: 72% of the new English data is synthetic, on a line whose recurring failure is in-domain-good and real-news-zero, so the gates are on AP prose.

Method notes from the day. The smoke measured 18.5 samples/s against an extrapolated 12.1, so every cost estimate before it was 34% high — and `num_workers`, the obvious suspect for 28–81% utilisation, was refuted (18.4 at 0, 18.5 at 4); the idling is variable sequence length, and memory sits at 10.6 GB of 40, so batch_size is the real untested lever. The gate harness was validated by running it against the incumbent and confirming it *fails*, because a gate that passes everything measures nothing. And rams finally got a val split, carved by document, which found 101 duplicate rows in its test set alone.

Phase 19 — the rebuild won every benchmark and appeared to lose the job (21–23 Aug; conclusion overturned in Phase 20)

The front end from Phase 18 trained cleanly and self-terminated: 35,484 steps, 6 epochs, 17.3 h, ~$35, eval loss falling every epoch 1.2672 → 0.8917 and still falling at the end. Then scoring it produced three reversals in a row.

Reversal 1: the harness had never swept anything. `frontend_gates.py` set `Schema().events(trigger_threshold=…, argument_threshold=…)` — values read only by the *span* engine (`inference/runtime.py`). The boundary greedy decode this model runs gates on the single global `extract(threshold=)` and never looks at them. So all five rows of a five-point sweep ran at the default 0.5. Proven both directions: via the Schema, thresholds 0.999 and 0.01 return byte-identical output; via the global knob, the Katrina case is empty at 0.05 and above and emits at 0.01.

The Phase 18 note above says the harness "was validated by running it against the incumbent and confirming it *fails*". That validation is exactly why the defect survived a month. The harness returned the expected answer, so nobody asked how. **A gate that fails the thing you expect it to fail is not thereby verified.**

Reversal 2: corrected, the incumbent passes the gate the rebuild fails. Over the pre-registered 0.1–0.5 range, on the same 60 Helene windows: the rebuild forms a trigger plus ≥1 bound argument on **25.0%** of them, the incumbent on **65.0%**. The claim that justified the rebuild — "the incumbent is ~0 at every threshold" — was true only at 0.5. Its real curve is 0.0 / 0.0 / 8.3 / 20.0 / 65.0 across 0.5→0.1. The Phase 18 premise survives (nothing usable at 0.3+, nonsense at 0.1); the strengthened version, which is what was actually acted on, does not.

Corrected at all three sites that carried it — harness docstring, the config's pre-registration block, and TODO — by dated correction rather than erasure.

Reversal 3: on held-out corpora the rebuild beats the incumbent on every single head. Both scored by one command on one A100, same 11 files / 15,456 rows, threshold pinned 0.5 ($1.83). entity +0.0158, relation +0.0593, classification +0.0160, structure +0.0096, event_type +0.0155, event_trigger +0.0145, event_argument +0.0133, event +0.0043; fair entity/trigger/argument +0.0278 / +0.0147 / +0.0570. Structure swept to the record head's own thresholds (0.178 max object probability, so 0.5 is unreachable): 0.1184 vs 0.1132.

So gates 1–2 FAIL and gates 3–4 PASS. Every benchmark up, the target behaviour down.

> **This reading is WRONG — see Phase 20.** Gate 1 counts firings, not correct ones. The rebuild's bindings are right 67–100% of the time against the incumbent's 0–7.7%, so the mix change worked. The paragraph is kept because how it failed is the lesson. This is the strongest case the programme has produced for pre-registering gates on the real distribution: scored the normal way, this model is an unambiguous improvement and ships.

A category error we came within one step of committing. Gate 3 was written as "event_trigger ≥ 0.710 and event_argument ≥ 0.506 (the incumbent's)". Those are **relaxed** figures from the incumbent's *own* test

set. The rebuild's strict argument F1 is 0.214 — set against 0.506 that reads as a halving, when like-for-like it is a doubling (0.1046 vs 0.0913 strict on the shared split). Fixed in the harness and the config.

Why the pre-registered remedy is declined. The config fixed "downsample `casualty_events` first" as the response to a gate-1 failure. Two measurements say it cannot work. `_decode_events` returns a single-element list per event type and pools every trigger and argument into it — its own docstring says the mention path "carries no instance dimension" — so a passage naming two hurricanes returns `n_event_instances=1` with Helene's 246 and Katrina's 1,400 both bound as `dead` on the same event, at 0.1 and at 0.01. Gate 2 takes min..max over that one instance, so it rewards sparsity rather than binding. (Not impossible: the incumbent does produce a local Katrina block at 0.01, and that passage carries one Hurricane-typed event, not two extracted ones. Fragile and threshold-lucky, which no corpus fixes.) And `casualty_events` carries 8 event types and no named identities, so it holds no same-type discrimination to teach. **The next constraint is the instance dimension — the record head — not another corpus.**

A silent-truncation bug, caught by arithmetic. Provisioning the eval box, the blind test printed "scoring against 3 files" where a count verified an hour earlier said 11. `_event_split` dropped any event file not already on disk — no fetch, no warning — while the `corpora` path fetched from the Hub. On a fresh box that silently removed every event corpus, i.e. the entire subject of gates 3–4, and reported the remainder as a completed blind test. It could only ever appear on a box, because locally those files exist. Fixed to fetch, and to name anything it still cannot resolve.

Housekeeping with a real finding in it. Eight run memories still read LIVE against zero running instances. Reconciling them turned up the 4th cell of the real-vs-synth 2×2, measured but never written down: the real+synthetic **mix preserved worst of all three arms, 0.3267, −38.6% vs base** — worse than either arm alone, so mixing did not split the difference. It had been cited second-hand as "−39%" and now has a primary source. Those JSONs lived only in gitignored `out/`; they are committed now.

Phase 20 — the gate was wrong, and the free measurement caught it (24 Aug)

Phase 19 closed with "eight benchmarks up, the target behaviour down" and a recommendation built on it. Asked to defend that recommendation, it came apart in two steps.

First, the recommendation only addressed one of the two failing gates. Gate 1 asks for a trigger plus ≥1 bound argument; one event instance satisfies it, so pooling is irrelevant to it. Gate 2 is the one pooling breaks. "The next constraint is the instance dimension" therefore answered gate 2 and left gate 1's regression unexplained — while the pre-registered remedy it declined, downsampling the synthetic corpus, was aimed precisely at gate 1.

Second, and worse, gate 1's numbers do not mean what they were read to mean. The two models' curves cross: the rebuild leads at 0.5 and 0.4, ties at 0.3, and loses only as the threshold opens up. The incumbent's 65% is entirely a low-threshold phenomenon, and gate 2 had already shown that its low-threshold bindings are nonsense. Gate 1 counts FORM.

So we measured the thing neither gate measures, free, on CPU: locate each window's gold death toll by character offset, and count a `dead` argument as a hit when its span overlaps.

threshold	rebuild fired → hit (prec)	incumbent fired → hit (prec)
0.50	3 → 3 (100%)	0 → 0
0.40	4 → 4 (100%)	0 → 0
0.30	5 → 4 (80%)	4 → 0 (0%)
0.20	6 → 4 (67%)	10 → 0 (0%)
0.10	12 → 9 (75%)	39 → 3 (7.7%)
0.05	20 → 16 (80%)	55 → 16 (29%)

The incumbent's gate-1 win is 39 firings carrying three correct death tolls. It binds `dead` to `"car Hurricane Helene"`, `"Mexico"`, `"Pacific coast"`, `"Carolinas"`. The rebuild matches or beats its yield at every matched threshold and triples it at 0.1, off a third of the firings.

So the rebuild worked. The mix change did what it was built to do; there is no real-news regression to repair; and Phase 19's headline — which had already reached the paper's abstract, TODO, MODEL_LINEAGE and the pushed model card — was wrong. Corrected in all of them.

The lesson is about the gate, not the model. A form-only criterion scored best-over-a-threshold-range rewards a model for firing indiscriminately. That is the same error MODEL_LINEAGE's matched-threshold caution exists to prevent; we had applied it to A/B arms and never to the gates those arms are judged by. It cost a day and would have cost a training run. **Every form gate needs a correctness companion before it is used to compare two models** — and the companion here was twenty minutes of local CPU, cheaper than any of the reasoning built on top of the wrong number.

Three of this programme's own instruments have now been the error rather than the model: the inert threshold sweep (Phase 19), the strict-versus-relaxed gate 3 (Phase 19), and gate 1 itself. In each case the instrument returned a plausible answer, which is why none was audited until something else contradicted it.

What still stands. Gate 2 fails for both models; the decode emits one instance per event type and pools every span into it, so the router still has no per-event input. Yield is 15% at 0.1 — the rebuild is the better extractor and not yet a sufficient one. The next constraint remains the instance dimension, now for a clean reason rather than a confused one: not "the mix failed" but "the mix succeeded and this is what is left".

Phase 21 — the process-noise question, asked properly and closed (24 Aug)

A review of the pipeline turned into a question about the filter: are the vectors normalized? Three answers, and the third is the useful one.

The shipped filter has no vectors. `est_ekf_rise` is 1-D per role. It avoids needing normalization by making every noise term relative — `R = (sig · max(ref,1))²`, growth `q_rel · max(mu,1) · dt`, init `P = (0.4·max(z,1))²` — so a 5% error on 120 and on 12 are treated alike. That is the right call and it is why the question does not arise there.

The vector arm's default was the trap this file already documents. `--q-prop` defaulted to 0, i.e. isotropic. TODO item 10 called proportional Q "a precondition, not a tuning knob", and Phase 12 records a

session mistaking the isotropic default's +1.3379 for a new catastrophic result. This session reproduced that number for the third time before noticing. The default is now 0.15 and matches every recorded table; isotropic needs `--q-prop 0`. **A documented trap that is also the default will keep being paid for.**

The real question was whether Q should be diagonal at all. Both options assert state tolls accrue independently, and the aggregate row `H = [1..1]` is exactly where that bites because `Var(sum) = Σᵢⱼ Pᵢⱼ`. If the aggregate constraint was rejected on a process model that cannot represent what the aggregate observes, that is a reason to revisit a closed question. Tested with `--q-rho`, uniform correlation with marginals preserved:

ρ	10%	35%	80%
0.0	+0.174 (4/40)	+0.057 (10/40)	−0.004 (30/40)
0.6	+0.286 (5/40)	+0.096 (3/40)	+0.016 (4/40)
0.9	+0.308 (5/40)	+0.098 (5/40)	+0.024 (2/40)

Monotonically worse. The rejection survives isotropic, proportional-diagonal and correlated Q. And the *reason* is the finding: correlation degrades parts-only as well, so these trajectories genuinely are near-independent. One storm drives all six states, yet the dynamics are dominated by the reporting and revision process — NC's 123 → 102 → 96 is a reclassification about NC — not by the physical event. Any future joint model of these streams has to start from that.

Two caveats kept on the record: uniform ρ is the crude first model, where geographic adjacency or a common-mode factor with per-state loadings would be more physical; and 40 trials over 31 real snapshots with a simulated reporting process is thin. Neither looks likely to flip a result this monotone, but neither has been ruled out.

The method note. The candidate reason came from outside the work — a standard attitude -estimation formulation, `Q_t = σ²W_tW_tᵀ`, where isotropic source noise becomes non-diagonal through the Jacobian. A closed question reopened by an analogy from another field, tested in minutes, and closed again with more support than it had before. Cheaper than leaving it shut on one process model.

Phase 22 — a closed result reopened by asking what the column was (24 Aug)

Phase 21 closed the aggregate-constraint question for the third time, having checked it against isotropic, proportional-diagonal and correlated process noise. Then: "*the vector column is that mse?*"

It is not. It is RMSE per state, divided by **that state's own range**, macro-averaged. Virginia ranges 1 → 2, so its denominator is 1 and a 1.4-death error reads as 1.4 nRMSE; the same error in North Carolina (6 → 123) reads as 0.012. Virginia carried **110.5% of the vector arm's excess error** while all five reported states improved.

Scored in absolute units, on the same runs, the verdict inverts:

density	nRMSE Δ	deaths Δ	national total RMSE Δ
10%	+0.1737	+0.35	87.6 → 28.5 (**−59.1**)
35%	+0.0568	−0.61	49.6 → 29.8 (−19.8)
80%	−0.0036	−0.69	30.9 → 24.3 (−6.6)

The aggregate improves the national total at every density, most where §6.2 said it "loses worst", and by less as parts get dense — the predicted shape.

The mechanism was right and the verdict was wrong, from the same sentence. §6.2 says an aggregate "constrains the SUM and says nothing about the SPLIT". True, and the columns are that sentence made measurable. What followed was "therefore it does not pay off" — decided by a scorer that only measures the split. The metric was never wrong about what it measured; it was answering a question nobody had asked out loud.

How close this came to standing. The conclusion had survived three deliberate robustness attacks in two days, each of which strengthened confidence in it: proportional Q, correlated Q, and a seed/noise floor. None of them could find the problem, because all three varied the *model* and none varied the *scorer*. Robustness checks that all sit downstream of the same metric cannot detect a metric error, and passing more of them feels like increasing confidence.

Two detours worth recording as errors. The real arrival pattern (NC 84%, FL 52%, VA 0%) made the vector arm look far worse — I read that as strengthening the rejection, when it was the metric artifact concentrating. `--drop-unobserved` then "rescued" it by deleting Virginia, which flips nRMSE positive and clears the floor; that treats the symptom. Both are kept because both were wrong in the same direction, and the direction was "defend the existing conclusion".

This is the fourth instrument defect in three days, after the inert threshold sweep, the strict-versus-relaxed gate 3, and gate 1 counting firings. The pattern is now explicit enough to state: **every one was found by someone asking what a number meant, and none by running more of the analysis that produced it.**

Phase 23 — the modelling finally paid, and my own instruments failed ten times (25 Aug)

The phase where the association layer got its first genuine win on both events, and where the defect count moved from "the analysis I inherited" to "the analysis I just wrote".

The supervision question closed, and not the way it was posed. The `scope` field is a zero-shot no-op, so the plan was to buy LLM labels. A $2.25 probe against Haiku 4.5 and Opus 5 on 200 records answered two questions, and the second mattered more. Haiku is not cleared — 83.0% agreement but **kappa 0.121**, which is the number that matters when 86% of a corpus is one class. Then hand-adjudicating fourteen place↔sub-place disagreements came out **7–7**. Neither model is the variable: `sub-place` conflates a genuinely narrower counting unit ("three districts IN Iraq") with a narrower incident SITE whose number is still the bound place's full toll ("across 2 districts OF Baghdad"). Case two mislabelled makes `apply_extracted_scope` **drop a valid observation**. Independently, `national` is **0.0%** of the corpus on

both strata — so `casualty_events` cannot teach the one distinction the ratio gate exists to arbitrate, and no labeller upgrade fixes that. The $11 batch is not worth buying under this scheme.

A cheap idea priced and killed before it cost anything. The DETR reading suggested span-GIoU. Soft-IoU turned out to already exist and be on by default (`losses.py:288–311` , weight 0.2, annealed), so it was a four-line delta — and the wrong one. 71.9% of candidate pairs score IoU exactly 0, but only 8.3% of those sit within three tokens of gold, so the near-misses GIoU would newly distinguish are **5.9% of candidates**. Meanwhile the BCE consumer needs targets in [0,1] while GIoU is [-1,1], and the rescale moves **66.6% of all candidates** off zero — 28% of the zero-IoU ones would get a target above 0.25 despite no overlap with gold. An 11× blast radius for a 6% signal, in a model whose binding constraint is precision. Rejected on a free measurement rather than a $35 run.

The reject headroom is the opposite shape to the one the gate was built for. Helene's three-way oracle reaches 17.6 against the shipped gate's 29.3, and it gets there purely by dropping more. Restricted to the gated population — the two-way oracle keeps everything else by construction, which was handing a `gated` feature AUC 0.763 for free — the rejects have **median value 6.0 against the kept set's 99.0**. Split by direction: **63% stale-LOW, 20% over, 17% no truth coverage**. The gate rejects upward only. It is blind to two thirds of its own prize by construction.

Five mechanisms built and tested against both events.

#	mechanism	verdict
1	Student-t measurement model	Helene −1.7, Turkiye +651. Retires no knob. OFF
2	**Viterbi decode, 3 states with reject**	**Helene −29.4%, Turkiye −9.8%. SHIP**
3	IMM / PDA soft association	loses to hard decode on both. Not adopted
4	4th state for downward revision	correct, inert. Not shipped
5	document structure → HMM emission collapse	false rejections **19.8% → 9.9%**

The through-line: every winner was a **hard, global, one-sided** decision. Soft assignment lost because the measured headroom is in removal, not down-weighting — and because PDA arbitrates *independent* targets while ours are nested, so an ambiguous reading gets soft-assigned to both the part and the whole (106 observations became 118 assignments). One-sidedness mattered in three of the five; the symmetric textbook forms were measurably worse every time.

The result that nearly did not happen. Viterbi first read 1-for-2 — good on Helene, worse on Turkiye — which would have been the fourth intervention in a row with that shape. I proposed the divergence was structural: Turkiye's `global-max` reference is self-referential for its dominant stream. **The per-stream diagnostic refuted it**, and in the opposite direction: Viterbi helped the self-referenced dominant stream (0.403 → 0.377) and hurt the *minor* one (0.923 → 1.858). Chasing that inversion found the real cause. `warmup=2` , copied from the greedy gate, pins the first readings to `own` — and Syria's first two are contaminating Turkiye figures (9057, 17674 against a true peak of 5800). It poisoned the level and the genuine 3317s were then rejected as stale. That is precisely the greedy commitment the global decode exists to remove, smuggled back in as a knob. With `warmup=0` , Turkiye goes 12306 → 10441 and the result is 2-for-2.

What Turkiye actually cannot test. Its contaminant is the 1999 İzmit toll of 17,500 against a 50,000-death event — the same order of magnitude, and it **crosses** the true trajectory (truth 2,316 at t=23h, 17,674 at t=96h, 31,643 at t=194h). Rejecting it costs more coverage than the impurity costs accuracy: forcing all 16 copies out takes pooled 10695 → 15763. Helene's contaminants differ by 6× (Katrina's 1,400 against ~230). **Scale separation between an event and its contaminants is what decides whether cross-event rejection can pay for itself**, and it is why a third event is needed rather than more tuning.

One decode absorbing three gates. Gates [5] date and [6] scope are complementary — [5] catches the 1916 hurricanes' "80" keyed to North Carolina, which scope membership structurally cannot, since North Carolina is in scope — but the series is a bad trade: [5] adds **+1 catch for +10 false rejections**, because a hard `continue` cannot be outvoted. Folded into the emission as additive log-odds they argue instead of vetoing: **5/6 at 19.8% false becomes 4/6 at 9.9%**, at no trajectory cost. The weights are hand-set, not fitted — composition of knowledge we already have, which is what keeps it consistent with the earlier finding that a learned cross-event signal loses to declared scope membership 31.7% to 7.4%.

Ten defects, all mine. Whole-record instead of passage windowing (kappa 0.247 vs 0.469); `str.find` matching the "6" inside "2026" (22% of figures); location-less figures asked for a relative judgment (18% of disagreements); an emission scoring 83%-of-total as a perfect part; the warmup above; `revise_cost < reject_cost` inverting the revision logic; `out_of_window` structurally unable to fire on a feed whose events block has no `date` key; a control run at warmup 8 against a true control of 2; a `gated` feature separating for free; and a fabricated "145 assignments" in a commit message, amended.

Phase 22 ended by noting that every defect so far was found by someone asking what a number meant. These were found differently, and the mechanisms are worth naming because they are repeatable: **comparing a new measurement against a published one** (gate 6 reproducing its own docstring's 7.3% only after the scope set came from `rollup.json`), **a smoke test disagreeing with stated intent** (500 against a national 600 decoding as `own`), **an arm beating a control that looked wrong**, and **a result inverting where the hypothesis said it would not**. Two conclusions were proposed and retracted — the structural divergence, and the revision state being silenced by the noise floor, refuted at 4.2σ.

Phase 24 — the third event, and a pre-registered prediction that failed (25 Aug)

Phase 23 ended with the association layer resolved and a third event pre-registered to test the one thing two events could not. It was built, it ran, and the prediction that justified building it is false.

The decode won a third time, and by the widest margin yet. On the 2020 Aegean earthquake the shipped gate scores 74.38 pooled deaths and is *inert* — 74.38 at ratio off, 4.0, 3.0, 2.0 and 1.5 alike. The Viterbi decode scores **15.74, −79%**. Across three events now: Helene −29.4%, Türkiye −9.8%, Aegean −79%.

And the collapse showed nothing, for a reason that was not the hypothesis. The event was chosen for scale separation: 119 deaths against an İzmit 1999 reference of ~17,000, 143× where Helene's contaminants differ by 6× and Türkiye's by 3×-and-crossing. The feed delivered exactly that — 17,000 ×2, 17,800, 220,000 ×2 and 2,679 against a true peak of 117. The emission features catch them (drops 15 → 21, three of six rejected at weight 3) and pooled RMSE does not move: **15.74 at every weight from 0 to 3**.

The contaminants are keyed `unknown` and `marmara`, never `Izmir`, and only **12 of 53** observations are bound to a gated place at all. **Association isolates the contamination before the gate ever sees it.** Helene has the same shape — its cross-event figures live in mexico / puerto rico / bosnia / reading-pennsylvania streams, none of them scored. Two events, same structure, and it means the collapse cannot be validated on pooled RMSE on either. Its effect is real but only visible on cross-event catch and false-reject rates, where Helene went 5/6 at 19.8% false to 4/6 at 9.9%. **A fourth event would not fix this**, which is the useful part of the negative: the limit is structural, not a sampling problem.

Where the errors were this time. Three in the harness, all mine, and the third is the one worth remembering. The emission treated a *missing* reference as evidence a reading was too large — this feed has no aggregate stream, so `natl=0` and every value scored above the whole event, dropping **52 of 53 observations at zero feature weight**. The dataset was registered against an `aggregate` reference that does not exist here. And `key_of` lowercased the place while this rollup capitalises canonical names, so `score()` matched nothing and every arm read `nan` — **including the controls, which is what made it visible**. A defect that breaks the arm alone is dangerous; one that breaks the control too announces itself.

A fourth, in the pipeline rather than the harness: `out_of_window` missed the 17,000 completely, because nearest-date-by-character-distance picked **2020 at +117 chars over 1999 at −152**. The function's own docstring defends that proxy on competing dates being "far apart rather than adjacent". 117 against 152 is not far apart, and the justification had never been checked on a feed that violates it.

Two findings from the same day that belong here. First, the relevance gate does not filter Turkish at all: the shipped model admits **199 of 200** clean non-disaster articles, because it is DeBERTa-v3 with a 128k English vocabulary and cannot read the text. That is worse than the 58.5% failure that forced the v1→v2 rewrite, and it is silent — nothing reports that the gate has stopped discriminating. The multilingual model cuts it to 28%, and translating the articles into English does *not* help (17/60 still admitted by the English model on English text), so it is the label descriptions and not the language.

Second, and prompted by a question rather than a measurement: **rescued is not injured.** Twelve of Helene's 106 `dead` observations are audited non-casualty, and six are exposure counts — 300 rescued, 50 patients rescued, 32 evacuated, 11 swept away. A rescued person is a counterfactual casualty. The other six are unit confusion: a two-day period, six states, dozens of vehicles, 1,400 landslides. The root cause is the schema — `casualty_events` has `location`, `injured`, `missing`, `dead` and no role for exposure, while the prose is full of it. **A number with no correct home lands in a wrong one.** That is a data-modelling fix, not another gate, and it is the same diagnosis as item 10a reached for `scope`.

Phase 25 — the operating point was the finding (27–28 Aug)

The phase began by finishing a measurement a reboot had killed: a per-class comparison of the two gate models on identical rows. It ended with three published conclusions overturned, none of them by a new model.

The four-way rebuild bought nothing. Its GPU run had already come back plateaued at epoch 3 with the binary task flat (relevance 0.8341 against 0.8368), and the per-class comparison at first looked like a

rescue: stratified toward the hard classes, v2 won 37 rows to 17, exact McNemar $p = 0.0091$. That result did not survive its own follow-up. Read at each model's own validation-chosen threshold the two models are **18 to 18, $p = 1.0000$**. The significant-looking win was the distance between two operating points that a shared argmax placed differently — the programme's own §5 rule, broken while writing a new rule into TRAINING.md about paired testing.

The threshold was worth more than either training run. The gate's softmax is saturated; it needs 0.998 to sit at the stated recall bar, and 0.5 sits deep inside its positive region. Choosing on validation and scoring the blind test once: overall 0.719 → 0.847, `exposure_only` 0.444 → 0.903, 108 rows fixed against 39 broken. `exposure_only` was the open failure two GPU runs and a bought four-way label had been aimed at. It was never a capability gap.

The gate switch does not survive a sweep. `fastino/gliner2-base-v1`, replaced in b607fae on the strength of false positives at one threshold, leads on AUC (0.9635 against the incumbent's 0.9241) and on recall at every operating point. The FP half of the original comparison reproduces; the conclusion does not, because there was no recall column.

Turkish: not multilingual, and the corpus says why. AUC 0.4733 — below chance, with the negatives' median above the positives'. The corpus is 95.9% English, 4.1% Chinese, 0.19% incidental Turkish; the encoder is multilingual and the training signal never was. Two things followed. A stage-0 language gate now rejects what the gate cannot read (1,441/1,441 in-corpus rows supported, 0/300 Turkish admitted). And translation, which a stored ablation said would not help, moved AUC to **0.8359** — that ablation was right about the model it ran on, which reads Turkish and over-admits, and wrong about ours, which cannot read it. **A stored verdict is valid only for the model it was measured on.** A $0.72 adjudication pilot then found 43.3% positives in Turkish news, so a corpus is not positive-limited.

Two instruments failed, one of them mine, on the same day. `benchmark_gate.py --sweep` derived P(mass_casualty) as `1 - confidence`, which turned every message the new language gate had dropped at confidence 0.0 into a maximum-confidence false positive. It put an identical 13-message floor under three different architectures — the tell, because three models do not agree to that precision — and briefly produced a table showing a model winning for the wrong reason. The lesson generalises past this bug: **when you add a sentinel value to a decision path, audit every consumer that does arithmetic on the score.** The language gate and the sweep were written the same day, by the same author. `annotate_gate.py` had the other one: it iterated a hardcoded six-stratum list, so a corpus whose stratum was not in it would be read, counted, and silently dropped — Turkish would have annotated zero documents and printed a plausible log.

M4 was closed, and two documents still argued for it. `PIPELINES.md` §4.1 and the `mht_associate.py` docstring both end with "this strengthens rather than kills the case for M4", written 2026-08-19. The global decode (M4′) shipped six days later and takes most of the +0.111 without a hypothesis tree. Both now carry dated supersession notes; neither paragraph was deleted, because the diagnosis in them is still right and only the remedy changed. M2 — the cost matrix — was found to have no row in the divergence table at all, so "never built" was inferable only from absence.

The through-line of the phase is that nothing here required a new model, a new corpus, or a GPU. Every result came from measuring something already shipped at more than one operating point.

Phase 26 — a third language, and proving the extractor could not read it (29–31 Aug)

Turkish was added to the gate, and the obvious next question — can the extractor read it? — was answered before any money was spent on the assumption that it could. **It could not, and the proof needed an English control to be worth anything.** On Turkish documents the extractor returned a digit in the `location` field 78.2% of the time against 5.8% on English (p = 1.5e-39): not a lower score, a different failure. It was confidently wrong. That result is what made a Turkish extractor *purchase* necessary rather than optional, and it is also why a Turkish gate alone would have been net-negative — a gate that admits documents an extractor cannot read moves work downstream, not accuracy.

Annotation was priced from a labelled sample of the pool rather than from a similar corpus, and that method caught two errors that would have been paid for. The first price used a 42.3% positive rate measured on a single-outlet pilot; the multi-outlet pool is 25.1%, which alone moved 30,000 positives from $95 to $160. The second was feasibility rather than price: a high purity cut can be unreachable, because purity rises while recall falls and the surviving positives may be fewer than the target at any price. A free regex composed with the model beat the model alone (78.8% vs 65.3%) and halved the scoring cost.

The same window brought a Simplified Chinese pool, `zh_multitask`, and the 137k base's **label-space unification** — English labels, native spans. Two traps surfaced there and both are now rules: a map key containing the roll-up separator is dead because roll-up runs first, and an empty inline `labels:` block silently overrides `labels_file`.

Phase 27 — the viewer had been showing nothing (30 Aug)

Structures silently returned nothing on every boundary model, the three real event feeds were never selectable, and ground truth never loaded for any of them. None of this was a model defect; the pipeline behind the demo worked. It is recorded here because the failure mode is the one this project keeps meeting from a different direction: *a component that returns empty looks identical to a component that found nothing.*

Phase 28 — measure the base before buying supervision (1–3 Sep)

`data/` was restored from the Hub, which exposed the backup gaps rather than closing them, and `run_all_converters.sh` was found to rebuild the Chinese labels it was supposed to remove. Two measurement disciplines were established before any purchase: **per-language and per-domain perplexity on the base**, and a **blind test that is actually blind** — `casualty_ml` became three real-news arms with the test set held out by construction rather than by intention.

Domain-prompted synthetic buys the register, not the distribution. That is the phase's finding and it prefigures Phase 30's: a synthetic corpus can match the surface style of a domain while missing what makes real documents hard.

Phase 29 — gate3 passed, then failed, then had its verdict withdrawn (4 Sep)

Gate3 passed both admission bars and dominated both predecessors. Then it **failed a third bar** — it fixed Chinese by taxing English. Then the English regression **did not replicate** and the verdict was withdrawn. Three states in one day for one model, and the third is the one that matters: a single-run regression on a metric with a ±0.02 floor is not a finding.

`gate3-mixed` was rejected on a distinction worth keeping: **it wins both F1 numbers and loses the job.** The Turkish event corpus completed the same day — type and per-type roles, bought in two conditioned passes and joined into DocEE's shape. And a defect that had been live the whole time was found: **the length proxy was blind to scripts without whitespace**, so `len(text.split())` returned ~1 for Chinese and sorted nearly half a corpus into one length bucket.

Phase 30 — the instruments, not the experiments (5–7 Sep)

The phase's through-line is that almost nothing here was a failed experiment. **The experiments ran; the instruments were wrong**, and every finding came from catching one.

Warm starts were silently ignoring configuration. A stage-1 run died in 22 seconds because `boundary_head` keys cannot be set on a warm start — and the fix exposed that **eight parameter-changing flags had been unguarded**, then **six more that a key-set diff cannot see because they resize rather than add**. An unguarded flag is worse than a refused one: it lands on the saved config, so the run trains one architecture while writing a config describing another.

60,948 structure records had been training nothing, silently, across 13 models. The record head cannot decode a structure without `record_metadata`, and its absence is valid and undetectable — no error, no warning, a row that simply teaches nothing. Repaired across 12 corpora, with an auditor to keep them that way. The repair was then **priced**: re-running the real-vs-synthetic arm on repaired data moved structure F1 **0.2179 → 0.3999** and moved *nothing else* — the specificity is the evidence.

Four measurement defects were found in the eval path itself, each of which had been producing numbers: `eval.py` **scored a different test set than training's blind test** (29,615 records against 18,786, because 9 of 13 corpora are listed in both `corpora:` and `event_files:`); a corpus with no such split was **fatal** rather than named; an optional dependency was imported **after** training rather than before, costing a 3h34m run its blind test; and a single transient 400 on the Hub push **destroyed a completed run** because `upload_folder` had no retry.

Catastrophic forgetting was measured directly for the first time. Three no-replay warm cells lost on *all eight heads* of the base's own test set — classification −0.403 (−69%), entity −0.178 (−31%) — for +0.024 on the task they trained. Three seeds agreed tightly. The −31% lands inside the 23/32/39% band measured three weeks earlier on a different base and task family, and *Pioneer Agent* (arXiv 2604.09791, from the GLiNER lineage) independently reports naive retraining degrading by up to 43 points.

Composed supervision passed its pre-registered bar by 4.5×. Adding four already-owned corpora that carry 3+ tasks per document — train-only, so the test split stayed byte-identical — moved structure **+0.089** against a +0.02 bar, classification **+0.261**, relation +0.064, event_type +0.066, at a cost of −0.034 on

event arguments. The prediction that classification would *regress* (54.5% of the added supervision is near-constant) was wrong, and interestingly so.

Label-space collapse was separated into three failures that share a name, and measuring our own data took three passes to get right — counting the label menu instead of the answer, then measuring fallback share when the real failure was majority-class concentration, then discovering that three of the four "annotated" corpora were **written and labelled in one call** and so measure a generator's preference rather than an annotator's judgement. What survives is one genuine annotation sample. `uncertain_types` was added so an annotator can record doubt instead of resolving it, because all three alternatives — catch-all, guess, omit — inject *systematic* bias: median type purity on repeated surfaces is 100%, so a model does not flip a coin, it applies the same prior every time.

The phase closes on the programme's own central question, and on a fifth naming collision. `--global-decode` was measured and found neutral — but it is the **cross-window event merge**, operating on results already decoded, not the typed-constraint beam over candidate scores that the thesis describes. The real switch, `boundary_head.decode_mode: greedy|joint`, was **built and unreachable from eval**: `from_pretrained` constructs settings from the checkpoint's config, and `evaluate_checkpoint` never applied a config's `boundary_head` over it. Wired, smoke-tested, and run.

Phase 31 — the programme's central question, answered, and two models lost (7 Sep)

The greedy-vs-beam comparison finally ran, and it took three attempts to aim it. The first measured `--global-decode` — the cross-window event *merge*, which operates on results already decoded — and found it neutral. That is a real negative for the merge layer but it is not the thesis. The switch the thesis needs is `boundary_head.decode_mode`, and `gliner2/joint_ie/` turned out to be a complete stack: JointProblem, candidate_scores, constraints, lattice, optimizers. **The mechanism was built and simply unreachable from eval** — `from_pretrained` constructs settings from the checkpoint's config and `evaluate_checkpoint` never applied a config's `boundary_head` over it, so `decode_mode` could be set in a YAML and silently ignored.

Wired, and the answer is **no**: six heads inside the noise floor, structure −0.0454, at ~2.5× the wall clock. A 16× beam-width sweep moves structure by 0.0018, and narrower is marginally better — the opposite of a search-capacity story.

The first version of that result was wrong, and the operator's debugging method is what caught it. "Watch the data at every step, not just the sections you think are critical." Following that literally on one document showed the joint arm returning `HD-2024-0001` and `2024-03-18` — the same two fields greedy found — and being scored zero for both. Three steps, only the last where anyone would look: the two paths compile one schema into two **cardinalities**; so one emits `{"text": ...}` and the other `[{"text": ...}]`; and `_pred_structure_set` had a dict branch, a str branch, and **no else**. Lists fell through in silence. That alone was 48% of the apparent collapse (0.0343 → 0.0754), and the confident mechanical story already written down — "the model was never trained to emit structures as role edges" — was wrong twice over.

Eval throughput was profiled, and the profile lied about the hardware that matters. `keep.nonzero` forces a device sync, 32% of wall time on MPS, one call per batch. Deferring it across batches gives **2× on MPS** and **1.4% on CUDA** — the launch queue already hides it. The accumulation window is correct, output-neutral and DDP-safe, and buys nothing on the fleet. `eval.batch_size: 8` buys 14% and is the only real speedup; past 8 the padding cost overtakes it. Profiling on the available accelerator produced a confident, well-evidenced, wrong conclusion about the target one.

Two models were lost, the second after the fix for the first. The realsynth re-run went to a transient 400 with no retry. Phase 0's `eb16-composed` went to something the retry could not see: `upload_folder` **returned without raising and wrote nothing**. The runner printed PUSH OK, the trap terminated the box, and the repo holds one file — `.gitattributes` — against ~15 hours of A100. Its *metrics* survived because they upload before the push, so the finding stands (structure +0.0890, classification +0.2611) while the weights do not. `push_to_hub.py` now verifies the files are really on the Hub instead of trusting a clean return, and `save_or_die.py` rescues weights into a repo that works when the model repo will not.

The through-line of the whole day was instruments, not experiments. The experiments ran. What failed, repeatedly, was the measuring: a scorer that dropped a shape, a flag that measured the wrong mechanism, a profile from the wrong accelerator, a guard that asserted exact float equality across hardware, a verdict that reported "scores differ" when the run had crashed, and a push that reported success while writing nothing.

Phase 32 — the headline result was a parameter nobody passed (8 Sep)

Phase 31's one finding outside the noise floor is withdrawn. "Joint decode costs structure, −0.0454" was 87% an artefact. `compile_record_specs` had always accepted `field_dtypes`, `_default_cardinality` had always mapped `dtype: "str"` to a scalar cardinality, and `build_boundary_batch_metadata` had always declared `field_dtypes_list` — **which no caller ever passed.** Every non-anchor field compiled `ZERO_OR_MORE` whatever its schema said.

That is not a formatting flag. Cardinality selects the training loss, the occurrence policy, the joint beam's utility (`logit − absent` against bare `logit`) and its exclusivity slot — so **the beam's entire scalar machinery had never engaged for a structure field.** Greedy looked nearly right because it re-derived the dtype at format time; the joint arm had no such rescue, and the gap between them *was* the rescue. Connected: greedy 0.1208 → 0.1640, joint 0.0754 → **0.1582**, deficit −0.0454 → −0.0057. The joint arm gained nearly twice what greedy did.

The verdict survives; its content changes. Joint decoding now matches greedy on all seven heads and costs ~2.5× the wall clock. A null at parity is a *better* negative than a loss, because a loss can be explained away as a defect — and this one was exactly that.

Three instrument failures, in series, each hiding the next. The scorer dropped list-shaped output in silence (48% of the apparent collapse, caught 7 Sep by watching the data at every step rather than at the step that looked interesting). Then the fix's own first re-baseline **measured nothing**: the schema carrier was dropped one layer above, in a branch whose own comment warns about that exact defect for `record_metadata`, and the blind test declares every structure field `dtype: "str"` — so the fix was inert

precisely where it mattered, and the joint arm returned byte-identical. Nothing in the log said whether the fix had run, so its execution had to be *inferred from the numbers it was supposed to move*. That is the wrong direction, and it cost a GPU run.

The fix for that is an instrument, not a patch. Runs now print `[records] compiled N field spec(s): X scalar, Y list; Z of W schema(s) declared field dtypes` before scoring, and the third box was **gated on that line** — read first, metrics second. It printed `6 scalar, 0 list; 2 of 2` and only then was the table read.

Mechanism-or-model was also settled, and the pre-registered floor rule earned its keep. The pre-fix deficit reproduced on an independent lineage (`joint-boundary-mmbert-137k-clean`, −0.0529), so it was never one checkpoint's property. The third pair, `casualty-multilingual-eb16tr`, was declared **unreadable** by a rule written before the numbers existed: greedy structure 0.0070, a sixth of the floor, with

> **SUPERSEDED 2026-09-15.** These figures are of uncertain provenance and are NOT at threshold 0.5 — the incumbent's model card says `Decision threshold: 0.3`. Measured at the validation-selected 0.2: event_argument strict 0.0991 / relaxed 0.5884, event_trigger 0.5984, event_type 0.8650. See EVENT_ARGUMENT_DIAGNOSIS.md §4c. entity 0.0801 and event_type 0.0209 against the base's 0.5695 and 0.7545 — catastrophic forgetting, exactly the shape the gate3 warm cells measured. Running only the checkpoint the resume list named would have returned nothing at all.

Elsewhere the same day: the generation A/B was regenerated from scratch after its only copy was lost with an unmountable disk — recovered not from memory but from committed code, since `--write-only`'s docstring *is* the hypothesis and `--estimate` pinned the model by price. Two-stage wins **8 of 12 tasks** (mean TVD 0.2661 against 0.3032), and `sentiment` shows the mechanism directly: one-call writes *positive 53%* to fit labels it chose, while two-stage lands *neutral 62%* against real news's 49%. The scorer that was lost with the disk is now committed, and TVD is defined in `METRICS.md`.

Retired working documents

Each of these had a job and finished it. What they concluded is recorded here; what is still load-bearing was moved into the paper that needs it; the files were removed. **Full original text is in git** — retrieve any of them with `git log --diff-filter=D --name-only` and `git show <commit>^:tools/events_working_papers/<file>`.

Retired 2026-09-07.

Executed plans

`DOCUMENT_EXTRACTION_PLAN.md` (212 lines) — spec for OneIE-style global graph decoding over windowed candidates. Built; reported in `PAPER_0_FOUNDATION.md` §9. Its problem verification (2026-07-17) and build increments are all spent.

`SCALING_CURVE_EXPERIMENT.md` (174) — spec for the mmBERT head-init data-scaling curve. Ran; the measured curve is `PAPER_0_FOUNDATION.md` §10.7 (10K does nothing, 40K lifts arguments ~2.3×, still

climbing at 100K with no plateau, knee between 10K and 40K).

`HEAD_INIT_DATA_SCALE.md` (89) — a *reasoned bracket* for how much data warms a head, written before the curve existed. Entirely replaced by the measurement above. Kept in mind as a method note: the estimate was made explicit and then falsified by measurement, which is the intended lifecycle for an estimate.

`EVALUATION_PLAN.md` (161) — four-step per-language blind-test plan, every step marked `[DONE]`. Executed; reported in `PAPER_0_FOUNDATION.md` §8.

`MAIN_MERGE_CONFLICT_MAP.md` (135) — pre-merge conflict scouting across 17 overlapping files, for a merge that has since been executed (this branch *is* `merge/main-20260805`). Self-marked "regenerate before executing"; nothing left to regenerate.

`CORE_CHANGES.md` (412) — diff analysis of `mmbert_training` against `main`, for the same completed merge. Its refactor recommendations (272–412) were never executed and are written against a branch state that no longer exists.

Abandoned or superseded plans

`EVENT_LOSS_PLAN.md` (357) — per-task event loss on the **span** architecture. Dropped in the port to the boundary rewrite and never trained with. Its successor already declared it "kept as history".

`EVENT_LOSS_PHASE3_PLAN.md` (302) — the boundary-architecture analogue, plus an appended result. The finding never graduated to a paper, and every number in it predates the split repair.

`RECOMMENDATIONS.md` (118) — five options for argument↔entity linking, coref and doc-level events, written against the span architecture before the pivot. Option 5 became `DOCUMENT_EXTRACTION_PLAN.md` and then `PAPER_0` §9; the header's claim that "every option below is still unimplemented" stopped being true in August.

`KALMAN_BEAM_SEARCH_EXPLORATION.md` (274) — the origin analysis that noticed beam search and MHT are the same top-K hypothesis inference at different scopes. **That observation is the programme's thesis and survives in `RESEARCH_PROGRAM.md` §1.** The document itself was written against the span `global_decode.py`, which the boundary pivot replaced, and `BOUNDARY_DECODE_AND_EKF.md` already stated it supersedes the boundary-relevant parts.

Superseded vision documents

`EKF_PIPELINE_VISION.md` (371) and **`EKF_PIPELINE_VISION_REVIEW.md`** (208) — an ASCII-art vision of the full pipeline and a critique of it. Both absorbed by `PIPELINES.md` §3 (as-designed) and `GATES.md`, and cited by nothing else in the canon. The vision's "BLOCKING PREREQUISITE: no current model can produce the input" was contradicted by the front-end rebuild; the review's argument that the front end was mis-thresholded and used the wrong model was correct at the time, acted on, and is now history.

Scratch

`DEFINITIONS.md` (19) — two unrelated pasted chatbot answers on "IoU" and "dose curve". Not project content; deleted outright rather than retired.

`EVALUATION.md` (11) and `INSTRUCTIONS.md` (8) — the raw prompts that produced `EVALUATION_PLAN.md` and the project itself. Kept as seeds until now; both are wholly overtaken by what they produced.

`HMM_TITANS_MIRAS.md` lines 1–113 — pasted chatbot output on HMM × Titans/MIRAS routing, self-labelled as such. **The document's own assessment (114–285) survives as a paper**, and it rebuts the premise of the pasted section: the two-sided emissions idea already exists in this codebase, already ran, and is inert.

Closed items from the resume list

Nineteen items retired from `TODO.md` on 2026-09-07, kept because each records **what was measured and what it cost to find out** — several are negatives that still scope a current claim, and the resume list is meant to be short enough to read.

Three had headings that still read OPEN and were verified closed before moving:

- **0.1 — cc_news/synthetic converters emit `json_structures` with no `record_metadata`.** Repaired 2026-09-06 across 12 corpora; `cc_news_haiku45` measures 1,033/1,033 decodable. The defect had cost 60,948 structure records across 13 models, silently.
- **-1 — `eval_metrics.py` cannot score `json_structures`.** It scores them: the Phase 0 metrics file carries `structure` F1 0.2098 on support 4,167.
- **15 — the relevance gate does not filter non-English.** Shipped in `a5fc6a0`, which rejects unreadable languages *before* the model runs rather than guessing at them.

The items follow verbatim, in the order they were filed.

FRONT-END REBUILD — DONE AND NEGATIVE — `ekf-frontend-mmbert` (run 2026-08-20/21, scored 2026-08-23)

The EKF pipeline's router work is blocked on an extractor that emits, per event, a trigger and arguments bound to *that* trigger. Nothing in the line does.

Span arch emits a bag of triggers, all sharing one role; no threshold fixes it — the Katrina block is either the bare name (missing its own 1,400) or swallows Helene. **Boundary `137k-clean`** yields nothing above threshold 0.3 on English disaster copy and nonsense at 0.1, despite trigger 0.710 / argument 0.506 on its own test set. *(0.710 / 0.506 are RELAXED own-test numbers — strict on the shared split is 0.7487 / 0.0913. And "nothing usable" holds at 0.3+ but NOT below: the corrected curve is 0.0 / 0.0 / 8.3 / 20.0 / 65.0% across 0.5→0.1. See the scored result further down.)*

The arithmetic behind it. Counting only corpora that bind arguments to a trigger — DocEE, ChFinAnn and DocFEE do *not*, they are `entities` + `classifications`:

	English	Chinese	English share
`137k-clean` as built	**798** (CASIE alone)	20,884	**3.7%**
every available corpus	39,783	20,884	65.6%

MAVEN and Mendeley are trigger-only. So argument F1 0.506 is very nearly a Chinese-only number and English trigger→argument rests on 798 examples.

`tools/train/config/casualty/ekf-frontend-mmbert.yaml` — cold start, 189,284 records, 50× the English trigger→argument supervision, Chinese kept. Split gate CLEAN (180,660 / 11,486 / 20,571). `rams` gained val+test by carving its 871-row test **by document**, which found 101 duplicate rows in test alone — the same hazard this file records for rams train.

Declared risk: 72% of the new English trigger+argument data is synthetic against 20% human-annotated real news, on a line whose recurring failure is in-domain-good / real-news-zero. Gates are on AP prose for that reason, not held-out DocEE.

Smoke (1× A100-40GB): 18.4 samples/s, 34% faster than the extrapolation every earlier cost estimate used. `num_workers` is NOT the bottleneck — 18.4 at 0 workers, 18.5 at 4 — so do not re-try it; utilisation swings 28–81% on variable sequence length while memory stays flat at 10.6 GB of 40. **batch_size is the untested lever.**

RUN DONE (2026-08-21), GATES FAIL, AND THE HARNESS THAT JUSTIFIED IT WAS BROKEN.

Smoke on 1x A100-40GB passed at ~$1.70: cu128 + FA2 clean, **18.5 samples/s** against an extrapolated 12.1 -- every cost figure produced before it was ~34% high. `num_workers` is NOT the bottleneck (18.4 at 0, 18.5 at 4); memory flat at 10.6 GB of 40, so **batch_size is the untested lever**. Logs in `tools/train/smoke_results/ekf-frontend-mmbert/`.

The full 6-epoch run completed autonomously and self-terminated: 35,484 steps, 62,249 s (17.3 h), 18.24 samples/s, ~$35, eval_loss falling every epoch 1.2672 -> 0.8917 (still improving at 6 -- no plateau). Model + artifacts on `whr778/gliner2-ekf-frontend-mmbert`.

Gate results, both models re-measured on the fixed harness (registered range 0.1-0.5; `tools/ekf_showcase/frontend_gate_results/`):

gate	rebuild `ekf-frontend`	incumbent `137k-clean`
1 -- trigger + >=1 bound arg on >=50% of Helene windows (FORM only)	**FAIL** 25.0% @ 0.1	"**PASS**" 65.0% @ 0.1
1b -- of those, the toll is CORRECT (added 2026-08-24)	**75%** @ 0.1	**7.7%** @ 0.1
2 -- Katrina block holds "1,400", not "Helene"	**FAIL**	**FAIL** (local only at 0.01)

CORRECTED 2026-08-24 -- the rebuild is a POSITIVE result and gate 1 was the error. Gate 1 counts firings, not correct ones. Measured at matched thresholds with the gold toll located by character offset (`tools/ekf_showcase/binding_accuracy.py`), the rebuild binds the right death toll **67-100%** of the time against the incumbent's **0-7.7%**, with yield

> = at every threshold and 3x at 0.1 off a third of the firings. The incumbent's 65% gate-1 pass is 39 firings carrying THREE correct tolls, binding `dead` to "car Hurricane Helene", "Mexico", "Pacific coast". The mix change did what it was built to do.

The instrument was the problem, and this file carried the bad claim. The sentence that used to sit here -- "run against the incumbent it returns 0.0% and 'no block contains the figure' at every threshold" -- was an artifact. `frontend_gates.py` set only the Schema's per-event `trigger_threshold` / `argument_threshold`, which the boundary greedy path never reads (only the span engine does), so all five sweep rows ran at the default 0.5 -- where 0.0% is genuinely what the incumbent scores. Fixed in bf42950; the gate now drives `extract(threshold=)`. **The config's original premise still stands** (nothing usable at 0.3+, nonsense at 0.1); the "~0 at EVERY threshold" strengthening of it did not.

Gate 1 measures *form*, not content -- the incumbent's nominal pass at 0.1 still binds the documented nonsense, which is what gate 2 catches. Both models fail gate 2.

Gates 3 and 4: SCORED 2026-08-23, both PASS. Both models run by one command on one box, same 11 files / 15,456 rows, threshold pinned 0.5 (`tools/ekf_showcase/frontend_gate_results/GATES_3_4.md`). The candidate leads on **every** head: entity +0.0158, relation +0.0593, classification +0.0160, structure +0.0096, event_type +0.0155, event_trigger +0.0145, event_argument +0.0133, event +0.0043; fair entity/trigger/argument +0.0278 / +0.0147 / +0.0570. Structure also swept to the record head's own thresholds (0.1184 vs 0.1132) because its max object probability is 0.178 and a default-0.5 number measures an unreachable cutoff.

Verdict: gates 1-2 FAIL AS WRITTEN, gates 3-4 PASS, and gate 1 is not a valid comparator. The rebuild improved every corpus metric AND wire-copy binding precision. Pre-registering gates on AP prose was right; scoring a FORM gate best-over-a-threshold-range was not, because it rewards indiscriminate firing. Every form gate here needs a correctness companion before it is used to compare two models.

0. CLOSED 2026-08-19 -- the record head was fine; `runtime.py` dropped `record_metadata`

The five zeros were one line of plumbing. `runtime.py` rebuilt each schema through `Schema.from_dict(...).build()` (which produces `record_metadata`) and then copied only `json_structures` and `json_descriptions` out. With no metadata `compile_record_specs` returns `{}`, no `RecordSpec` compiles, the head decodes nothing, and **nothing raises**. Fixed in `d754132`, regression-tested in `tests/test_record_metadata_roundtrip.py`.

Re-scored from the same checkpoints, the head shows an ordinary monotone curve: 0.0238 / 0.0552 / 0.1043 / **0.1119** strict F1 at 10k / 40k / 100k / 137k. Full write-up, including why the four points are not yet on one test set, in `JOINT_IE_SCALING.md` §0c.

The "+45% structure supervision" in the warm-start was real in record count and **zero in effect**: those 3,494 cc_news/synthetic records carry `json_structures` with no `record_metadata`, so they supervise the record head with nothing. See item 0.1 -- that is the part still open.

0.1. cc_news and synthetic converters emit `json_structures` with NO `record_metadata`

Successor to item 0, and it needs a data-design decision, not a patch. On the training path `Structure.get_record_metadata()` returns `None` unless `mode` is set, so a structure record without metadata produces no `RecordSpec` and no record targets. Counted across the warm-start mix:

source	structure records	reaching the record head
cc_news_haiku45	1,033	0
synthetic_haiku45_5k	996	0
synthetic_sonnet5_1k	1,465	0
replay_137k30	519	519
137k base pool (text2json)	*7,754*	*7,754*

So the warm-start **cut** record-head supervision from 7,754 to 519 (-93%) while appearing to add structure capability, and still only lost 0.0059 -- read that as replay working, not as the head failing.

Any future arm claiming structure capability from these corpora will add none. Note the 2026-08-19 builder default (item 0.3) does **not** fix this: it changed the inference/eval path, while training reads `record_metadata` out of the corpus through `Structure.get_record_metadata()`, which still returns `None` without a stored `mode`. The symmetric change on the training side is exactly the decision below, and it is bigger than the inference one because it alters what every future run learns rather than what a decode emits. Fixing it means assigning `mode` and `anchor` per structure type in the converters. `natural` + first-field anchor is the obvious default but is a real modelling choice about what anchors a `person_profile` or a `transaction`, so it wants a decision before it is written.

0.3. CLOSED 2026-08-19 -- the fluent builder now emits `record_metadata` by default

Item 0 fixed `runtime.py` dropping the key. This was a different drop, upstream of it: the fluent builder never produced the key at all unless the caller passed `mode=`, so anyone following the obvious API got an

empty extraction and no error. What it looked like:

```
Schema().structure("casualty_report").field("dead", dtype="str")
  -> build()["record_metadata"] is None        # head decodes NOTHING, silently

Schema().structure("casualty_report", mode="natural", anchor="dead") \
        .field("dead", dtype="str", cardinality="required_one")
  -> {"casualty_report": {"mode": "natural", "anchor": "dead", ...}}
```

Fixed. `StructureBuilder._auto_finish` now defaults to `mode="natural"` anchored on the first declared field -- the same choice `_store_record_metadata` already made for a caller who set mode and omitted anchor, so this is not a new convention. A structure with no declared fields still emits nothing (no anchor is possible), and `mode="latent"` is unchanged. Verified end-to-end on the 137k checkpoint: plain and declared forms return identical records where plain previously returned `None`.

Scope of the change: inference and eval only. It does NOT touch training supervision -- see item 0.1, which stays open. One behaviour change to expect: `Schema.from_dict` defaults too, so `_schema_from_gold` now compiles specs for metadata-less gold, and in-loop eval on corpora like cc_news/synthetic will start reporting nonzero `structure` where it reported 0.0000 before. Published curve numbers are unaffected -- all 148/856-record test sets behind them are text2json, 100% of which carries explicit metadata.

This also invalidates, in the other direction, the earlier "records return None" measurements recorded in `JOINT_IE_SCALING.md`: they were taken against a schema that could never have worked and can now be retaken.

-1. `eval_metrics.py` CANNOT SCORE `json_structures` — every casualty model is affected

Found 2026-08-17 during the Track A run, and it is the most consequential thing that run produced. `gliner2/training/eval_metrics.py` builds gold/pred sets for exactly six families:

```
_gold_entity_set  _gold_relation_set  _gold_event_trigger_set
_gold_event_type_set  _gold_event_argument_set  _gold_classification_pairs
```

There is **no structure/record scorer**. `casualty_loc_split` is 100% `json_structures`, so the evaluator found nothing it could score and the entire 36-minute run emitted **one** metric key: `eval_loss`. Verified by scanning the log for `eval_[a-z_]+` — one match.

Consequence, and it is not confined to Track A. With no F1 available, `metric_for_best` can only be `eval_loss`, and on this corpus it *latched at epoch 1*: one "New best eval_loss: 82.1840" and never again, while train loss fell **35.16 → 4.50** over six epochs. `best/` was therefore the epoch-1 model — confirmed by mtime, written 6 minutes into a 36-minute run. Measured cost of trusting it (40 Helene windows):

checkpoint	location filled	event filled
`best/` (epoch 1, what the selector chose)	43/48	17/25
`checkpoint-epoch-6` (what training actually produced)	**51/54**	**30/36**

Every casualty model ever trained — `casualty_ft`, `casualty_multi`, `casualty_docee`, and the `casualty-docee` checkpoint the EKF pipeline runs in production — used the same selector on a corpus the evaluator cannot score. Whether they latched as early is unknown; their logs are gone. This is the same failure class as the MAVEN Tier 2 "+0.049 win".

Fix: add a structure/record scorer emitting `eval_structure_strict_micro_f1`, then re-select both Track A and its `casualty-docee` baseline on it. Until then, no casualty checkpoint selection can be trusted, and the readouts in item 0 and item 1 below are the missing metric computed by hand.

-1b. CLEAN RE-RUN DONE 2026-08-17 — both arms, bf16 + structure-metric selection

Item 1 and item 3 of the follow-up list, one A100, ~$3.00, instance terminated. Commit `1b6e0f6`; scorer from `fb456e4`.

bf16 fixed the crash. Both arms ran 8/8 epochs with no non-finite loss, where the fp16 arm died at 79%.

The metric now drives selection — where there is anything to select. `casualty-docee` re-selected **six times** (0.9563 → 0.9794 on val). `casualty-loc-split` selected **once, at epoch 1**, and never improved through epoch 8 while train loss fell 34.65 → 3.42.

That second result is REAL, not another selector artifact — the same recipe on the other corpus tracked fine, so the metric works. Field extraction on `casualty_loc_split` saturates after one epoch. **Practical consequence: epochs 2-8 buy nothing measurable there, so ~85% of that arm's GPU cost is waste.** Cut `num_epochs` before re-running it.

Item 3 — the published models rescored on the metric nobody could compute before. Same 400-record blind slice per corpus:

model	corpus	strict F1	relaxed	support
published `casualty (ft)`	casualty_ft	0.9959	0.9959	610
published `casualty-docee` (production EKF extractor)	casualty_docee	0.9784	0.9784	920
new `casualty-docee` clean	casualty_docee	**0.9822**	0.9822	920
published `loc-split` (fp16 run)	casualty_loc_split	0.7693	0.9159	2,155
new `loc-split` clean	casualty_loc_split	**0.8351**	0.9374	2,155

Read this within a corpus, never across one. `casualty_docee` has three numeric fields; `casualty_loc_split` adds free-text `location`, which is far harder and carries 2.3x the support. The 0.98-vs-0.84 gap is task difficulty, not model quality.

Within-corpus, the clean re-runs win on both: **+0.004** on docee and **+0.066** on loc-split. So the old `eval_loss` selection cost little on the numeric-only corpora — the production `casualty-docee` extractor was not badly damaged — but cost a lot on the corpus with a hard field.

What item 3 could NOT answer: whether each published model latched early during its own training. Those per-epoch checkpoints are gone; only a re-run would show it. `casualty-multievent` was skipped

entirely — no local `casualty_multi` test split, and the rescore script checks the filesystem directly instead of going through `_fetch_if_missing`.

Blind tests: docee **0.9766** (P 0.9655 / R 0.9879, support 4,619); loc-split **0.8485** strict / 0.9543 relaxed (support 15,187).

Models: `whr778/gliner2-base-v1-casualty-{docee,loc-split}-clean` (private).

0c. TRACK B RAN 2026-08-17 — NEGATIVE, and the corpus is the bottleneck, not the formulation

`casualty-events-boundary.yaml`, A100, ~$2.30, terminated. The same documents, splits and figures as Track A, re-emitted as trigger + typed arguments so the loss reaches the event path — the formulation `EKF_MHT_BUILD_RECORD.md` §27.2 says is missing.

The run itself was clean. 8/8 epochs, zero non-finite losses, FA2 confirmed active (0 sdpa fallbacks — on mmBERT that is correctness, not speed). Selection re-selected 0.3494 → 0.4973, where the Track A structure arm froze after one epoch. So the event formulation was still learning where the structure one had stopped.

In-domain it works. Blind test on 2,852 documents / 14,614 argument instances:

metric	strict	relaxed
event_argument (the binding)	**0.5320** (P 0.673 / R 0.440)	0.7521
event_trigger	0.9610	0.9612
event_type	0.9897	0.9897
event (combined)	0.7566	0.8652

Trigger and type are near-ceiling and largely **circular**: triggers were derived by matching a fixed per-type surface list, so the model mostly learns that list back.

On real news it produces NOTHING. All 104 Helene windows unbound — with the eight trained types, and zero-shot with a `Hurricane` type. Verified against a working in-domain extraction on the same checkpoint in the same session, so this is transfer failure, not a broken probe. (An earlier reading of "zero" *was* a probe bug — the event type was given as `casualty_report` instead of the trained DocEE types — and was caught before being believed.)

This was predicted. Item 0b, written before the run: "*the boundary base fills ~0 on real wire copy … any events-form arm trained on a boundary base inherits that domain gap and will be unmeasurable for the same reason.*" It did, and it is.

So the Track A vs Track B comparison cannot be made. Track A scores 3/11 @ 9.6% FP on these windows; Track B scores nothing at all. Taken together with Track A's positive, the reading is: **the formulation is not the bottleneck — the corpus is.** Synthetic-realized prose does not transfer to AP wire copy on the boundary/mmBERT path, while the span path fine-tuned from `fastino/gliner2-base-v1` does. Effort belongs in data, not architecture.

Model: `whr778/gliner2-casualty-events-boundary` (private).

0. The cross-event readout was unsound — FIXED 2026-08-17 (`63249ed`), and C is dead

Fixed and re-measured. Read this section for what the numbers now are; the diagnosis below is kept because it explains why the published ones cannot be reused.

Three arms, 104 observations, current code, CPU. `C` is now validated; `C-raw` is the old unsound scoring, retained as a diagnostic:

model	arch	A	B	C	C-raw	cross-event binding coverage
`fastino/gliner2-base-v1`	span	3/11 @ 53.0%	3/11 @ 50.6%	**3/11 @ 30.1%**	3/11 @ 36.1%	ours 7, competitor 3, unbound 1
`gliner2-base-v1-casualty-docee`	span	3/11 @ 19.3%	3/11 @ 15.7%	**0/11 @ 2.4%**	7/11 @ 37.3%	**rejected 7**, ours 2, unbound 2
`gliner2-warmstart-natural-clean`	boundary	6/11 @ 77.1%	4/11 @ 63.9%	**0/11 @ 0.0%**	0/11 @ 1.2%	**unbound 11**

Three readings, none of them good for signal C:

1. **The casualty-docee "9/11 win" was 100% artifact** — 0/11 once bindings must name an event. Its 7 rejected cross-event cases are `raw='230'`, `raw='1,400'`, `raw='250'`.
2. **The boundary arm binds nothing** — 11/11 unbound. Note it scores `0/11 @ 0.0% FP`, which under the old table reads as *perfect precision*. This is exactly why the coverage table exists.
3. **The only model that binds sanely gets it confidently wrong.** `fastino` binds *ours* (Helene) for **7 of 11** cross-event cases. The failure is not uncertainty that a threshold could recover — it asserts the wrong event.

So C via the structure/record path is dead on every model available. Best is 3/11 at 30.1% FP, unshippable, which agrees with §27.2's conclusion even though its numbers do not reproduce. What this does *not* refute is the item-1 event-formulation hypothesis: none of these three was trained events-form, so the training arm remains untested — but it now has an honest floor to beat, and a working instrument to beat it with.

New prerequisite this surfaced. The boundary base fills fields at ~0 on real wire copy (separately measured: `location` 1/14, `event` 0/11 over 40 Helene windows, versus 26/33 and 20/21 for the domain-adapted span model). Its record head was trained on synthetic templated casualty text. Any events-form arm trained on a boundary base inherits that domain gap and will be unmeasurable for the same reason this arm was — so the conversion has to carry real contexts, `casualty-docee` style, not just a reformat of `casualty_multi_loc`.

The diagnosis, kept for provenance. `event_binding_probe.py` scored signal C as:

```
"C bound event is a competitor": bool(r["bound"]) and not OURS.search(r["bound"])
#  OURS = re.compile(r"\bhelene\b", re.I)
```

C therefore fires on **any non-empty string that does not contain "helene"** — including a string that is not an event mention at all. Three arms on the same 104 observations, all on current code, CPU:

model	arch	C catches	C false pos	what `bound` actually holds
`fastino/gliner2-base-v1` (§27.2's model)	span	3/11	44.6%	event names — Helene, Katrina, John. Sane
`whr778/gliner2-warmstart-natural-clean`	boundary	0/11	3.6%	**nothing** — the record head fills no field
`whr778/gliner2-base-v1-casualty-docee`	span	**9/11**	50.6%	**the casualty number** — `'230'`, `'1,400'`, `'250'`

The 9/11 is an **artifact**, not a result: `casualty-docee` was trained on `casualty_report{dead, injured, missing, location}`, so an `event` field is out of distribution and it copies the anchor number into it. `'230'` contains no "helene", so C scores it as a caught cross-event. The 50.6% false-positive rate is the same artifact firing on genuine observations.

Two more defects in the same function:

- **§27.2 does not reproduce.** Published C was 2/11 at 26.5% FP; the same model on current code gives 3/11 at **44.6%**. Something in the eval path moved. Do not compare any new arm against the published numbers — re-run the control.
- **The fallback is unsound.** `if bound is None and recs: bound = recs[0]["event"]` attributes the *first* record's event to a span that did not match any record.

All three fixed in `63249ed`: `validate_binding()` requires the bound string to name an event the event schema also found (and rejects purely numeric strings); the `recs[0]` fallback is gone; and a coverage table separates competitor / ours / rejected / unbound. `validate_binding` is unit-tested against all three observed artifacts. The two fixes moved the false-positive rate independently — on `fastino`, dropping the fallback took C-raw 44.6% → 36.1%, and validation took it to 30.1%, while catches never left 3/11.

0b. TRACK A RAN 2026-08-17 — location supervision WORKS, and Track B is warranted

`casualty-loc-split.yaml` on a Lambda GH200, `fastino/gliner2-base-v1` + `casualty_loc_split`, one variable against `casualty-docee.yaml` (the corpus). **Crashed at 79%** — step 4,719 of 5,936, epoch 6.3 — with `FloatingPointError: 1 non-finite micro-batch loss(es) were zeroed` (`trainer.py:1280`). That is `fp16` overflowing. `checkpoint-epoch-6` was on disk and train loss had plateaued (5.81 → 4.74 → 4.50), so the readout uses epoch 6. **The arm is therefore 6 epochs against the baseline's 8 — a real if small confound, stated not hidden.**

Binding, via the fixed probe, 104 observations:

model	C catches	C false pos	what it binds
`fastino/gliner2-base-v1`	3/11	30.1%	event names
`casualty-docee` (no location supervision)	0/11	2.4%	**the casualty number**
Track A, epoch 6	**3/11**	**9.6%**	**event names**

The finding: location supervision fixes the field-semantics collapse. `casualty-docee` answers an `event` query with a number — the numeric-field collapse `_locate_place`'s docstring predicted. The Track A model binds `Hurricane Helene`, `Hurricane Katrina`, `Georgia`. Same catch rate as the base model at **a third of the false positives** (30.1% → 9.6%), and it correctly binds Katrina's 1,400 to `Hurricane Katrina`.

Location fill also passed its pre-registered gate comfortably: **51/54** against the baseline's 26/33, on more records (54 vs 33).

What it does NOT show. Catches are still 3/11, unshippable as a detector. And the 11 is contaminated: §27.2 established that six of the `230`s are mislabelled — they bind `Hurricane Helene` because Helene's toll genuinely reached 230, so they are *correct* predictions counted as misses. The real denominator is nearer 5. The model also binds non-events (`the election`), so precision is better, not good.

Verdict: positive, so Track B is warranted — supervision changed binding behaviour in the right direction in the cheapest possible setting. Before spending on Track B, fix item -1: a 6-epoch-vs-8-epoch comparison selected on an unscoreable metric is not a foundation to build the expensive arm on.

Model: `whr778/gliner2-base-v1-casualty-loc-split` (private, epoch 6).

1. Number-to-place attachment — DEMOTED to P2 on 2026-08-17; see item 2 for the live defect

Nothing is blocked on this, and the two routes below should not be run as written. Both were re-examined against the code on 2026-08-17. Kept here rather than deleted because the *question* survives; only the proposed answers do not.

The routes are genuinely unrun — verified, not assumed. `deaths_in` appears only in this file, [[PROJECT_HISTORY]], and unit tests: no training config exists. `casualty_multi_loc` is wired into training, but as `STRUCTURE_DEFAULT` in `build_warmstart_mix.py`, i.e. as `json_structures` — never as a relation or an event. And `run_pipeline.py` has no joint/beam/decode-mode flag at all, so the `TypedEndpoints` arm has never touched this task.

Why not to run them anyway:

- **The beam arm is predicted-negative for the live defect, by our own measurement.** `EKF_MHT_BUILD_RECORD.md` §27.2 found the type signal catches **0/11** on cross-event "*because the type is RIGHT there*" — Katrina is a storm too. `TypedEndpoints` is a type constraint, so it targets the scope problem the magnitude gate already mitigates, not the one that is live.

- **The relation arm trains the wrong head.** A supervised `deaths_in(value, place)` is *satisfied by the contaminating pair*: Katrina's 1400 beside "North Carolina" is a well-typed `(value, place)` edge. A relation can fix place-pairing at best; it cannot express which event owns the number. Only the event formulation carries `event_key`, which is the field the EKF observation needs.

The unstated prerequisite, and the real reason "runnable" was misleading. Every mechanism named above — `TypedEndpoints`, joint decode, `event_records`, the record head — is **boundary architecture**. The casualty extractor is a **span** model: all three `casualty-*.yaml` sit on `fastino/gliner2-base-v1` with no `architecture:` key, and a live run loads `gliner2/models/span/model.py`. Any of these routes first requires moving the casualty extractor onto a boundary base.

What survives, and it is the finding not the fix. Proximity, GPE tags, record-internal location and admin rollup have all been tried and all failed. Rollup did what it was supposed to (58 keys → 21, 84% of observations in six clean streams) and per-state tracking is *still* catastrophic: North Carolina 5.637, Georgia with 0 of 5 values in plausible range. The reason is not fragmentation — national totals get filed under whichever state the article happens to be about. Zero-shot is close but fragile, and the fragility is still the argument for training over prompt-tuning: `explicit-scope` phrasing got the hard aggregate case exactly right (120 → North Carolina, 17 → Tennessee, correctly *excluding* the national 227) while two other phrasings of the same request, same model, same text, got it wrong.

Note also that attachment is **not** "solved" by the gate: `EKF_MHT_DESIGN.md` §5 records a 9x win on Helene but a **2.3x loss on the clean held-out stream**. It is a stopgap with a measured cost, which is an argument for trained binding rather than for complacency.

P1 — known-wrong

6. MHT — ANSWERED: not the bottleneck, do not build it yet

§3 specifies gate → Hungarian → top-K hypotheses → track birth/death; none is built. Measured 2026-08-11 by assigning every observation to the scope it actually fits using ground truth — a ceiling, not a method:

```
shipped scope gate      0.591
oracle association      0.537
headroom               +0.055     (9.3% relative)
```

MHT is a hypothesis tree, cost matrix, Hungarian assignment and track management, competing for a 9% residual. Sharper still, **the gate already beats a perfect two-way assignment** on Florida (0.704 vs 0.734) and South Carolina (0.365 vs 0.558) — it has a third option the oracle lacks: *drop*. Florida's 300 and North Carolina's 1400 are not misassigned; they belong to no Helene scope at all.

Tennessee is the one genuine association gap (0.817 vs 0.320), and it is diagnostic: its contaminants are 32, 32, 32, 36, 50 against a truth of 18 — **too large for the state, too small to look national**, exactly what a magnitude rule cannot catch.

Revisit when multi-source feeds land (item 7): sources disagreeing about one event is real association ambiguity in a way one wire service's copy is not.

REVISED 2026-08-25 — the conclusion holds for ASSIGNMENT and fails for the REJECT OPTION. "Do not build MHT" remains right: the two-way oracle scores exactly what the shipped gate does, so perfect assignment is worth nothing and a Hungarian cost matrix has no prize. But the third option this item already identifies -- *drop* -- turned out to carry the entire residual. Re-derived on the pooled metric: shipped gate 29.3 deaths, three-way oracle 17.6, and it gets there purely by dropping more (106 kept -> 76). A global Viterbi decode over {own, aggregate, reject} captures 73% of that and beats the gate on BOTH events (Helene -29.4%, Turkiye -9.8%). See item 13. The lesson is that the decode had to be global and had to include the reject state; neither alone would have paid.

8. Beam vs greedy — RAN, and the result is "the beam is not the story"

Ran 2026-08-10 on Re-DocRED (`joint-boundary-redocred-137k` , 96 relation types, the schema that raised before the qualified-key fix). Same checkpoint both arms, eval-time `decode_mode` switch, threshold 0.5, full 500-doc test:

	greedy	joint (W=16)
relation strict F1	0.0740	0.1803
entity strict F1	0.6960	0.6786

Do not quote that +0.106 as a beam win. It is largely a threshold artifact — 0.5 is near the worst operating point for greedy, which reaches 0.2082 at 0.1 in its own shipped sweep. Three real findings did come out of it:

(a) Beam width should be 1. Sweep over W ∈ {1,2,4,8,16,32,64} on a 20-doc slice, relation strict F1: 0.2406 / 0.2290 / 0.2260 / 0.2211 / 0.2170 / 0.2152 / 0.2058. **Monotonically decreasing.** Widening drops predictions 157 → 117, of which 18 were correct (45% precision on the dropped set, below the 61% overall), so precision rises and F1 falls. Entity metrics are byte-identical at every width — `_finish_nodes` sweeps in every positive-score node regardless of beam state, so width touches only edges. Classic score-vs-F1 divergence: the wider beam maximizes the objective better, and the objective is not F1.

(b) The gain is the formulation, not the search. W=1 barely searches and wins. The working contrast is *independent thresholding vs constrained joint selection*, not *greedy vs beam*. Phase A's framing is mis-specified and the papers should say so.

(c) It exposed the hard-wired threshold — see item 9, which was the actual bug.

Best-vs-best, settled on the slice after item 9 was fixed: both arms peak at threshold 0.2 — greedy **0.2835**, joint W=1 **0.3357**. **Joint wins by +0.052 (+18% relative)** and beats greedy at every threshold on the grid. Real, but a third of what the fixed-0.5 comparison implied. Remaining: confirm on the full 500-doc test. Wall clock 1.5x greedy on a clean slice (the 2.0x full-run figure was CPU-contended).

9. Joint decode ignored `--threshold` for edge selection — FIXED 2026-08-10

`joint_decode` filtered mentions by `mention_threshold` but never passed `decision_threshold`, so it stayed at its 0.5 default and every node/edge utility was centered on 0.5. `gain > 0` therefore demanded p > 0.5 for edges no matter what threshold was requested. Nothing raised; the decode simply stopped responding to `--threshold`, which reads as a model insensitive to calibration rather than as a plumbing bug.

Measured before the fix, relation recall across thresholds 0.5 → 0.1:

arm	R @ 0.5	R @ 0.1
greedy	0.0461	**0.4134**
joint W=1	0.1498	0.1591

Fixed by threading `decision_threshold` from the eval threshold through `joint_decode`. Record **role edges bypass** it via a new `pre_scored_edges` path: a scalar role's utility is the ABSENT-relative log-odds `logit_c − logit_ABSENT`, a comparison against the record head's own ABSENT class rather than a probability cutoff, so shifting it would move scalar roles against a baseline they do not have. That was documented at `candidate_scores.py:223` and is now enforced by a test rather than by a comment.

Consequence for anything already measured: every joint-arm number produced before this fix — including the 12-arm curve's joint rows, if any were run — was measured at 0.5 regardless of the threshold requested.

10a. It is a DATA MODELLING problem being fixed with a gate (2026-08-24)

The observation tuple is `(t, role, value, qualifier, source)` plus a `location`. **There is no field for EXTENT.** `location` says which place is named; it cannot say whether the figure is that place's toll or merely adjacent to it. The example below is the whole issue: 225 binds to south carolina with a *perfectly correct* location.

Every mechanism built against this -- ratio gate, innovation gate, rate filter, membership filter -- reconstructs that missing field downstream from magnitude and position. The oracle says that is exhausted: **perfect hard assignment buys ZERO deaths** (two-way oracle 29.3 against the shipped gate's 29.3), and all ~11 deaths of headroom sit in the REJECT option. A perfect assigner cannot win a problem whose inputs do not determine the answer.

Built 2026-08-24: `--with-scope`. A `scope` field on the casualty record -- `place` / `national` / `sub-place` / `unclear` -- extracted beside the number by the same decode step, exactly the argument `with_location` already makes for place. Routing: national -> aggregate, sub-place -> dropped (this is what puts `"one"` against a North Carolina truth of 123, and the ratio gate is blind to it since it only rejects figures too LARGE), place -> keep, unclear -> defer to the ratio gate.

CATEGORICAL, not a confidence float, and that is measured. A `scope_confidence` was proposed and rejected on evidence: this architecture's confidence SATURATES. All 106 of Helene's `dead` observations carry exactly **1.000** -- contaminants included -- and Turkiye's 89 sit in 0.997-1.000. `CONF_R`, built to

consume that field, defaults to off for the same reason. Consistent with the extractor result that span precision is flat at 44-64% across a 100x threshold sweep: **this model's confidence does not discriminate**, so a self-reported scope confidence would very likely be constant too. An abstention CLASS is something a model can express and be supervised on.

The tunable signal is `scope_agreement()` -- do the extracted scope and the ratio gate's inferred scope agree? Two independent routes to the same field, checkable rather than self-reported, and it degrades gracefully: high agreement means trust the field and skip the gate, low means the corpus needs work before leaning on it.

Status: plumbing complete and tested (18 tests). Field quality MEASURED 2026-08-24 -- it is a zero-shot NO-OP and needs supervision.

model	records	scope values
`casualty-docee` (span), 12 articles	11-12	**`unclear` x all**
`ekf-frontend-mmbert` (boundary), anchor 0.5	2	`unclear` x2
`ekf-frontend-mmbert`, anchor 0.15	8	`unclear` x8
`ekf-frontend-mmbert`, anchor 0.05	15	`unclear` x14, **`sub-place` x1**

The span model returns `None` for `scope` on 10 of 11 records. The one non-empty cell it produced was **`'94,000'`** -- a NUMBER, not a scope class, and the exact value 9.1 names as the reproduction signature. The model does not treat `scope` as a categorical slot; it reaches for the nearest figure, as it does for every other field in the record.

The `unclear` abstention class earned its keep on first contact. A confidence float would have returned 0.62 on `'94,000'` and looked meaningful. The categorical routed it to "defer to the ratio gate", degrading to current behaviour instead of asserting a wrong scope. That is the design question answered by measurement rather than argument.

One real signal: mmBERT at anchor 0.05 produced the only non-`unclear` value seen, a single `sub-place` out of 15. That is noise as a rate, but the span model produced zero across 12 articles, so the field is REACHABLE rather than inert.

Also measured: the boundary model is a viable stage-2 extractor at its own threshold. It does not fail silently as predicted -- at the 0.5 default it under-fires 6x (2 records vs the span model's 12), and at anchor 0.05 it produces MORE (15) and without the span model's `None` cells. Its record head's max object probability is 0.178, so 0.5 is a cutoff it can barely reach. Same shape as the extractor gates being read 100x too high. `boundary_settings` is a FROZEN dataclass -- use `dataclasses.replace`. See `tools/ekf_showcase/record_threshold_probe.py`.

Next: SUPERVISION. No model fills this field without training data, and swapping architectures did not change that. The routing above stays off until a supervised `scope` exists.

The scope direction is open and is where the remaining error lives. The failure is filing a national total under a state — measured on real text: "The number of deaths stood at 225 on Friday; two more were

recorded in South Carolina" binds **225 → south carolina**. That is not a rival claim about South Carolina, and a wrong state silently poisons a state stream where an unbound total is recoverable.

Measured contamination. Every state stream receives larger-scope numbers, and the leak is always UPWARD — never once downward:

stream	truth (final)	contaminants received
Florida	26	64, 150, 150, 160, 180, 230, 230, 300
North Carolina	96	200, 215, 215, 227, 230×3, 250, **1400**
South Carolina	51	72, 200, 227
Georgia	34	178

Sub-part 1 (unlocated → `__aggregate__`) is a NO-OP: 4 of 106 observations. It was proposed first on the reasoning that it had no bootstrap dependency; measurement says it is not worth doing on its own. Multi-state scope phrases (sub-part 2) are already handled by `rollup.json`'s 38 aliases.

Sub-part 3 — the scope gate — WORKS (`scope_gate_test.py`, 2026-08-10). Judge each state observation against the running **national** total rather than against the state's own scale (a state's early history legitimately jumps 6 → 25, faster than any ratio tolerates), and classify three ways: keep / reroute to `__aggregate__` / drop as exceeding the whole.

ratio	Total	per-state mean
off	0.402	5.247
2.5	**0.316**	0.592
2.0	**0.316**	**0.591**
1.5	0.317	0.591

Per-state **5.247 → 0.591 (8.9x)** and the national stream *improves* too. Flat from 1.5 to 2.5, so it is not a knife-edge setting. **Control:** removing the same 25 observations at random over 40 trials gives 4.427, so the gate is selecting rather than thinning.

Headline metric changed 2026-08-24 to POOLED (micro) RMSE in deaths -- one RMSE over every (place, time) point, no per-stream vote. A macro-average lets a small-toll stream outvote the largest, which is how the same normalisation reversed 6.2. The per-stream table is kept for DIAGNOSIS (it is what says where the work is); the normalised column is now a GEOMETRIC mean, since those are ratios spanning 0.4 to 9.8 and one blown-up stream dominates an arithmetic mean of them. Pooled: **314.5 -> 29.3 deaths (10.7x)**, control 244.7.

Two recorded claims do not survive the change.

1. **Association headroom on Helene is ZERO for assignment, and all of it is in the REJECT option.** The two-way oracle -- perfect hard assignment -- scores 29.3 deaths against the shipped gate's 29.3: **no headroom at all**. The three-way oracle, which can say "none of these", scores 18.4 (tol 0.5) and 17.6 (tol 0.25), so the whole ~11 deaths of association headroom is the null hypothesis. In macro-

averaged nRMSE this read as +0.055 vs +0.111 -- the same shape, but "build better assignment" was never on the table and the deaths number says so outright.

2. **The Turkiye transfer does not hold.** Recorded as "partly transfers" on the nRMSE mean (1.815 -> 0.723). In pooled deaths at the shipped ratio 2.0 it is **13,603.7 -> 14,765.0, a LOSS**: Turkiye is 87.6% of the total and the gate hurts it (0.228 -> 0.522) while helping Syria, which the macro-average rewarded. `--reference implied` is a no-op there (0 observations moved) because Turkiye-Syria has no `__aggregate__` stream to imply from. The gate is validated on Helene and does not currently transfer.

Re-checked in ABSOLUTE deaths 2026-08-24, and it SURVIVES — worth stating because the same macro-averaged nRMSE reversed the aggregate-constraint verdict in [[EKF_MHT_DESIGN]] 6.2. Per-place RMSE goes **217.4 → 21.9 deaths (9.9x)** and the national stream 80.9 → 63.5. The normalisation was *understating* this gate, not flattering it.

But it misdirects the remaining work. Residual at ratio 2.0, in deaths:

stream	RMSE (deaths)	nRMSE	n_obs
Total	**63.5**	0.316	45
North Carolina	**60.6**	0.518	16
Tennessee	14.7	**0.817**	9
Florida	12.7	0.704	7
South Carolina	11.7	0.365	5
Georgia	10.0	0.553	4

Tennessee has the WORST nRMSE and nearly the smallest absolute error; North Carolina has a middling nRMSE and the largest. **Optimising the macro-average sends you to Tennessee when the deaths are in North Carolina and the national stream** — those two carry ~80% of the residual between them. Rank remaining association work in deaths, not nRMSE.

Not wired in. The ratio gate lives in `scope_gate_test.py`; `run_pipeline.py` applies only the hierarchy-membership `scope_filter`. A measured 9.9x sits outside the pipeline.

Three-way classification is load-bearing. A two-way version that rerouted every reject wrecked the national stream (0.402 → 2.110), because North Carolina's **1400** is not a national total — it is not a casualty count at all, and it poisoned `__aggregate__`.

Held out on Turkiye-Syria (2026-08-10), ratio fixed at 2.0, not retuned. Partly transfers, and the failure is the informative half.

As validated it **cannot run**: the gate judges against the `__aggregate__` stream and Turkiye-Syria has none — turkey and syria are siblings with no declared parent, and the combined toll never got its own stream. With `--reference aggregate` the gate is a no-op at every ratio.

Generalizing the reference to the running max across all streams (`global-max`) makes it run:

	turkey	syria	mean
off	**0.228**	3.401	1.815
gate @2.0	0.522	**0.923**	0.723

Syria — the contaminated small stream, 11 of 17 values were Turkey's tolls — improves 3.7x. But **Turkey, which was clean, degrades 2.3x**, because `global-max` is dominated by Turkey's own values, so Turkey is judged against a reference it defines itself. It rerouted 1,014 at t=12.5h, which is Turkey's *true* value at that time. Circular by construction.

Mean still improves 2.5x with the control at 1.440 vs 0.723, so the mechanism does transfer. The **reference definition does not generalize for free**.

The finding: the gate needs a declared scope hierarchy, not just a magnitude. Helene has one (`__aggregate__` in `rollup.json` declares states ⊂ national). Without it, a magnitude test cannot separate "this is a larger scope" from "this is the largest part". Next step is to declare the hierarchy per event rather than infer it — cheap, and it is the same information `rollup.json` already carries.

Other caveats: the ratio was chosen after seeing Helene's contaminated values, so the 1.5–2.5 plateau mitigates but does not remove the post-hoc problem. And 0.591 is 9x better than catastrophic, not good in absolute terms.

11. GIST query-axis hard negatives — RAN 2026-08-14, and it is NEGATIVE

The A/B is done and the veto lost. Two arms on one 2xH100, one per GPU, differing in exactly four keys, `mix_natural` 84,280 records x 3 epochs (15,804 steps, 1h44m / 1h43m). Both arms swept to threshold 0.3 on val, so this is best-vs-best:

metric	control	gist	delta
entity strict	0.5858	0.5610	**−0.0248**
entity fair	0.6248	0.6041	−0.0207
relation strict	0.1439	0.1108	**−0.0331**
classification	0.6301	0.6292	−0.0009
event_type	0.9531	0.9515	−0.0016
event_trigger fair	0.7527	0.7399	−0.0128
event_argument fair	0.5786	0.5702	−0.0084
event strict	0.3433	0.3443	+0.0010

It loses on every metric but one, and that one is +0.0010. The sharpest reading is that it is **down on `event_argument`** — the axis the query veto was built to sharpen — so this is not "right idea, wrong dosage" on the evidence available.

The control validates the harness rather than the conclusion resting on it: retrained from scratch it reproduces the historical `warmstart-natural` reference (relation 0.1439 vs 0.154, entity 0.5858 vs 0.580),

so the gap is the treatment.

The veto was live, not inert — `[gist] loaded 46149 cached guide records` in the training log, which is the failure mode the wiring notes below warn about.

Caveats before this is called settled: one seed, and no variance estimate on this corpus, so anything under ~0.005 is unreadable. −0.025 and −0.033 are well outside that. Artifacts (both `best/` checkpoints, sha256-verified off the box, plus metrics and sweeps) are local under `out/gist-ab/`, so a re-probe needs no retrain.

The original specification, kept because the cache and wiring are still sound

The measured gap: with specific rival types, `people evacuated` outscores `death toll` on **11.2% of genuine death tolls**. "N people killed" vs "N people evacuated" — both counts of people, separated only by the verb. No type description fixes it (`EKF_MHT_BUILD_RECORD.md` §27.8); it is a training-time boundary the model has never been taught.

Hard negatives are mined on the **span** axis only — `select_hard_negative_candidates` picks negative *spans* per query. The missing axis is **query**: for a span, which sibling type queries score it highly.

Wired 2026-08-11. Set `guide_scores: <cache.jsonl>` in a training config and the veto is live; leave it unset and nothing in training changes.

piece	state
`apply_guide_veto` + abstention `floor`	`losses.py`; takes an explicit `reference`
guide choice	self-guide validated **82.5% vs 25%** chance on 40 gold records
rival selection	wide-pool top-k; **no embedder needed**
rival **injection**	`GuideScores.inject` — dataset-side, hardest-first
cache -> `[B,Q,C]`	`models/boundary/guide.py`, with hit-rate counters
`precompute_guide_scores.py`	batched; format frozen (`sha1` key + rival descriptions)
the cache itself	**BUILT 2026-08-12** — `data/guide_scores.mix_natural.dedup.jsonl`
a RAMS cache too	**BUILT 2026-08-12** — `data/guide_scores.rams_baseword.dedup.jsonl`

Running the arm — four things checked 2026-08-14, before any spend

1. **The A/B is TWO training runs, not one.** Five commits touched the training path after the control trained on 08-10 (`ca3e362`, `e189362`, `bf2c9b4`, `3a83c8d`, and `210af17`, the GIST wiring itself). `mix_natural` is 7.6% events (379 of the first 5,000 train records), so the Tier 2 event-record changes are **not** inert here and the existing control checkpoint is not a valid arm against a fresh GIST run.

2. **Neither checkpoint is local.** `out/joint-boundary-mmbert-137k/best` — the GIST config's `pretrained` — and `out/warmstart-natural/best` are both absent. Both are on the Hub privately (`whr778/gliner2-joint-boundary-mmbert-137k`, `whr778/gliner2-joint-boundary-warmstart-natural`).
3. **Both warmstart configs select on `metric_for_best: eval_loss`.** Kept deliberately for this arm: with 3 epochs there are 3 candidates, so selection is a small lever, and the decision that matters is the swept-threshold comparison between arms. Revisit if the arm is ever run longer.
4. Pull the checkpoints off the box **before** terminating it, and sweep thresholds on both arms before reading the comparison. Item 12 is what skipping either costs.

The cache — built 2026-08-12, and the merge needed a fix

Four local shards, **21.2 hours**, each verified at exactly `21070 records read` (4 x 21,070 = 84,280, the whole corpus). 46,581 cached records, 0 malformed. Hit rate on the corpus is **54.5%**, which is right: only records with gold spans are cached (46,581 / 84,280 = 55.3%). Loads in 1.1s. Train with `tools/train/config/warmstart/warmstart-natural-gist.yaml`, which differs from the `warmstart-natural` control in exactly four keys — `guide_scores`, `rivals_per_record`, `output_dir`, `experiment_name` — with the data section identical.

Use the DEDUP file, not the raw concatenation. 194 of 46,343 sha1 keys collide, and all 194 **conflict**: the same text appears twice in `mix_natural` declaring *different entity type sets* (indices 104 and 46250 share text but declare `{name, severity}` versus `{address, symptom}`). That is a property of the corpus, not a sharding bug — the shards partition by index, so a repeated text lands in different shards.

It matters because `GuideScores.load` does a plain `entries[row["sha1"]] = ...` (`guide_scores.py:80`): **last wins, no warning**. Those records would be vetoed against another record's own-types — and own-record types must never be vetoed, since gold is authoritative within a record. Dropping the colliding keys costs 0.42% of the cache and leaves the veto explicitly *inactive* for them rather than quietly wrong. The four shard files are retained, so any variant rebuilds in seconds.

Two things the wiring turned up, both of which would have made it silently inert:

1. **Injection is not optional.** A sample's query axis carries only the types its own record declares, and only 0.23% of records name a competing count type natively. The cross-record rivals GIST exists for are *never* on the tensor unless something puts them there. Without injection the veto is a no-op by construction.
2. **`apply_guide_veto` could not fire under the default candidate pool.** It derived each candidate's own positive by taking a max down the query axis at a fixed column — which assumes column c is the same span for every query. True for `candidate_pool="shared"`, **false for the default `"per_query"`**, where each query proposes its own list. The reference is now resolved by span identity and passed in explicitly.

Own-record types are deliberately never vetoed: within a record gold is authoritative, and a same-record rival outscores the gold owner 23.5% of the time — all of it correct hard negatives. Enforced structurally, by

only ever filling injected-rival cells: everything else sits at exactly 0.0 and cannot clear `floor`.

Still to run: the precompute — and it is a LOCAL, SHARDED job, not a GPU one. Renting an A100 to find out was worth the $3: same 96 records, byte-identical output, **376.0s on the A100 (3.9 s/record, 4-13% GPU utilisation) against 186.3s on a laptop (1.94 s/record)**. The accelerator was half the speed, because the cost is Python post-processing rather than the forward pass — ~100 type queries at `threshold=0.0` decode every candidate for every query and the cache then throws nearly all of it away. So `--score-threshold` (now default 0.01) is the real knob, and `--shards` across cores is how the job gets shorter.

Filtering does not close the cost either — a numeric-gold filter keeps 66.6%, a count-type-name filter 37.1% — because only 3 of 8 records yield a coherent rival at all and there is no cheap way to know which in advance.

Nor does renting a bigger box — run it locally. A 240-vCPU / 1771GB instance ($22.32/h, 120 shards × 2 threads) cached **zero** records in 15 minutes: >33 s/record per shard against **3.3 s/record on a laptop**, ~3.6 rec/s aggregate versus 1.2. Workers were at 142% CPU with 1.4TB RAM free while load stalled at 172 of 240 — the ceiling is **memory bandwidth**, not cores (~30 concurrent processes is about where a mid-size server's bus saturates). Choosing the box on `$/vCPU-hour` assumed throughput scales with cores; it does not. **Measure one shard's s/record on the target box before renting.**

Local shape that works: **4 shards × 2 threads with `--pool-cache`**, ~1.2 rec/s, ~19h for the full mix. More shards than that exhausts a 32GB machine and swaps it to a standstill.

Do not use the live model as the guide. A cell is mined *because* the live model scores it highly, so a live self-guide vetoes exactly the negatives it should select. The guide must be a frozen checkpoint.

12. Base-word (lemmatized) duplicate samples — BUILT, alignment proven; not yet trained on

`tools/data/augment_baseword.py` + `tests/test_augment_baseword.py` (5 tests). Measured on 300 RAMS records with the deterministic `mock` backend:

augmentation rate	**91.7%** (275/300)
texts actually rewritten	275/275 — not a silent no-op
labels no longer verbatim	**0**
extra mentions lost vs original, through the real collator	**0**

The 8.3% that are refused are labels covering only *part* of a token — `Armenian` inside `Armenians` — which cannot survive lemmatization of their host token. Those records are dropped whole rather than emitted with a broken span; partial augmentation is precisely the silent-supervision-loss failure this is guarding against.

Example (mock backend, so `urging`→`urg` is crude on purpose — it tests alignment, not lemma quality):

```
ORIG : Transportation officials are urging carpool ... death of Freddie Gray
LEMMA: transportation official are urg carpool ... death of freddie gray
args : ('victim', 'Freddie Gray')  ->  ('victim', 'freddie gray')
```

RUN 2026-08-12 on `--backend simplemma` — and the stated gate was VACUOUS

Full RAMS train, simplemma 1.2.0, `--lang en`: **7,329 → 13,291 (5,962 augmented, 81.3%)**, 3 seconds. The gate passes — gold mentions 27,599 against 27,599, zero records changed — but only after two corrections, both of which the arm would otherwise have been trained under.

1. `missing_surface_counts()` cannot serve as this gate. It increments only for `task_type == "entities"` (`boundary_preprocessing.py:443`). RAMS supervises **events**, and for non-entity types an unlocatable surface is treated as legitimately absent and skipped with **no counter at all** (`:465`). A lemmatized copy could lose every argument and the counter would still read 0. What is observable is the target graph: `targets.mention_mask.sum()` is the gold the collator actually built, and each lemma copy must produce exactly as many as its source record.

Collate with sampling OFF when measuring this. `collate_fn_train` sets `is_training=True` and the default `remove_events_prob=0.2` drops the whole event group a fifth of the time — one record collated ten times gives `[5,0,0,5,5,5,5,5,5,0]`. The first version of this measurement was reading that noise.

2. The real failure is INVENTED gold, not lost gold. Lemmatization *collapses* surface forms, so a label starts matching positions that were never annotated. Gold `guns` occurs once in its source; as `gun` it occurs **three times** in the lemmatized text, so collation builds three mentions where one was annotated. Before the guard: **+1,085 mentions on 31,773 (3.4%), in 718 of 6,680 augmented records** — every one a silent false positive, and invisible to any missing-surface check by construction.

Guarded by refusing any record where a label's occurrence count changes, in the same tokenization collation uses. That ruler is load-bearing: `WhitespaceTokenSplitter` is a regex tokenizer that splits trailing punctuation and lower-cases, so `they,` contains the token `they` while `str.split()` sees only `they,`. A `str.split()` guard still let four records through, netting to a delta of 0 by coincidence — two gaining, two losing.

Cost of the guard: augmentation rate **91.1% → 81.3%**. Those are refusals, not losses; the un-augmented original is always emitted.

Still to do: train the arm. `simplemma` is installed to a scratch dir and used via `PYTHONPATH`, deliberately not `uv add`, which re-locks and syncs the whole environment and could rewrite packages the four precompute workers have mmap'd. Make it a real dependency once they exit.

Arms A/B/C RAN 2026-08-12 — PROVISIONAL, and unreadable until thresholds are swept

Three arms, configs differing from `gliner2-base-v1-rams.yaml` in three lines each (train file, `output_dir`, `experiment_name`), val and test un-augmented. Trained on a Lambda A10, which is terminated; `test_metrics.json` for all three plus trimmed logs are local under `out/gliner2-base-v1-`

`rams{,-baseword,-dupcontrol}/`. **The checkpoints went with the box**, so the sweep below cannot be run without retraining.

Confirmed 2026-08-14: those three directories contain `test_metrics.json` and nothing else — no `best/`, no `threshold_sweep.json`, no per-epoch checkpoints. Recovering the sweep is three full retrains (~4h, ~$5 on an A10), and this file's own "higher-value uses of the same GPU hour" note argues against spending it here.

arm	train file	records
A baseline	`data/rams.train.jsonl`	7,329
B lemma	`datasets/rams_baseword/train.jsonl`	13,291
C duplicate control	`datasets/rams_baseword/train.duplicate_control.jsonl`	13,291

Blind test on the un-augmented RAMS test set, support 2,016 arguments / 848 triggers on every arm:

metric	A base	B lemma	C dup	B − C
argument strict	0.4474	**0.4582**	0.4463	**+0.0119**
argument fair	**0.6192**	0.6124	0.6072	+0.0052
argument relaxed	**0.6873**	0.6805	0.6781	+0.0024
event strict	0.6797	**0.6892**	0.6885	+0.0007
trigger strict	0.9127	0.9313	**0.9369**	−0.0056
event type strict	**0.9970**	0.9887	0.9941	−0.0054

The one number that carries the item: C sits at baseline. 0.4463 against A's 0.4474 — duplicating those 5,962 records verbatim bought **nothing** on strict argument F1, while B beats both by +0.0119. On the metric base-word supervision actually targets, the gain is therefore attributable to **lemmatization, not duplication**. That is the B > C row of the decision table below, so arm D's dosage question becomes legitimate.

Do not act on it yet. Three reasons, in order of severity:

1. **Every arm sits at the config's fixed threshold 0.5, unswept.** The standing rule in this project — promoted from lesson to rule *because it changed a conclusion three times* — is that no arm or curve comparison is readable until every arm sits at its own swept threshold. A +0.0119 gap is comfortably inside the range that rule exists to protect against. **This result is provisional until swept, and the checkpoints needed to sweep it no longer exist.**
2. **One seed, no variance estimate.** +0.0119 is ~2.7% relative on a single run.
3. **The picture is mixed, not clean.** B wins strict argument but *loses* to C on triggers (C best at 0.9369) and to plain A on argument fair/relaxed and event type. Winning strict while losing relaxed means the spans land more exactly without more of them being found — sharper boundaries, not better role routing. That is the opposite of the noun-phrase-routing failure item 12 was proposed to fix, and it should temper any claim that base-word supervision addresses `convoy` as a `victim`.

The validation curves said the opposite, which is itself worth recording: C tracked B closely at every shared epoch (C ahead at 1–2, within ~0.006 thereafter), implying the gain was duplication. The blind test reversed it. Validation and blind test also diverged on arm A alone (val 0.6797, blind-test strict argument 0.4474) — read the blind test.

On a combined arm D — HOLD, and make it conditional on B vs C

Do not plan D now. It is worth running in exactly one of three outcomes:

B vs C	reading	D worth it?
B > C	lemmatization adds something beyond duplication	yes — the dosage question is real
B ≈ C	the gain is duplication, not lemma	no — D adds more of what did not help
B < C	lemmatization actively hurts	no — D would hurt more

And D would need its own control or it reproduces the confound C exists to remove. A combined arm is originals + lemma + verbatim = **19,253** records against B and C's 13,291, so D-vs-B differs in record COUNT as well as composition and a gain is again unattributable. The matched control is arm E: originals + *two* verbatim copies, also 19,253. That is two runs (~2.5h, ~$3.30) to answer a dosage question, and only after B > C is established.

Note also that B and C are not two treatments to combine. **C is a control — the null version of B.** Combining a treatment with its own control is "double the augmentation", not a factorial design; a genuine 2x2 needs a second *factor*, not a second dose.

Higher-value uses of the same GPU hour, both of which are the confluence rather than more dosage:

- **base-word applied to the casualty/event line** — tests whether the lever generalizes off RAMS, which is what would justify it in the papers.
- **GIST on RAMS** — the literal river-join with item 11. The RAMS guide cache is being built (see item 11). It carries a concrete prediction to test rather than assume: the veto drops mined negatives the guide judges positive, and base-word negatives ARE mined negatives, so a frozen guide carrying the same NP-routing bug will score `convoy` highly for a `victim` query and **veto exactly the negative item 12 exists to add**. That says the merge order must be: measure base-word alone, then add GIST, then check whether the base-word gain survives. Merging first gives a null nobody can attribute.

The remainder of this item is the original specification, kept because it states the constraints the implementation had to satisfy.

Proposal. For each training sample, emit a **second** sample in which every surface word in *both* the text and the labelled spans is reduced to its base form. Surface and normalized variants both stay in the mix (1:1 duplication, not replacement). Reported from prior practice as helping training substantially. *Not measured in this repo.*

Prior art, if replicating: PURE (Princeton) is recalled as doing a **partial** version of this **in its code rather than its paper** — reportedly inherited from the DyGIE/DyGIE++ preprocessing it reuses. Recollection is

several years old and unverified here; do not go looking in the PURE paper's method section for it, which is where this note originally went wrong.

Why it is plausible here specifically. The event corpora are small — RAMS 7,329 train, CASIE 795, WikiEvents 206 — while role fillers and triggers inflect freely (`killed` / `killing` / `kills`). Normalizing collapses those into one form, so a trigger–role association is learned once instead of three times under-powered. It is also a second angle on the noun-phrase routing in item 2: normalization strips the morphological cue the model may be latching onto instead of the role semantics.

The constraint that decides whether this works: spans must stay verbatim. Boundary collation locates each gold surface inside the text; a mention that cannot be aligned is **silently dropped** under `on_missing_surface="skip"` (counted in `missing_surface_counts()`, `boundary_preprocessing.py`). So the failure mode is not an exception — it is quietly reduced supervision, which looks like "augmentation didn't help".

The rule that avoids it: **lemmatize the token sequence ONCE, then re-derive every label from its token offsets.** Never lemmatize the text and the label string independently — lemmas are context-sensitive (`left` → `leave` or `left`), so the two passes diverge and the label stops matching. Verified today that `text_tokens[start:end]` reconstructs gold surfaces exactly (69/69, and cleanly under truncation), which is the property an offset-based rewrite must preserve.

Acceptance gate, cheap and decisive: run the augmented corpus through the collator and assert `missing_surface_counts()` gains **zero** entries relative to the un-augmented run. If it gains any, the alignment is broken and the measurement that follows is meaningless.

Language gating. The mix is multilingual (mmBERT; CMNEE/DuEE/ChFinAnn Chinese, KLUE Korean, MasakhaNER across 20 African languages). Lemmatization is a no-op for Chinese and a different operation for agglutinative languages, so this must be opt-in per corpus rather than applied across `data/`. No lemmatizer is currently a dependency — a dictionary-based, token-wise one (no per-language model download, deterministic) is the right shape, because token-wise is exactly what the alignment rule above requires.

Write path. Any new emitter must route through `_split.dumps_record`, per the repo rule — NFKC plus line-separator stripping, `ensure_ascii=False`.

What the metrics fixes did and did not touch (2026-08-14)

Two eval-side defects were fixed. Neither reaches training, and the blast radius was measured rather than assumed, so **no number in this file needs redoing**.

`c0ab89c` — `metric_for_best` silently fell back to `eval_loss`. A run configured to maximize an F1 maximized loss instead. Now raises.

`7586411` — `_schema_from_gold` dropped `entity_descriptions`. Corpora that name types `e_0` / `e_1` and carry the meaning in a parallel map were scored by asking for the empty label. On 100 `pile_ner_def`

val records against pristine `fastino/gliner2-base-v1`, strict entity F1 0.0174 without the map against 0.5381 with it; recall 0.0092 → 0.4771.

(`bbacce6` claimed this fix and was a **no-op** — it put the map under `schema["entities"]` as the values, which are label targets, not prompt text. Cite `7586411`, not `bbacce6`.)

Selection was never affected. Every training config except `eval-preservation-ner.yaml` selects on `eval_loss`, which the trainer computes from the forward pass (`trainer.py:2109`) and which never passes through `_schema_from_gold`.

Blind-test reach, as share of each config's test records carrying `entities` **and** `entity_descriptions`:

config	affected
`eval-preservation-ner`	78.5% (4,715/6,003) — built 08-13, never had a valid number before
`mmbert-base`	**49.3%** (106,657/216,154) — its blind-test entity row is understated
`joint-boundary-mmbert-{10k,40k,100k}`	0.5%
`warmstart-{natural,anchorless,struct}`, `mmbert-137k`, `natural-gist`	0.2%

The record-mode A/B (item 11's control, `0ca9447`) is **0.2%** and stands: `data/mix_natural.test.jsonl` is 0 bytes, so `mix_natural` contributes a val split only and its 35.5% description share never reached a blind test. No working paper quotes an entity number from `pile_ner_def`, `nuner_full` or `pubmed_abstracts_ner` — the headline numbers are event metrics on RAMS, which carries no descriptions.

Notes for whoever picks this up

- **RESOLVED (2026-08-17): the "~49% stall" is not a hang — it is one 6.5-minute test.** `test_public_api_e2e_real_deberta.py::test_boundary_public_api_lifecycle_real_deberta` takes **390.5s standalone and passes** (`--durations`), and it lands at the 52% mark in the combined run. There is no deadlock and no cross-test pollution: the suite was simply sitting in a slow test with no timeout, on a machine also holding a 15GB job.

Four tests carry `@pytest.mark.slow`, all real-DeBERTa; three exceed 120s. They are slow because `DebertaV2Model` rejects `sdpa` and falls back to **eager** attention (the loader warns), so a real training loop runs unaccelerated on CPU.

Run this and the problem disappears — the whole suite in ONE process, no chunking:

```
uv run pytest tests/models/boundary tests/processing -m "not slow" --timeout=120
```

→ **328 passed, 4 skipped, 4 deselected in ~30s.** With the slow tests included it is 454s and the three time out. Keep `--timeout` on in CI so a slow test reports as a failure with a stack instead of looking like a hang.

Diagnosis cost one command; `pytest-timeout>=2.1` was already in the dev group. The earlier note that this "needs a machine not already holding a 15GB job" was wrong — you never needed the suite to *finish*, only to hang, which it already did reliably.

Two things fixed on the way: `pythonpath = ["."]` is now set in `pyproject.toml`, because `tests/` is not a package while `tests/conftest.py` imports `tests.fixtures` — plain `pytest` used to die at *collection* with `ModuleNotFoundError: No module named 'tests'` and only `python -m pytest` worked. Both invocations work now.

- **Summarizer-as-segmenter was tested and is not the answer** (`bullet_premise_test.py`). Hand-written bullets on 5 real Helene sentences, rollup-aware scoring: raw text 3/5 with 1 false positive; *free* bullets 2/5 with 3 FP and **2 fabricated figures**; *extractive* bullets (every digit copied from source) 3/5 with 1 FP and 0 fabrications. Restructuring does not improve attachment on this corpus. The free variant actively harms — its most useful act, turning "they died together" into "2 people died", is exactly what a verbatim-number guard must reject, so guard and summarizer are in direct tension. Also note the corpus does NOT contain the tidy "120 NC / 17 TN / 227 total" sentence everyone reaches for; the real numbers are distances, populations, years and rainfall.
- **Everything new is off by default.** `--rollup`, `--event-year`, `--record-mode` and `--associate envelope` all have to be passed explicitly on `run_pipeline.py`. The defaults reproduce the older numbers, on purpose.
- **`probe_records.py` is the record-extraction check, not the blind test.** The blind test scores tasks; it does not tell you whether record mode is emitting the fields you think.
- `datasets/helene2024/_cache/` and `datasets/turkey2023/_cache/` hold harvested article text, are gitignored by design, and both harvesters regenerate from the Wayback archive.
- The anchorless arm is deliberately **not** published: it learned nothing (1 of 9 instances), so it is evidence for the papers rather than an artifact worth shipping. The natural arm is on the Hub as `whr778/gliner2-joint-boundary-warmstart-natural`, private.

13. The association layer, RESOLVED 2026-08-25 — ship the HMM decode

Five mechanisms built and measured against Helene AND Turkiye, each at that event's best shipped setting rather than the shipped default:

#	mechanism	verdict
1	Student-t measurement model	Helene -1.7, Turkiye +651, retires no knob. **OFF**
2	Viterbi decode,	Helene -29.4%, Turkiye -9.8%. **SHIP**
3	IMM / PDA soft association	loses to the hard decode on both. **No**
4	4th state for downward revision	correct, inert on this data. **No**
5	date + scope + boilerplate folded into the emission	false rejections 19.8% -> 9.9%. **SHIP**

What to ship: `scope_gate.hmm_gate`, which is `viterbi_gate` plus emission features -- a strict superset, same three states, same decoder. Recommended sigma 0.3, reject_cost 4.0, stay 0.1, **warmup 0**. It replaces the ratio gate and lets gates [5] `out_of_window` and [6] `scope_filter` be deleted as separate stages.

Design rule, measured: keep feature weights BELOW `reject_cost`, so no single feature can force a reject alone -- it can only tip a case magnitude has already made marginal. The sweep shows a cliff exactly at that boundary, and it is what protects Turkiye.

Why item 4 is inert, which is the more useful negative. Helene's North Carolina truth falls four times as deaths are reclassified, but the REPORTS never follow it down -- after each revision the later readings are 230, 230, 230, 1400, 98, 250 while the official toll falls to 84-123. No filter, state or change-point detector can track a revision that is never reported. This reframes `CENSOR_AT_LEAST`: the filter is not wrong to refuse to descend, **the data never descends**. Fixing it needs a source carrying reclassification bulletins, not a better filter.

13a. The third event — RUN 2026-08-25; prediction 1 FALSIFIED

Turkiye cannot validate the cross-event collapse, and the reason is now diagnosed rather than guessed: its contaminant (Izmit 1999, 17,500) is the same order of magnitude as the event (~50,000) and **crosses** the true trajectory, so rejecting it costs more coverage than the impurity costs accuracy. Helene's differ by 6x. **Scale separation is the property that decides whether cross-event rejection pays.**

2020 Aegean Sea earthquake, pre-registered in `THIRD_EVENT_AEGEAN2020.md`: 119 deaths against an Izmit reference of ~17,000, a **143x** separation.

- Ground truth **BUILT** -- `datasets/aegean2020/ground_truth.json`, 55 points from the Wikipedia revision history, zero parse failures, Izmir 12 -> 116 with a genuine downward reclassification (116 -> 114 on 5 Nov).
- Feed **IN FLIGHT** -- `build_aegean_feed.py`, Hurriyet Daily News + Daily Sabah via Wayback. First 76 articles: 68 quake-relevant, 60 carrying a toll, **20 mentioning 1999** with the contaminant in the expected shape ("the 7.4-magnitude earthquake in Golcuk in 1999. It killed more than 17,000 people").
- Half the pre-registration was **falsified before the build**: the Izmit and Smyrna comparisons live in the CURRENT Wikipedia article, not the 2020-era revisions (0 of 55 mention 1999). Wrong SOURCE, not wrong event -- 15 of 16 Al Jazeera articles about Turkiye 2023 carry the same reference. Hence GT from Wikipedia, documents from news, which also makes this the first feed with genuinely independent sources on both sides.

RESULT. Feed built (71 articles, 18 carrying the contaminant), pipeline run (61/71 relevant, 108 observations).

```
shipped gate @2.0    74.38    inert -- same at ratio off/4.0/3.0/2.0/1.5
viterbi (magnitude)  15.74    -79%
oracle (tol 0.25)    19.10
```

The decode's third win: Helene -29.4%, Turkiye -9.8%, Aegean -79%.

Prediction 1 FALSIFIED. The emission features move pooled RMSE by nothing (15.74 at every weight 0-3) although they work -- drops 15 -> 21, three of six >500 contaminants rejected. The contaminants are keyed `unknown` / `marmara`, never `Izmir`, and only 12 of 53 observations are bound to a gated place: ASSOCIATION isolates them before the gate sees them. Helene is the same shape (cross-event figures sit in mexico/puerto rico/bosnia streams, none scored).

Consequence, and it is structural rather than a sampling problem: the collapse cannot be validated on pooled RMSE on either feed, and a fourth event would not change that. Its effect is real but only measurable on cross-event catch/false-reject rates -- Helene 5/6 at 19.8% false becoming 4/6 at 9.9%. Prediction 2 partial (`_f_date` carries it, cannot reach the score); prediction 3 held.

15. The relevance gate does not filter non-English at all (2026-08-25)

Measured on 200 clean negatives from denizzhansahin/Turkish_News-2024:

```
fastino/gliner2-base-v1   199/200 = 99.5% false admits   <- SHIPPED DEFAULT
fastino/gliner2-multi-v1   56/200 = 28.0%
```

The shipped default is DeBERTa-v3, vocab 128,011, English-only. It cannot read the text so it answers mass_casualty to nearly everything -- worse than the 58.5% that forced v1 -> v2, and SILENT, since nothing reports that the gate stopped discriminating. Translating to English does not fix it (17/60 still admitted by the English model on English text), so it is the LABEL DESCRIPTIONS, not the language. A v3 rewrite has to decide two things the current text does not cover: whether a war casualty total counts, and how to exclude a rescue or evacuation carrying a number but no deaths.

Archive: the full TODO list as it stood on 2026-09-21

TODO.md was rewritten on 2026-09-21 to hold ONLY outstanding work, because it had grown to 1,331 lines in which resolved narratives outnumbered live items and the live ones could not be found. **The entire prior contents are preserved verbatim below rather than summarised**, so that anything mis-classified as historical during that rewrite is still recoverable. Sections here are dated and many are superseded -- read TODO.md for what is actually open, and the [[EXPERIMENT_CATALOG]] for what was run.

Open items — resume list

STATUS 2026-09-21 -- where the three event levers actually stand

question	status	evidence
do absent negatives help?	**ANSWERED**	`absneg2` : event_argument strict **+0.0376**, classification **-0.1977**, 3 up / 8 down outside the floor. Operating point verified IDENTICAL, both arms shipped their final epoch, treatment proven applied. NOT shippable as-is
does the derived corpus help (option 4)?	**OPEN**	first run negative on 7 of 8 heads (event_argument -0.0300, trigger -0.0541); its entity row and the over-proposal diagnosis are RETRACTED (different test sets). `roles2` re-run VOID (epoch 2 vs 5, and different test sets). `roles3-*` configs fix both; only the CONTROL needs retraining, the treatment's final-epoch checkpoint is reusable
does a typed-role constraint help (option 2)?	**ANSWERED -- NO, for the trained margin**	`typed_margin_mask` built 2026-09-21 and the ceiling MEASURED rather than argued. The mask fires (0.339% of candidate cells, 91 queries with a disallowed competitor) and moves nothing: the loss effect at delta=ln2 is **0.00% median, 0.28% mean**, with only 5 of 91 affected queries above w_S 0.01. The model already scores type-incompatible fillers near zero. A 0.28% mean gradient perturbation cannot move F1 against a +/-0.02 floor, so the ~$34-65 trained arm is CANCELLED. Decode-time remains a null on F1 with a real precision/recall trade (98% of drops were false positives); map reach is 99.3%. See `OPTION_2_TYPED_ROLE_CONSTRAINTS.md`

Three defects found and fixed in two days, each of which had silently voided experiments: the label-negative injector never reached a sliding-window dataset (since the feature shipped); `metric_for_best: eval_loss` selecting two arms on objectives they do not share; and an A/B treatment scoring on its own added test split. All three are now gated, and the runner publishes `final/` so a selection confound is diagnosable after the fact.

Closed items live in [[EXPERIMENT_CATALOG]] — every run, newest first, with its outcome and cost. This file is what is still open.

Completed work is removed rather than struck through; history lives in `PROJECT_HISTORY.md` and the commit log. Everything below is a defect with evidence attached, or a decision with a stated next test.

Every claim below was re-checked against the code on 2026-09-08, after two entries were found stale — one of them quoted back as a live recommendation. Verified still true: four converters split row-wise (0 `SplitWriter` in `docee` / `docfee` / `cmnee` / `mendeley_ed`). Corrected: the gate entry, the structures-are-never-scored entry, and one date.

State at 2026-09-08 close. No GPU running, nothing billing. The phase that just closed produced no new model either: the programme's central decode question was answered by repairing two instruments and re-measuring, and a 500-document generation A/B was rebuilt after its only copy was lost with a disk. See `PROJECT_HISTORY.md` Phase 32.

Open after 2026-09-15 -- metrics, from reading OneIE

- **We cannot compare event arguments to the literature, because we do not compute the literature's metric.** OneIE's Arg-C is *offsets + event type + role* with NO trigger

  > **SUPERSEDED 2026-09-15.** These figures are of uncertain provenance and are NOT at threshold 0.5 — the incumbent's model card says `Decision threshold: 0.3`. Measured at the validation-selected 0.2: event_argument strict 0.0991 / relaxed 0.5884, event_trigger 0.5984, event_type 0.8650. See EVENT_ARGUMENT_DIAGNOSIS.md §4c. requirement. Our `strict` ADDS trigger identity (lower bound, 0.1178); our `relaxed` DROPS exact spans for overlap (upper bound, 0.5783). The comparable number lies between and is not computed. **Next action, no GPU:** add an Arg-C variant — exact surface, type + role, no trigger — to `eval_metrics.py`. Until then, quoting 0.1178 against published work overstates the gap, and this has been done repeatedly today.

- **OneIE is an alternative answer to the pooling problem, not just a baseline.** It tags triggers token-level with BIO + CRF, so one node per trigger SPAN and multi-instance falls out with no cap to lift. `event_records` is the record-head route to the same end. **Hold as the fallback** if the record head disappoints — it has a published result behind it. *(`EVENT_ARGUMENT_DIAGNOSIS.md` §4c.)*

IN FLIGHT 2026-09-15 -- the event-capable base

- **`eb16-eventrecords-tr` is training on an A100 (~14h, ~$28).** eb16's data with `event_records: true`, so the record head is warmed on events FROM STEP 0 rather than having the path flipped under a naive head (which is what made both Tier 2 arms uninterpretable). Throughput was measured before booking: the flag costs **9%** (18.4 -> 16.7 samples/s on A100), NOT the 5x the record-head defect implied, so this is a $28 run and not a $167 one. **Read `eval_event_argument_strict_micro_f1` against

  > **SUPERSEDED 2026-09-15.** These figures are of uncertain provenance and are NOT at threshold 0.5 — the incumbent's model card says `Decision threshold: 0.3`. Measured at the validation-selected 0.2: event_argument strict 0.0991 / relaxed 0.5884, event_trigger 0.5984, event_type 0.8650. See EVENT_ARGUMENT_DIAGNOSIS.md §4c. eb16-rebuild-tr's 0.1178, and report RELAXED beside STRICT** -- strict moving toward relaxed while relaxed HOLDS is the binding hypothesis confirmed; both moving means something else changed. NOT a one-variable A/B: it differs by the flag AND three added corpora (professorbob_re, scierc, paraloq_json).

Open after 2026-09-15 -- event arguments

- ** `event_argument` 0.118 is a BINDING failure, not an extraction failure, and the fix is

  > **SUPERSEDED 2026-09-15.** These figures are of uncertain provenance and are NOT at threshold 0.5 — the incumbent's model card says `Decision threshold: 0.3`. Measured at the validation-

> selected 0.2: event_argument strict 0.0991 / relaxed 0.5884, event_trigger 0.5984, event_type 0.8650. See EVENT_ARGUMENT_DIAGNOSIS.md §4c. a head that was never trained.** Strict 0.1178 against relaxed 0.5783 on the same predictions -- dropping the trigger link multiplies the score by five -- while boundary and label errors together are 6.6% of gold. 64.2% of gold event instances share their type with another instance in the same document (cmnee 1,606 docs, maven 346, casie 102), and the mention path compiles ONE instance per event type, so two-thirds of gold events are structurally inexpressible. `event_records: true` lifts that cap and is set in ZERO base configs -- verified, `eb16-rebuild-tr.yaml` and every file under `config/base/` have 0 occurrences -- so the RECORD HEAD HAS NEVER BEEN TRAINED ON AN EVENT. The two Tier 2 arms that flipped the path (CASIE 0.0036 vs a 0.2998 control; MAVEN trigger -0.008) changed the path and the head's competence together and measured their sum. **Next test:** warm the record head on events FIRST, then re-run Tier 2 on CMNEE rather than CASIE. Full evidence and caveats in `EVENT_ARGUMENT_DIAGNOSIS.md`.

- **Half the arguments are missing before binding is even considered, and that is a SECOND problem this does not explain.** Relaxed recall is 0.492 and 9,059 of 18,557 gold arguments (48.8%) are never proposed. Pooling explains mis-binding, not absence. Unowned.

Open after 2026-09-08 -- the joint decode path

The re-measurement RAN and closed the question: 87% of the structure deficit was the never-connected `field_dtypes_list`. Deficit -0.0454 -> -0.0057, greedy +0.0432, joint +0.0828, all seven heads now inside the floor. What remains:

- ~~**Cardinality is not declared by any CORPUS**~~ -- **RAN 2026-09-15, and the answer is NULL.** casualty_loc_split, same base, same blind test, 3 epochs, gate PASSED (control `NONE DECLARED`, treatment `optional_one=48,610`, so the arms genuinely differed):

  structure strict F1 control 0.7388 treatment 0.7326 delta -0.0061 precision 0.8228 0.8424 +0.0196 recall 0.6703 0.6481 -0.0222

Inside the +/-0.02 floor. The mechanism behaved exactly as designed -- the scalar softmax plus ABSENT forces one choice, so precision RISES and recall FALLS -- and the two cancel. **Declaring cardinality is not free money.** SCOPE: one seed, one corpus, and casualty_loc_split is structures-only so no other head was measurable. It does not say the regime change is worthless in a multi-task mixture. Do not spend a second run on this without a reason beyond "try it on more data".

- **Phase B (beam in the loss) is now MORE interesting, not less.** Every measurement of the beam to date ran with its scalar constraints disconnected. The decode-time null stands, but it is a null about a beam that has never been trained for and, until today, had never been decoded with its own constraint machinery live.
- **The pre-fix numbers in any document dated before 2026-09-08 are superseded.** The reproduced -0.0529 on `mmbert-137k-clean` was measured under the same defect; it corroborates that the artefact was mechanism-wide, not that joint loses structure.

Open after 2026-08-28

- **CLOSED 2026-09-08 — the gate questions below were answered by two commits and this entry was never updated.** Verified against the code, which is the authority: `run_pipeline.py --gate-model` defaults to **`whr778/gliner2-gate2-mmbert-tr`** (changed 2026-09-04, coupled with adding `tr` to `language_gate.SUPPORTED` — neither change alone makes Turkish work), and `--gate-threshold` defaults to **0.9** (chosen 2026-08-28 by `gate_threshold_sweep.py`: 0.9 is the tightest cut holding full stream coverage; 0.998 only *appears* better because coverage collapses to 2 of 6 streams).

And `fastino/gliner2-base-v1` was never a candidate for the pipeline it was being compared in. Its 0.9635 AUC is English-only: it is DeBERTa-v3, vocab 128,011, and it false-admits **199 of 200** Turkish negatives. The sweep that showed it leading measured English, on a pipeline that has been trilingual since 2026-09-04. A model that cannot read two of the three languages does not win a gate benchmark, whatever its AUC on the third.

The transferable part is the reason this entry survived two commits that killed it: a resume list is only as good as the last person who checked it against the code, and nothing here forced that check. The stale entry was quoted as a live recommendation on 2026-09-08 before being caught.

- **Turkish is affordable but not bought.** The pilot ($0.72) found 43.3% positives in 989 articles; the remaining cue-bearing region is ~2,100 more positives for ~$2.84, plus one ~$2 training run. Two caveats stand: the source is one outlet in one year, and its positives skew to conflict rather than natural disasters. **Decision, not a test:** buy the region and retrain, or ship translation-at-ingest (measured, AUC 0.4733 → 0.8359) and spend nothing.
- **M2 — the cost matrix — remains unbuilt and unmeasured**, and with M4 superseded by the global decode there is currently no reason to build it. Recorded so its absence stays visible in the divergence table rather than being inferred.
- **`exposure_only` is closed as a training problem.** 0.444 → 0.903 by moving a threshold. Do not spend another run on it.

Read this before quoting any historical number. The blind test leaked. `SplitWriter` drew one random per ROW, so a document emitted more than once scattered across train/val/test — **1,080 documents, 7.03% of the cold-start blind test, were in train**. Fixed (grouped splits are now the default, `check_leakage.py --config` gates a config, and `train.py` repairs before every run); contamination is down to 21 documents. The 137k reference `entity 0.586 / relation 0.170 / event_type 0.956 / event_argument 0.098` is **superseded** by `0.6306 / 0.1573 / 0.9365 / 0.1014`, and the delta is not a contamination estimate because the test was also recomposed.

Closed since 12 Aug.

- **Item 11 (GIST query-axis veto) — DONE, negative.** Negative on 7 of 8 metrics. Entity −0.025 survives the measured noise floor; relation −0.033 does **not**.
- **Item 12 (base-word arms) — the provisional +0.0119 remains provisional**, and now also rests on a contaminated blind test.

- **Phase-3 event loss — the flat weight is a null lever, for a mechanical reason.** `task_loss_weights` reaches only start/end/pair, 18.5% of the loss, so `w=4` moved events 6.6% → 10.6% of the gradient. `task_loss_weight_scope: all` raises reach to 94.3%; that is what the live arms test. `pos_weight` belongs in the **cold-start** rebuild, not here: event positive fraction is 0.052 at init but 0.562 at convergence.
- **A measured noise floor exists.** Two seeds of one control on `mix_natural`: relation strict **±0.041**, event_type ±0.014, event_trigger ±0.013, entity fair ±0.010, event strict ±0.008. Anything smaller is unreadable on one seed. Being re-measured on the rebuilt mixture.

New open items from 15 Aug.

- **Every one-seed curve in this project is now suspect.** A control re-run of the published RAMS 137K recipe scored **+0.023** above it, so single-run variance on that metric is >=±0.02. The head-init boundary curve (0.177/0.191/0.202/0.192) is **flat within noise** and its shape must not be cited; the "turns at 100K" reading was retracted the same day it was made. The span-vs-boundary verdict survives (3.5× at 10K). **Any curve claim needs >=2 seeds per point before it is quoted.**
- **`scope: all` is a real lever and wants a proper sweep.** +0.013 event strict at both w=2 and w=4, above the floor, entities unharmed; cost is event_type −0.019 at w=4. A dose sweep with 2 seeds per point would locate the knee.
- **An intermediate `mix_natural` stage is a wash on RAMS** (arguments span 0.005 across three arms), which also means warm-starting through a broad multi-task corpus does NOT cost event capability. `event_type` +0.021/+0.018 in both treatment arms is the one delta worth a second seed.
- **Four converters still split row-wise** (`docee`, `docfee`, `cmnee`, `mendeley_ed`) — no `SplitWriter`. 21 residual contaminated documents, gate-removed each run, unfixed at source.
- **`data/scaling_joint/` is IN ACTIVE USE, not an archive, and its format is CURRENT.** Re-checked 2026-09-15 after it was proposed for deletion to free Hub space. Three things the earlier entry did not say:
- **Every live config sources its `val:` splits from here** (`chfinann`, `docee`, `docfee`, `duee`, `cmnee`, `text2json`, `maven`, `events_biotech`, `mendeley_ed`, `casie`), including the base training right now. It is not a leftover.
- **The format is not the old silently-undecodable one.** `text2json.val.jsonl` — the only structure corpus in there — carries `record_metadata` on 146 of 146 structures. The rest are event corpora with no structures, so none is expected.
- **The aggregate leakage gate passes** on the assembled mixture (21,138 val documents, all three intersections zero).

Still true and still deferred: the files are frozen from 18 Aug, they predate later split fixes, and rebuilding them also rebuilds the j10k/j40k/j100k slices the scaling curve rests on — so regeneration produces DIFFERENT files and breaks comparability with the published curve. **That is precisely why the Hub copy must NOT be deleted**: it is the only off-machine copy of data that cannot be regenerated identically, which

is the single-copy risk that cost a corpus on 2026-09-08. - ~~**Structures are never scored by the blind test.**~~ **FALSE SINCE 2026-08-17** and left standing for three weeks. `_schema_from_gold` gained a `json_structures` branch in `fb456e4` ("eval: score json_structures"), so a structure-only record now builds a real schema instead of an empty one and is scored rather than skipped. Today's decode-arms run reports `structure` strict F1 over 797 structure records on the 18,786-record blind test — the entry was refuted by the project's own headline measurement while still being quoted. `probe_records.py` remains useful for per-field detail, not as a substitute. - **The wider contaminated corpora are unregenerated**: gliclass_logic 38%, knowledgator_gliner 27%, klue_re 17%, finer_ord 14%, MasakhaNER 12-14%, nuner_full, pubmed_abstracts_ner. Not in a live config, so not blocking. - **`rams` has 13.7% duplicate documents inside train alone** — a different problem from cross-split leakage, and RAMS is a key downstream.

Where the line stands. The scope gate took per-state error 5.247 → 0.591 (item 10) and largely closed the attachment blocker. Two candidate next steps were then *closed by measurement rather than argument*:

- **MHT is not the bottleneck — but the number that said so was wrong, and is now corrected.** Perfect *two-way* association is worth +0.055; that oracle cannot reject, so it never priced a null hypothesis. With a reject option the ceiling is **+0.111 (18.8%)**. The cheapest piece that delivers one — M5 track birth by innovation gating — was then **built and lost** (0.608 against the magnitude gate's 0.591), because judging a stream against its own track is circular. `PIPELINES.md` §4/§4.1, `tools/ekf_showcase/mht_associate.py`. Item 6.
- **Extraction recall is not the bottleneck either**, and the claim that it was rested on a stale count from a superseded run. `extract_long` had already fixed it **4.2x**.

The live bottleneck is **cross-event contamination** (item 2, quantified at 4.7% and resistant to all three signals tried) and underneath it the query-axis training gap that GIST was built for (item 11 — **RAN 2026-08-14 and it is NEGATIVE**).

Update 2026-08-20 — item 2's data-side fix RAN and is SUPERSEDED. The muting arm trained and the suppression is real, but a declared per-event **plausibility ceiling** — one threshold, no model — beats it outright, and the large false positives it removed were never other storms' tolls: populations, insurance policies, power crews, years. Both genuine cross-event figures survive both mechanisms. So **cross-event is still the live bottleneck, now with two mechanisms measured against it and neither touching it.** What did change: **item 3 (non-casualty numbers) is promoted** — at 3.8% of observations it carries the largest values and dominates nRMSE, and it splits into classes needing different fixes (entity typing reaches insurance policies and churches; it does not reach 1,500 troops or 8,000 power crews, which are living people and need casualty-role semantics).

Why events and not relations, when a fix is proposed. The programme's priority is event extraction; NER, relations and structures are carried along with it. That is why the losses were separated in the first place — lumping structures, relations and events into one loss made the event signal invisible, and phase 2 split them (re-implemented for the boundary head in [[PROJECT_HISTORY.md]]). So when a defect admits both a relation-shaped and an event-shaped fix, the event-shaped one is the one that serves the programme *and* the one that carries the right information: only the event formulation has an `event_key`, which is the field an EKF observation needs. See item 1 for the worked case.

The next move is NOT downsampling casualty_events

The config pre-registers that remedy for a gate-1 failure, but two measurements taken since say it cannot work:

1. **Gate 2 scores sparsity, not binding, so a mix change cannot reliably move it.** `_decode_events` emits ONE instance per event type and pools every trigger and argument into it -- its own docstring says the mention path "carries no instance dimension". Measured on the incumbent with two hurricanes in one passage: `n_event_instances=1`, with Helene's 246 and Katrina's 1,400 both filed as `dead` on the same event, at 0.1 and at 0.01. Gate 2 takes min(start)..max(end) over that single pooled instance, so it passes only when the model happens to emit *nothing but* Katrina-local spans. **Not impossible -- the incumbent does produce a LOCAL Katrina block at 0.01** (the passage carries one Hurricane-typed event; Helene is a mention, not a second extracted event). But what the gate rewards there is sparsity, which is why it only lands at the threshold where the rest of the output is nonsense. Pooling makes it fragile, and no corpus teaches a model to emit fewer spans on demand.
2. **`casualty_events` cannot teach the missing capability anyway.** It carries 8 event types and no named identities, so it has no same-type discrimination (Helene vs Katrina) in it -- which is the live defect in item 2.

The reasoning changed on 2026-08-24 but the conclusion did not. It is NOT "the mix failed"; the mix succeeded. It is that binding correctness is now good and the router still has no per-event input, because the decode cannot emit two events of one type. **The next move is the instance dimension -- the record head -- not another corpus.**

That path has one prior measurement, and it is not a clean negative: the CASIE Tier 2 arm produced REAL multi-instance output (2-9 instances of one type on 17 of 39 probed docs, structurally impossible before) and scored 0.0036 against a 0.2998 control -- diagnosed as head-init, the record head having never been supervised on events and given 375 steps to learn instance formation cold. The fair test it names: warm-start the record head on MAVEN, then fine-tune on CASIE, with far more steps. The head is also miscalibrated (max object probability 0.178 against a 0.5 default).

Open and worth cheap work first: yield is still only 15% at 0.1 and 26.7% at 0.05. The rebuild's remaining misses are mostly word-form golds (`six`, `three`, `dozens`) where it binds a nearby numeral -- a normalisation gap, not a binding failure, and probably fixable without training.

`tools/data/split_rams_test.py` gave rams a val split by carving its 871-row test **by document**, which found 101 duplicate rows in test alone.

State at **2026-08-23**. **No GPU running** (the front-end A100 self-terminated 2026-08-21, verified 0 instances). A programme-wide caveat landed with it: the cached Helene observation set behind every published Helene figure **cannot be regenerated from any committed state** — see `tools/ekf_showcase/muting_arm_results/PROVENANCE.md`. Comparisons among the published numbers stand; placing a new model on their scale does not. Earlier state at **2026-08-17**: Two eval-side defects fixed on 08-14 (`c0ab89c`, `7586411`); see "What the metrics fixes did and did not touch" below before re-reading any number in this file. A third is now open and unfixed — **item 0**, the cross-event probe's scoring — and it invalidates the readout item 2 would be measured with. Nothing is mid-flight.

STATUS 2026-09-17 — label negatives, measured end to end

The feature is built, measured, and running in a base rebuild. Full detail in [[LABEL_NEGATIVES_PLAN]]; the short version:

- **The machinery was already there and starved.** `negative_query_ratio` (0.5), `abstention_loss` (0.2) and `count_loss` (0.2) all act on ABSENT queries, and there were **0 of 574** in a real training batch — the menu was built from the answer key. Classification was the sole head with real negatives, which is why its precision is real where `event_type`'s is 1.0000 by construction.
- **`events: 1` is the whole active ingredient.** An arm with entity negatives ALONE was indistinguishable from the control (argF1 0.3515 vs 0.3517). Dose beyond `{entities: 1, events: 1}` moved argument precision by <0.004, and halving `abstention_loss_weight` moved nothing.
- **What it buys:** type-rejection precision **0.5977 vs 0.5310**, invented types **274 vs 363**, argument precision **+0.065 vs +0.015** over base, trigger **+0.013 vs +0.002**.
- **What it costs:** ~a quarter of the recall gain, so argument F1 **+0.031 vs +0.055**. **The trade is inherent** — the two available levers do not move it.
- **Two false alarms retracted by measurement:** no catastrophic forgetting (entity **+0.069** over base, not degrading) and recall was never lost (**+0.003** over base, a smaller gain). Both were written up before the base reference existed.

Open, in priority order

1. **Read the base rebuild** (`whr778/gliner2-eb16-eventrecords-neg`) against `eb16-eventrecords-tr` on the identical 20,602-record test. At epoch 2 trigger F1 has already crossed ahead (0.3000 vs 0.2955) with precision **+0.124**; argument still trails. Early epochs are NOT predictive — the previous run's epoch 3 went backwards on arguments.
2. **Link NER to event arguments** — three options costed in EVENT_ARGUMENT_DIAGNOSIS §4k. Cheapest first: `candidate_pool: shared` is an existing config flag never swept on this line.
3. **Fix the option-3 prototype**: the filter must hook into `compute_metrics`, not `extract_events` — the two decode paths differ by ~20x in arguments returned on casie.

TRACKED, NOT DROPPED — a beam-aware / structured loss

Deferred on 2026-09-17 in favour of a cheaper fix, and explicitly kept. The joint beam decodes over candidate scores trained GREEDILY: nothing in training optimises for the beam's constraint satisfaction, so the search is layered on scores that never knew it was coming. That is a genuine train/test mismatch and it is NOT what was fixed today.

What WAS fixed was a plain bug sitting on top of it — joint mode skipped `_decode_records`, so with `event_records: true` it lost 35 triggers per 60 documents (net −23 correct items). That is repaired and pinned.

The evidence that the structured-loss question is still live, from the same probe: where the beam SUBSTITUTES rather than deletes, the two arms are right at nearly the same rate — **41.4% greedy-only against 37.5% joint-only**. A coin flip is exactly what you see when the scores carry no information about which constraint-consistent assignment is correct. That is the half a beam-aware or structured loss would address, and no measurement here has touched it.

And a second symptom pointing the same way: on biored (entities + relations, no events) the beam ADDS 143 entity predictions of which only **5 are correct** — 3.5% precision on what it contributes. The beam is making confident, constraint-consistent, wrong choices, which is the signature of scores that were never trained against the constraints.

Candidate approaches, none evaluated: a structured hinge over the beam's assignment, a differentiable relaxation (Sinkhorn / SparseMAX) so the assignment carries gradient, or a cheaper constraint-violation penalty that needs no differentiable beam. **Re-open after the decode gap is confirmed fixed at scale.**

P0 — blocks the next experiment

0.2. `record_anchor_threshold` defaults to 0.5, which no model can use well

Separate from the decode bug and still live. Nothing calibrates the record cutoffs -- `threshold_sweep` moves the general decision threshold only. At the 0.5 default the models score 0.0000 / 0.0052 / 0.0654 / 0.0760 versus 0.0238 / 0.0552 / 0.1043 / 0.1119 at their swept thresholds: a 32-100% loss depending on scale. Either fold the record cutoffs into `threshold_sweep` or change the default; `tools/train/sweep_record_thresholds.py` is the sweep in the meantime.

2. Cross-event contamination — now the top real defect

> **Gated by item 0 (2026-08-17).** This is the live bottleneck, but the probe that would score any fix is unsound — it counts a copied casualty number as a caught cross-event. Fix the readout first; otherwise a training arm cannot be told from an artifact.

Whole-article reading via `extract_long` surfaced streams for `poland`, `bosnia`, `afghanistan`, `iran`, `japan`, `ukraine`, `cameroon` — casualty figures lifted from unrelated stories sharing an article body.

The gate answers "is this article about a mass-casualty event". It never answers "does this number belong to *that* event". The date filter is the temporal version of that check and it worked (Izmit 15 → 3 false bindings, zero genuine losses). **The spatial version does not exist.** This is not cleanup — it is the same research question as item 1 seen from the other end, and it should probably be solved once, for both.

Left deliberately unmapped in `datasets/helene2024/rollup.json`: mapping the foreign places would hide this problem rather than fix it.

Quantified 2026-08-11, context audit of all 106 'dead' observations: 82.1% genuine Helene casualties, **4.7% cross-event**, 3.8% non-casualty numbers, 9.4% unclear. The five are Katrina 1400, a Typhoon's 250,

Milton's 230, Bosnia's 16, and Hurricane John's 2 in Mexico — they carry the *large* values, so the most damage per instance.

Three signals tried, all failed (`EKF_MHT_BUILD_RECORD.md` §27.2): nearest named event 3/11 at 32.5% false positives, only-competitor-named 3/11 at 31.3%, record-head binding 2/11 at 26.5%. Helene articles routinely name Milton and Katrina for comparison. Bosnia's 16 is structurally invisible — Bosnia is a *place*, not a named storm.

Note the scope gate removes Katrina's 1400 **for the wrong reason** — because it is large, not because it belongs to another event — so it keeps any *small* cross-event figure, as it does with Bosnia's 16 and Mexico's 2.

Proposed next experiment: negative documents (data, not decode)

Do not reach for sharper type boundaries. The standard mitigations for event cross-contamination — span-based boundaries, contrastive/hard-negative objectives — target the failure this project already solved. Type energies separated unit errors 4/4 with 0/83 false positives and scored **0/11** on cross-event, because in every cross-event case the type is *right*: Katrina's 1,400 scores `death toll` 0.95. The boundary architecture and the GIST veto (item 11) both sharpen `death toll` vs `people evacuated`; neither can separate Helene's dead from Katrina's dead.

The gap is negative supervision on event identity. Measured on 20,000 records of each corpus: **0.0% of training documents have zero records.**

corpus	records/doc	zero-record docs
`casualty_ft`	all 1	**0.0%**
`casualty_multi`	mean 2.35, `{1,2,3,4}`	**0.0%**

`build_multievent_corpus.py` already concatenates *k* interference snippets from other streams — but gives **every** one its own record. The model is therefore never once shown a figure it is supposed to leave alone. Practitioner experience puts the healthy share of negative documents at **30–40% of the mix** (not measured here — a prior to test, not a result).

Note `remove_json_structure_prob: 0.2` does **not** provide this. It drops the structure from the *schema*, so no query is emitted at all; the model never sees the `casualty_report` query answered with nothing.

The change is small and local: in `build_multievent_corpus.py`, keep a fraction of interference snippets in the document text while *withholding their records*, so the gold for that document covers the focal event only. Per-snippet span location (already implemented, to avoid labelling one event with another's number) is exactly the machinery needed to know which spans to leave unlabelled.

RAN 2026-08-20 — TRAINED, and SUPERSEDED by a one-line threshold

> **Verdict first.** Both arms trained (4 epochs, A10, ~$2.35, terminated). The treatment is real — blind-test precision up / recall down, and 15 of the control's 20 large Helene false positives removed, cutting

> ungated per-place error 46.844 → 19.822. Then a **declared per-event plausibility ceiling** — no model, no training, no GPU — recovered and exceeded that gain: at a ceiling of 2,000 the CONTROL wins both ungated (5.853 vs 6.194) and gated (3.336 vs 3.729), while carrying 81 *more* observations. The arm's pre-registered guard passes only against an undefended control. **Do not cite this as a success.**
>
> **And it fixed the wrong class.** The large false positives were never other storms' tolls: they were Asheville's population (94,000), Boone's (19,000), FEMA flood-insurance *policies* (129,933), wellness checks (15,000), power crews (8,000), troops (1,500), churches (1,100) and years read as tolls (1,916, 2,004). Both genuine cross-event figures — Katrina's 1,400 and Maria's 3,000 — survive muting **and** the ceiling. **Cross-event is still open.**
>
> Two things this promoted out of the footnotes. **Item 3 (non-casualty numbers) is worth more than its 3.8% billing** — it carries the largest values, so it dominates nRMSE, and one figure destroyed one state's stream. And it splits into classes needing different mechanisms: entity typing reaches policies and churches but *not* troops and crews, which are living people in the affected area and need casualty-role semantics.
>
> Full write-up, raw probe output and the ceiling sweep: `tools/ekf_showcase/muting_arm_results/` (`README.md`, `FALSE_POSITIVES.md`, `PLAUSIBILITY_CEILING.md`, `PROVENANCE.md`).

BUILT 2026-08-12 — `--mute-interference-prob`, control proven

`build_multievent_corpus.py` + `tests/test_multievent_muting.py` (6 tests). Four things the implementation settled, two of which change what the experiment can claim:

1. No loss or model change is needed — the architecture already carries this. A muted snippet emits no record, so its figures are spans with no gold. `build_candidate_labels` scores a candidate 1.0 only on an *exact* match with a gold pair; everything else takes 0.0 at full candidate weight, and the mask encodes validity rather than goldness — there is **no ignore path**. Measured on the two-candidate case (focal gold, interference muted): scoring the muted span high costs **1.1269 against 0.1269**, an 8.9x penalty.

The corpus has always depended on this — documents are full of unlabelled displaced counts, magnitudes and dates — and guard 2's collision-drop only makes sense if unlabelled-vs-labelled matters in both directions. Muting extends it to the figures that actually confuse the model, and additionally drops the gold instance count from k+1 to the unmuted count, supervising instance formation toward focal-only.

Not the mechanism, though it exists: an all-empty record yields `count = 0` (`processor.py:968`) with its queries still counted (`model.py:1394`). That is the fully-negative document, which finding 3 rules out here. It does confirm at code level why `remove_json_structure_prob` is no substitute for either — it hits `continue` at `processor.py:911` *before* `schemas.append`, so no query is emitted at all.

2. The focal snippet is always `parts[0]`, so muting is learnable from POSITION. "Extract from the first paragraph" scores perfectly on this corpus without representing event identity at all. Real articles do lead with their focal event, so the prior is not pure artifact — but the corpus cannot distinguish the shortcut from the intended behaviour. **Required control before any gain is read as event identity:** a held-out probe with

the focal placed last. Without it this arm cannot answer the Bosnia question, which is the reason it was proposed.

3. A true zero-record document is not constructible from this corpus. Every snippet reports a toll, so a document with the `casualty_report` query answered empty would teach suppression of a *genuine* lead-event toll. What muting produces is the **partial** negative — focal record kept, interference figures unlabelled. The measured "0.0% of training documents have zero records" is real, but closing it needs negative *snippets* (disaster text carrying no casualty figure) drawn from another source; it is a separate lever, not this one.

4. The control arm nearly moved silently. Drawing the muting decision from the shared `rng` advanced it once per interference snippet, shifting every later `randint` / `choice` and rebuilding the corpus — 4,064 documents against the pre-change 4,065 **at `mute_interference_prob=0.0`**. Fixed with a dedicated `mute_rng`, so the arms now differ in labels only. Note the obvious test does *not* catch this: the buggy draw fired regardless of probability, so all arms shifted together and stayed mutually identical. Only comparison against a builder with no muting concept exposes it, so the control is pinned by hash.

Measured on 40 streams (4,106 snippets), `--mute-interference-prob 0.35`:

	control	muted
documents	4,065	4,033
instances	9,869	7,763
documents with a muted snippet	0	**1,688 (41.9%)**
unlabelled figures delivered	0	**3,075**

Read the 41.9% as documents with a muted *snippet*, not as documents whose gold actually changed: 69 of them lost nothing, because every value in the muted snippet had already collided and so carried no record in the control either. Gold differs on **1,619**.

`0.0` reproduces the pre-change corpus **byte-identically** (`cmp`). The 32 missing documents are focal-collision cases where every interference record was also muted; they are dropped rather than emitted empty, for the reason in (3). The reported counter is `dropped_empty` = 41 control / **73** muted — 41 of those are collision drops the control makes too, so the muting-attributable loss is the 32-document difference, not the 73.

Build the val split at `--mute-interference-prob 0.0`. Nothing in the flag enforces it, and a muted val is not comparable with the control arm or with any historical number.

Why this and not another association signal. It is the only candidate that would reach **Bosnia's 16**, which is structurally invisible to every signal tried so far — Bosnia is a place, not a named storm, so nothing keyed on storm names can see it. And the evidence says this is a training-data gap rather than a decode gap: binding collapses 1.000 → 0.369 the moment documents become multi-event, which no decoder change has moved.

Pass/fail as pre-registered — and note the readout it named could not be scored as written. The 106-observation reference set turned out to be a cached artifact reproducible from no committed state of the

repo (`muting_arm_results/PROVENANCE.md`), so the arm was read against a fresh baseline under one recorded invocation instead. Original text: cross-event share below 4.7% on the same 106-observation audit; single-event binding stays ~1.000; the §20 harness unchanged.

Second lever, same data side: base-word positive/negative samples

A *different* granularity from the above, aimed at a different failure — **noun-phrase routing**, where the head latches onto whatever salient noun phrase is nearby rather than a filler of the requested type. Two reproducible instances:

```
"Rebels attacked the convoy near Aleppo on Tuesday, killing three soldiers."
  schema: victim = "a person harmed"
  gliner2-joint-boundary-rams-137k  ->  victim: ["convoy"]      # not a person
```

and, from the guide-score cache, `Person/Entity` at **0.56** outscoring the gold casualty type on "*killed a man and his 14-year-old daughter*" — the span genuinely *is* a person reference, so a generic person type wins on a casualty query.

Both are the same mechanism: the model routes to the syntactically salient NP, and the type query only re-ranks among NPs rather than deciding whether the head word can fill the role at all. Supervision at the **base-word** level — positives for head words that can fill a role, negatives for words that cannot (`convoy`, `homes`, `customers` for a person role) — attacks that directly, where a span-level objective does not: every candidate the span objective sees is already a plausible NP.

Note this is orthogonal to the negative-document work above. Negative *documents* teach **which event** a figure belongs to; base-word negatives teach **whether a word can head a filler** at all. Item 11's GIST veto sits between them, on the query axis, and does neither.

Untested here. No measurement in this repo yet supports or refutes it; the two examples above establish the failure exists, not that word-level supervision fixes it.

P2 — research direction

2b. Does a fine-tune need an explicit regularizer, or is early stopping enough?

Raised 2026-08-18 while launching the real-vs-synthetic arms. The synthetic fine-tune gained hugely in-distribution (entity fair 0.7946 → 0.9134) and **lost 22% relative on real general NER** (entity strict 0.5320 → 0.4136, swept best-vs-best). FairEval says it is not a boundary regression — BES and BEL both fell while FN rose 27,267 → 36,244. The model stopped proposing spans and mislabelled more of what it proposed: the 125-type synthetic label space overwrote what the base knew.

No regularizer was added to the real arms, deliberately. The synthetic control was trained without one, so a regularizer in the new arms would confound the comparison — any preservation difference could be the penalty rather than the text source. There is also a real hypothesis that real news needs less of one: the

damage came from training on out-of-distribution prose, and cc_news is far closer to `pile_ner_def` than generated passages are.

What WAS changed is retention only: `save_total_limit` 3 → 10 on both arms, so every epoch checkpoint survives (`best` / `final` are exempt from rotation — `trainer.py:2332`). Selection is on in-domain val, so if forgetting grows with epochs the best in-domain checkpoint is the worst preserving one. Keeping all ten lets the preservation curve be scored per epoch and the knee found post hoc. **Early stopping is the cheapest regularizer and needs no change to the loss.** It doubles as crash insurance.

Order of levers if the curve shows real text still degrades: early stopping first (free, already instrumented), then replay of base-distribution data in the mixture, then parameter-space constraints (LoRA / L2-SP / EWC). Do not start at the expensive end.

2c. Chunking distorts the real-news arm — measured, and not where expected

`window_size` / `stride` are **subword tokens, not words**; the configs' "word window" comments are wrong and `gliner2/training/chunking.py` is authoritative. At 384/256 on 1,500 cc_news train docs (deberta-v3 tokenizer), 2.25 chunks per document.

The hypothesis was that cross-window structure is lost and that this argues for mmBERT's 8,192 context. Measured per document — supervision that survives in **no** chunk:

```
relations   1,794 in source,  84 lost = 4.7%
events        383 in source,   6 lost = 1.6%
```

So the long-context argument is real but modest, not decisive. **The larger distortion is classification inheritance**: doc-level labels are copied to every chunk (+124.5%, exactly the 2.25x expansion), so most classification training examples are fragments asserting a document label the fragment may not support. That is injected label noise, and it lands hardest on the arm with the longest documents. Quantify its effect before adding more classification data to a real-news mixture.

Raw annotation counts are useless for this — they rise across the board under chunking (entities +42.4%) because overlap duplicates them. Measure survival per document.

2d. What breaks a stochastic activation is FUNCTION-CLASS churn, not randomness

Explored 2026-08-18, prompted by the observation that GeGLU's gate reintroduces an unbounded gradient path. **Answered -- and the motivating premise was then REFUTED in a real transformer. CLOSED.**

Read this first. `tools/prototypes/lr_ladder.py`, 6-layer MLM encoder on wikitext-2, LR escalated to divergence: plain GELU dies at 1e-1, **GeGLU and hybrid4:fixed both survive it and both die at 3e-1**. So GeGLU is the MOST stable of the three, not the least, and hybrid4's toy-measured gradient-ceiling advantage (13.4 vs 23.2) does not transfer -- in the encoder its grad norms track GeGLU's and it breaks at GeGLU's threshold. The gradient-bound analysis below is correct as measurement and wrong as prediction: a bound on a layer's local derivative is not a bound on training dynamics, which depend on normalisation, depth, the optimiser, and the network's ability to adapt its own gates. Full write-up in `tools/prototypes/PARTIAL_GATING.md` section 11.

The function-class-churn result (below) is unaffected -- it is about stochastic masking, not about gating, and it still holds. Cost of the refutation: $0, ~90 min local MPS, against a ~$379 staged plan that the stop rule correctly cancelled.

The starting diagnosis is correct and worth recording. A pointwise activation has a bounded derivative -- ReLU exactly [0,1], GELU [-0.1289, +1.1289] -- so it cannot amplify a gradient. GeGLU's `y = v * gelu(g)` gives `dy/dv = gelu(g)` and `dy/dg = v * gelu'(g)`: each branch scales with the OTHER branch, and neither is bounded. Measured max |grad| through the activation, by input scale:

```
scale      gelu    geglu   stoch-gelu   1-gated-chunk-of-6
    1      1.13     4.42         1.13                4.36
    2      1.13     9.80         1.13                8.69
    4      1.13    19.48         1.13               17.41
    8      1.13    37.77         1.13               32.68
```

Partial gating does not partially protect. In the hybrid only 2.6% of channels exceed 1.13, yet the MAX is within 15% of full GeGLU. Explosion risk is set by the worst channel, not the mean, so gating one chunk in six buys ~1/6 the exposure and ~6/6 the tail. Note also that GELU never fixed explosion over ReLU -- both are bounded. What GELU fixed was dead units.

The negative: randomising WHICH CHANNELS GET THE NONLINEARITY costs ~10x. Five chunk layouts, param-matched 4-block MLP, test MSE (lower better):

```
layout            fixed    random
2-of-4           0.0567    0.7267
6-of-12          0.0702    0.8010
8-of-12          0.0492    0.6239
10-of-12              -    0.4326
HYBRID4 8-chunk  0.0655    0.6555
```

No overlap: deterministic 0.049-0.070, random 0.43-0.80. Per-sample masks (0.7783) and finer chunks (0.7439) do not help; the random arms improve with the activated fraction only because that dilutes the randomness (at p=1 it IS plain GELU). The random arms also have the TIGHTEST seed spread (HYBRID4 random +/-0.0089) -- they converge reliably to a bad solution, which is a method-level floor, not bad luck.

But randomness itself is not the problem, and HYBRID5 is the control that proves it. HYBRID5 keeps 6 slots always-GELU and gives each of the 2 linear slots a randomly drawn GELU'd partner to multiply. Same harness, same target, also randomized:

```
randomized variant   what the draw changes                    test MSE
HYBRID4              whether a slot is GELU or identity         0.6555
                     -> the slot's FUNCTION CLASS moves
HYBRID5              which chunk partners a gated slot          0.0623
                     -> function class fixed, only the operand moves
```

So the rule is: a draw that changes what KIND of function a slot computes is fatal; a draw inside a stable function form is free. `fc2` reads a fixed slot, and one weight cannot be correct for both GELU output and identity output. **Dropout escapes this only because it is linear in the mask** -- `E[mask*x] =`

`p*x`, so one scalar corrects it. Swapping a nonlinearity has no scalar correction, hence on HYBRID4-random weights: expectation blend 0.8203, sampled 0.8622, **plain GELU at eval 7.2860** -- the intuitive "stochastic at train, clean at eval" rule is the worst of the three. HYBRID5 has no such problem: `c6` is independent of the draw, so `E[c6 * g_j] = c6 * mean(g)` is an EXACT eval rule, not an approximation.

The positive, worth trying in a real model. HYBRID4 with a FIXED assignment -- 8 chunks, 6 GELU, the two linear slots holding `a*b` and a passthrough, every chunk staying in its own slot -- is parameter-identical to a plain GELU FFN (100,481 both), preserves width exactly (no 2x up-projection, so none of GeGLU's +50% or the 2/3-d_ff workaround), scores 0.0655 against plain GELU's 0.0636, and has the LOWEST gradient max in the study because only 1/8 of channels carries a product.

One gated slot in eight is the sweet spot; two is worse. Whole-network gradient max, every row statistically tied on MSE:

```
HYBRID4 fixed   13.4    1 gated slot of 8      MSE 0.0655
plain GELU      17.2    none                       0.0636
GeGLU (2/3)     23.2    all channels gated         0.0483   (+50% params raw)
HYBRID5 fixed   26.1    2 gated slots of 8         0.0601
HYBRID5 random  34.0    2 gated, random partner    0.0623
```

HYBRID5 is the better result scientifically and the worse design: it tolerates randomization but moves the gradient ceiling the WRONG way, above GeGLU, which is the thing this whole line of work set out to avoid.

Prior art -- the mechanism is Shazeer's, the fractional application is what is not covered. Noam Shazeer, *GLU Variants Improve Transformer*, arXiv:2002.05202 (2020), defines the family this work sits in: GLU with sigmoid, **GEGLU** with GELU on the gate (what mmBERT/ModernBERT use), SwiGLU with Swish, and **Bilinear** -- the variant that omits the nonlinearity entirely and is just the component-wise product of two projections. **HYBRID4's product chunk IS Bilinear**, applied to 1/8 of the channels instead of all of them. That paper is also the origin of the two-thirds `d_ff` rule this entry quotes for parameter matching, and it reports GEGLU/SwiGLU as the best variants.

Every variant there gates ALL hidden units. A search over the obvious phrasings found no published study of gating only a FRACTION of FFN channels with the rest left pointwise. **Treat that as weak evidence, not a novelty claim** -- web search is not a systematic review, the construction is simple enough to be sitting unremarked in someone's ablation appendix, and a negative like the randomisation result is exactly the kind of thing that never gets written up. Adjacent but not the same: Highway Networks (Srivastava et al., 2015) mix a transformed and a carried path under a learned gate; arXiv:2410.08417 studies bilinear MLPs for weight-based interpretability.

Do not over-read the small gaps. Harness is a 4-block residual MLP, D=64, H=192, AdamW 3e-3, 3000 steps, 3-5 seeds, synthetic regression target with multiplicative interactions. Plain GELU alone ranged 0.043-0.072 across runs. This harness separates 0.06 from 0.65 reliably and cannot separate 0.048 from 0.066 at all. Every claim above rests on the first kind of gap. Scripts were scratchpad-only; the recipe here is the record.

One harness bug, corrected mid-study and worth not repeating: applying the expectation blend at eval to a DETERMINISTIC mask is a train/eval mismatch, not a calibration. It reported FIXED variants at 7.54 and

81.1 before the fix; the real numbers are 0.0492 and 0.0702.

3. §10's crux is reopened; §14 does not reproduce

The harder-regime ablation concluded the EKF's edge *widens* under unreliability. On real Helene trajectories the gain is flat and *shrinks* at the hardest setting (+1.8% → +0.8%). The likely reason: §14 measured synthetic streams generated by the same rise/decay dynamics that `est_ekf` models. **A dynamics model validated on data generated from it is not validated.** Either re-derive on real trajectories or drop the claim from the paper. Do not leave it standing as written.

4. "Boundary beats span at 10K" is unverified

It compares against 0.158 from a different experiment whose blind-test support was never checked. A support mismatch (3,527 vs 20,845) already invalidated the cold-base row of this same curve once today. Re-derive on a shared test set before this goes anywhere near Paper 0.

5. The relation regression in the warm start (−0.037, −22% relative)

`task_lr` is 5.0e-4, tuned in the curve for **cold** heads. In the warm start the relation head is already warm and sees only 8% of the mixture — few gradients at a high rate. Test a lower `task_lr`, or a per-head rate. This targets the regression more directly than `encoder_lr`, which acts on the shared trunk. One run, one variable.

7. Still no benchmark that can score the filter

Turkiye's baseline was an oracle by construction — truth read from the sentence the extractor reads, so `est_last_value` scored 0.000. Helene's per-state streams were mis-bound until the scope gate. A real filter benchmark needs **multiple sources that disagree and revise** about one event, which is also the regime where MHT would finally earn its keep.

10. Aggregate SCOPE (not the aggregate constraint) — the sharpened target

Two different things wear the word "aggregate" and only one of them is open.

REOPENED 2026-08-24 — the constraint direction was rejected by the METRIC. `nrmse` normalises each state by its own range and macro-averages, so Virginia (1→2) counts like North Carolina (6→123) and carried 110.5% of the vector arm's excess error. On the same runs the aggregate cuts **national total RMSE at every density** — 87.6→28.5 at 10% reporting, the density where this section says it "loses worst". The mechanism recorded below ("constrains the SUM, says nothing about the SPLIT") is correct; concluding "so it does not pay off" while scoring only the split is not. See `tools/ekf_showcase/vector_state_results/METRIC_INVERTED_THE_VERDICT.md`. The text below is kept as measured, and its verdict is withdrawn.

The constraint direction is measured and it LOSES *on nRMSE*. `vector_state_test.py` feeds the national total in as a sum row over the six state components. Against `parts-only`, on real Wikipedia trajectories with `--q-prop 0.15`:

per-state report rate	parts-only	vector	delta	vector wins
10%	0.4348	0.6085	+0.174	4/40
50%	0.2030	0.2234	+0.020	22/40
80%	0.1556	**0.1520**	**−0.004**	30/40

It loses everywhere except 80% density, and loses **worst exactly where it was predicted to win**. An aggregate constrains the SUM and says nothing about the SPLIT, so when parts are sparse the filter must guess the division and the total injects error.

QUALIFIED 2026-08-24 against a measured noise floor (10 RNG streams x 40 trials, three noise levels -- `vector_state_results/NOISE_FLOOR.md`). The table above is ONE stream at noise 0.10; re-running it reproduces, which is determinism, not variance.

- The rejection **holds where it matters**: at 10-20% density it clears the floor at every noise level. Real feeds are in that regime.
- The **middle rows were never readable**: +0.0568 at 35% and +0.0203 at 50% sit inside floors of 0.0678 and 0.0275. They were quoted as measurements and they are noise.
- At **80% density AND noise 0.20 the vector arm WINS and clears the floor** (-0.0112, spread 0.0069, 350/400). The recorded experiment fixed noise at 0.10 and never looked.

So the constraint is not wrong in general -- it is wrong in the regime we measured and right in one we did not. It stays closed for THIS pipeline because sparse per-state reporting is what the feed provides, but "it was tried and it lost" overstates it: the correct statement is that it loses when parts are sparse, and pays when parts are dense and individually noisy. (Isotropic `Q` makes it 7.7x worse still — proportional process noise is a precondition, not a tuning knob, since Virginia ranges 1→2 while North Carolina ranges 6→123. As of 2026-08-24 `--q-prop 0.15` is the tool's DEFAULT, because the documented trap was the default behaviour and the journal records someone mistaking the isotropic run for a new catastrophic result.)

A candidate reason to revisit, tested and eliminated (2026-08-24). Both Q options are strictly DIAGONAL, which asserts state tolls accrue independently — and the aggregate row `H = [1,1,1,1,1,1]` is exactly where that bites, since `Var(sum) = Σᵢⱼ Pᵢⱼ` gives the off-diagonals a direct say. So the constraint may have been rejected on a process model that cannot represent what the aggregate observes. `--q-rho` adds uniform correlation with the marginals preserved. It does not rescue it — the vector arm gets monotonically worse:

rho	10%	35%	80%
0.0	+0.174 (4/40)	+0.057 (10/40)	**−0.004 (30/40)**
0.3	+0.256 (3/40)	+0.091 (1/40)	+0.011 (9/40)
0.6	+0.286 (5/40)	+0.096 (3/40)	+0.016 (4/40)
0.9	+0.308 (5/40)	+0.098 (5/40)	+0.024 (2/40)

Correlation degrades **parts-only** too (0.4348 → 0.5280 at ρ=0.9), so ρ>0 is simply a worse fit for these trajectories: the real revisions are idiosyncratic — North Carolina's 123→102→96 reclassification is about North Carolina — not common-mode. The rejection now survives isotropic, proportional-diagonal AND correlated Q.

16. Exposure counts are not casualties, and the schema has nowhere to put them

12 of Helene's 106 `dead` observations are audited non-casualty: six are EXPOSURE (300 rescued, 50 patients rescued, 32 evacuated, 11 swept away) and six are UNIT CONFUSION (a two-day period, six states, dozens of vehicles, 1,400 landslides). A rescued person is a counterfactual casualty -- averted harm, not realized harm -- and exposure counts run systematically LARGER than casualty counts, so a mis-bind is the same magnitude as cross-event contamination and points the same way, upward.

Root cause is the schema: `casualty_events` has location/injured/missing/dead and no role for exposure while the prose is full of it. Adding `displaced` and `rescued` fixes it at source. Same diagnosis as 10a reached for `scope`: a data-modelling problem being handled by a gate. Full taxonomy in `ekf_showcase/gate_results/EXPOSURE_VS_CASUALTY.md`.

14. Scope supervision — do NOT buy labels under the current scheme (2026-08-25)

Closes the open question in item 10a. A $2.25 dual-label probe (Haiku 4.5 vs Opus 5, 200 records, `scope_label_probe.py`) says:

- **Haiku is not cleared** -- 83.0% agreement but **kappa 0.121**, which is the number that matters when 86% of the corpus is one class.
- **The model is not the variable.** Hand-adjudicating 14 place<->sub-place disagreements came out **7-7**. `sub_place` conflates a genuinely narrower counting unit ("three districts IN Iraq") with a narrower incident SITE whose number is still the bound place's full toll ("across 2 districts OF Baghdad", "a mine collapse in Xinjiang"). Case two mislabelled makes `apply_extracted_scope` DROP a valid observation.
- **The corpus cannot teach the class that matters.** `national` is 0.0% of the unbiased sample and 0.5% of the hard stratum. `casualty_events` can teach place-vs-sub-place and not place-vs-national -- which is the only distinction the gate exists to arbitrate.

Next: rewrite the question to be about the COUNTING UNIT rather than the incident site, re-probe, and beat kappa 0.121. Separately, find a source of place-vs-national supervision; the 86 hand-audited Helene occurrences are the existing precedent.

Regional disaster profiles as a plausibility prior (idea, 2026-09-04)

Operator's observation, recorded because it converts a measured limitation into an instrument. Every region has a disaster profile. A tsunami in Turkey, or major snowfall in Fiji, is an outlier -- and the outlier is informative in BOTH directions: either a genuinely notable event, or an extraction error. Both are things the pipeline should surface rather than silently pass to a filter that consumes any arriving number as a measurement.

This reframes a result already in hand. The Turkish event-type pilots were reported as having a coverage gap -- 0 Tsunamis, 0 Volcano Eruption, ~199 Earthquakes projected at 30K. That distribution IS Turkey's profile. Turkey has earthquakes, floods, fires, mine collapses and road crashes; it does not have volcanic eruptions. The pilot did not fail to find those types, it measured that they do not occur.

It would make a filter shippable that this page currently says is not a method. Of `plausibility_filter`, GATES.md says: "a hand-set ceiling [that] has to be *told* the event's scale -- which it gets from the answer, so it is not a method." A learned `P(event_type | region)` and `P(magnitude | event_type, region)` takes the scale from the REGION instead of from the answer. That is the difference between an oracle and a filter.

It would catch contaminants nothing currently can. The Turkish feed quotes Haiti 2010 (316 bin = 316,000 dead) and Antakya 115/525 AD (260,000 / 250,000) as bare figures that the extractor binds CORRECTLY as `dead` -- they are death tolls. Today only `Date` can reject them, and Date is the weakest stage-1 field in Turkish (0.26 non-empty). A region prior says 316,000 deaths in a Turkish-datelined earthquake is ~3 orders of magnitude outside profile. Same shape as the 1999 Izmit 17,500 contaminant on the English feed.

Related measurement already recorded. `tools/data/notes/CHINESE_TOLL_DISTRIBUTION.md` found Chinese DOMESTIC tolls run smaller than the same outlets' FOREIGN reporting (median 17 vs 26; >=100 at 15.3% vs 28.4%). That is a reporting-regime profile, and treating it as a prior is more honest than treating it as noise -- the note already warns that the EKF and `max_plausible` both encode magnitude expectations and will be pushed downward by it.

The cheap version needs no new data. DocEE-en (21,842), DocEE-zh (36,729, still unconverted) and the ~30K Turkish buy are all in ONE label space and all carry Location. Three regional profiles fall out of data already bought. Cost is analysis time.

Cautions before building it. (1) A profile learned from news measures REPORTING, not occurrence -- the Chinese note's own selection-vs-suppression ambiguity applies here too. (2) It must flag, not veto: the whole point is that a real outlier and an extraction error look identical to the prior, so a hard reject would discard exactly the rare events the tracker exists for. (3) Scored on catch rate AND false-reject rate together, per the standing gate-1 lesson that a form gate scored on firings alone rewards indiscriminate firing.

The flag-not-veto MECHANISM already exists, twice, and both are switched off

Caution (2) is not an intention to be remembered -- the plumbing is built. Nothing new has to be invented, and the two mechanisms fail differently, so use both.

(a) `CONF_R`, per observation, continuous. The filter already implements `R /= confidence**2`, so a weight of 0.1 inflates that reading's noise **100x** (`imm_gate_sweep.py`). That IS flag-not-veto: a veto DROPS the observation and destroys the information; inflating `R` keeps it in the update while shrinking the Kalman gain `K = P/(P+R)`, so it moves the estimate less.

The property that makes this right for outliers: **a real rare event REPEATS, an extraction error does not.** With `R` inflated 100x a single Haiti-shaped 316,000 barely nudges the filter, but a genuine Turkish tsunami

reported five times applies five (down-weighted) pulls and the filter converges there anyway. The prior does not decide -- it buys time for corroboration to decide. A veto forecloses that on the first sighting.

`CONF_R` is OFF because the only signal available to feed it was extractor confidence, which was measured unreliable -- confidence "did not filter the mis-binds" on Venezuela. A regional prior is a better-grounded number for the SAME plumbing:

```
R  <-  R * exp(lambda * surprisal),   surprisal = -log P(value | event_type, region)
```

(b) `hmm_gate` feature, global, able to revisit. The decode already enforces this structurally rather than by convention -- the measured design rule is "keep every feature weight BELOW `reject_cost`, so no single feature can force a reject on its own; it can only tip a case magnitude has already made marginal. The sweep shows a cliff exactly at that boundary." A regional-profile feature enters beside out-of-window dates and out-of-hierarchy places, weight < 4.0, and ARGUES.

Why both. `hmm_gate` is a global decode that can revisit and routes to `own` / `aggregate` / `reject`; `CONF_R` is per-observation and continuous. A figure that belongs to a DIFFERENT STREAM is a gate problem. A figure that is right-stream but wrong-MAGNITUDE is an `R` problem. One does not cover the other.

The failure this avoids has been paid for here twice. `value_qualifier` returned `0` for an unparsable span and manufactured 30 fabricated zeros out of 114 Helene observations; the fix was returning `None` so "no number here" stays distinguishable from "the number is zero". A hard veto on prior-implausibility is the same mistake wearing a Bayesian hat -- it erases the tsunami-in-Turkey the tracker exists to catch, and leaves no trace that it did.

Open after 2026-09-18 -- recovering the negatives blind test

- **Option 4 (role -> NER label) is REQUESTED and unbuilt.** Step 1 landed (884d25d: a corpus can be PARTIAL, positives for a dimension and never negatives). Step 2 -- `tools/data/roles_to_entities.py` -- does not exist. The adjudication is written up in `OPTION_4_ROLE_TO_ENTITY.md`: map DIRECTLY to NER labels rather than emitting role names and remapping, because a derived corpus is new data we author, so no source corpus is rewritten and the `labels_file` indirection is not bent. Usable supply is **24.3%** (15,188 of 62,573), not 100%. Run it after the negatives verdict.
- **The strongest evidence for Option 4's thesis is a RECALL measurement, and the negatives run measures PRECISION.** On 50 cmnee docs / 583 argument mentions the base model does not PROPOSE the gold argument spans at all -- Subject 96% never predicted, Location 71%, Date 67% -- and where it does predict it agrees exactly. Negatives cannot touch that; they are a precision intervention. Do not read one as evidence about the other.
- **`extract_events` decodes nothing -- but on `eb16-rebuild-tr`, NOT the event model.** CORRECTED 2026-09-18. The probe ran against `/Volumes/Development/tmp/ckpt_incumbent`, which sha256 proves is `eb16_rebuild_tr` (`e1272b7b...`), NOT `gliner2-eb16-eventrecords-tr` (`84a9918f...`). Its config carries `boundary_head.event_records: False`, so a model whose record head was never trained on events returning `{"Earthquake": []}` is close to expected, and

the "scores 0.8156" figure belongs to a DIFFERENT checkpoint. The identity was assumed from byte size -- all three checkpoints are exactly 1,257,024,736 bytes, so size distinguishes nothing. THE OPEN QUESTION IS NOW UNTESTED, not answered: re-run against an `event_records: True` checkpoint on CUDA before claiming anything about the convenience API.

- **Checkpoint directories must be identified by sha256, never by name or size.** Three eb16 checkpoints share a byte size to the digit. Hub-recorded sha256: `eb16-eventrecords-neg` 25d05a3d, `eb16-eventrecords-tr` 84a9918f, `eb16-rebuild-tr` e1272b7b.
- **A `<split>_metrics.json` now records its operating point** (`eval_provenance` : threshold, full_menu, split, records, checkpoint, config), and `compare_runs.py` refuses across differing operating points. Files written BEFORE 2026-09-18 have no provenance; for those the threshold must still be recovered from the runner that produced them -- `negatives_ab/rebase-*.json` is `--threshold 0.3 --full-menu` , from `negatives_ab.sh:43` .

Open after 2026-09-18 -- the negatives verdict and the loss-commingling hypothesis

- **NEGATIVES: hit the target, broke an untargeted head. Do not ship.** Like-for-like at threshold 0.3, full menu, 18,786 records (both files carry matching `eval_provenance`): `event_argument` strict **+0.0322** (0.1497 -> 0.1819, +21.5% relative, outside the floor) -- and visible ONLY under the full menu; gold reads -0.0014. Cost: `classification` **-0.1492** (7x the floor, precision AND recall both down, so not a trade). Aggregate full-menu selection score -0.0024, INSIDE the floor -- a wash that hides entity +913 and event_argument +671 being cancelled by classification -1535 (support x delta).
- **LOSS COMMINGLING IS THE LEADING HYPOTHESIS, and it is testable locally for free.** There are no per-task losses: the boundary terms are per-MECHANISM (start/end/pair/inside/soft_iou/rerank/proposal/consistency/null/count) and entities, relations, events and json_structures all flow through them over ONE pooled query axis. `_reduce(mode="global")` (losses.py:167-170) divides by `keep_f.sum()` pooled across every task in the batch, and its own docstring states the consequence: "scaling positives by k multiplies a task's contribution by (k*pos + neg)/(pos + neg)". Injecting entity/event negatives adds queries to that denominator. Classification cannot compensate: it has a SEPARATE head and loss (`_classification_loss` , model.py:1761) with one global `classification_loss_weight` , and `TASK_TYPES = ("entities","relations","events","json_structures")` does NOT include it, so `task_loss_weights` has no lever for it. **THE TEST: one batch, built with and without negative injection, printing `keep_f.sum()` and the per-task contributions.** No GPU rental. Do this before any repair.
- **Candidate repairs, once the mechanism is confirmed:** (1) `reduction="per_query"` , already implemented at losses.py:171-176, removes the pooled coupling; (2) add `classifications` to `TASK_TYPES` so it can be weighted at all; (3) raise `classification_loss_weight` (crudest, and tuning on one run fits noise); (4) **negatives on events only** -- entity negatives bought +0.0116, inside the floor, so they may be pure cost while event_argument keeps the win.

- **The negatives A/B (`negatives_ab/{control,treatment}.json`) CANNOT corroborate this.** It scores no classification or structure at all, and its other heads are near-zero (entity 0.0147, relation 0.0000, event_argument 0.0000) -- barely-trained smoke arms, not a comparable measurement. The -0.1492 rests on a single run.
- **Long-document test pool is BUILT and SAVED:** `whr778/cc_news_long` (PRIVATE, 764 docs, unannotated). All 764 exceed 4,096 tokens, 98.3% span 2+ windows, 50% span 3+, 24.2% span 4+; median 9,274 tokens, max 38,412 against an encoder limit of 8,192. Harvested by `tools/data/harvest_long_ccnews.py` from all 708,241 CC-News documents. Yield was 764, not the ~513 extrapolated -- the first 40,000 in stream order under-sample the long tail. BEFORE ANNOTATING, three caps must rise: `fetch_cc_news.py --max-chars 12000`, the annotators' `MAX_CHARS` (4000-6000), and `max_tokens` (900-3000, which would TRUNCATE a 38,412-token document's annotation into invalid JSON). Cost ~$7 in batch mode for all 764; pilot 10 first and measure verbatim-span match rate against length, because a silently thinned gold set would make the sliding window look worse than it is.

2026-09-18 -- the denominator test RAN, and it REFUTED the dilution hypothesis

MEASURED, on 4,960 records sampled across 124 train files, with the real injector (`NegativeLabels.load(negative_pools.json, {entities: 1, events: 1}, seed=42)`):

dimension	before	after	growth
entities	13,214	13,701	+3.7%
events	3,377	3,555	+5.3%
relations / classifications / structures	—	—	**0.0%**
TOTAL	23,310	23,975	**+2.9%**

Records whose query set changed: **12.6%**. `no_candidate` fired on **82.5%** -- most records have no corpus pool containing all their gold labels, so they are never injected.

A +2.9% change in the pooled `keep_f.sum()` denominator cannot produce a -0.1492 (-18.4% relative) classification collapse. The magnitudes differ by an order of magnitude, and the naive dilution story also predicts the WRONG SIGN: a bigger denominator with near-zero added numerator LOWERS the boundary mean, which would raise classification's relative share and HELP it. Hypothesis dead. Do not resurrect it without new evidence.

The surviving explanation: labels are an INPUT, and injection concentrates on the classification corpora

Injection is not uniform -- it is near-universal exactly where classification lives: docee **100%** injected (120/120 have classifications), chfinann **100%** (120/120), turkish_event 82.5% (120/120), docfee 56.7%, docee_zh 50.0%. Across corpora that receive any injection, **684 of 1,032 classification-bearing records**

(66%) were perturbed -- against the 12.6% global rate. The global figure is diluted by corpora that get nothing.

And the perturbation reaches classification directly, not through the loss. Schema and text are encoded in ONE sequence: `processor.extract_embeddings_from_batch` gathers BOTH `all_token_embs` and `all_schema_embs` out of the same `token_embeddings` via `schema_special_indices` / `text_word_indices`, and classification logits are built from `all_schema_embs[i][g]` (model.py:1621). So an injected entity or event label changes the encoded context for the classification group's own schema tokens. Labels are an INPUT to GLiNER2 -- that is the standing rule in CLAUDE.md, and this is it biting.

Plus a train/eval mismatch: training offers a menu carrying negatives on 66% of classification records; eval offers the gold menu with none. The classification head is trained in one input distribution and scored in another.

STILL TO PROVE (next increment, local and free): tokenize one docee/chfinann record with and without injection and show the input sequence and the classification group's schema positions actually move. Code reading says they must; measure it before claiming it.

Repair candidates, re-ranked by this evidence: (1) negatives on EVENTS only -- entity negatives bought +0.0116, inside the floor, and entities are the bulk of the injection; (2) exclude classification-bearing corpora from injection (`build_negative_pools.py` already has a partial-corpus flag, 884d25d); (3) `loss_reduction: per_query` is now UNMOTIVATED as a fix for this -- no config in the repo has ever set it, but the dilution it would fix is not what is happening.

2026-09-18 (later) -- BOTH mechanisms refuted, and the verdict is downgraded

RETRACTED: "injection moves the classification token positions." That claim came from a single draw per arm (1306 -> 1801 tokens, cls markers 7 -> 514). It was NOISE. The control that should have been run first -- the SAME schema collated six times, nothing injected -- gives seq_len 1149-1884 and classification choices 2-59. An arm was read without a control, again. Pinning `random` / `torch` / `numpy` does NOT isolate it either, because injecting a label changes the RNG CONSUMPTION ORDER, so every downstream draw diverges.

Properly controlled (N=150 draws per arm, one docee record):

	no-neg	**with-neg**	**delta**
seq_len	1386.8 +/- 245.6	1403.6 +/- 256.6	+16.9 (~1.2%)
cls choices	25.63 +/- 22.73	24.37 +/- 22.97	-1.26, SE 2.64, **|t|=0.48**

Injection adds about one label's worth of tokens and leaves classification's presentation statistically unchanged. **So "labels-as-input perturbs classification" is REFUTED, alongside denominator dilution.** Neither mechanism survives contact with a control.

THE REAL FINDING: classification supervision is half-degenerate by design

`SamplingConfig` (processor.py:271) augments the classification menu hard. CORRECTED 2026-09-18: an earlier note here said "mean 25.6 labels, sd 22.7, only 2 labels 46% of the time". That was an INDEXING

BUG -- it read `structure_labels[0][1]`, and group order varies, so index 1 sometimes pointed at the ENTITIES group and returned 2. Measured properly over 400 presentations of one docee record (60-label task):

	rate
synthetic `label N` naming (all semantics stripped)	**48.0%**
true label ABSENT from the menu	**13.9%**
labels offered	mean **45.0**, sd 8.5, range 31-61

`synthetic_label_prob: 0.5` renames every label to `label 1..N` and remaps descriptions and few-shot examples with it (processor.py:1315-1324); `synthetic_entity_label_prob: 0.2` does the same for entities (`entity 1`). `include_true_label_prob: 0.5` is a RESCUE, not an inclusion rate -- the true label is re-appended only if dropping removed it, so absence needs a drop AND a failed rescue.

So classification supervision is NOT half-degenerate, and the claim that its floor must therefore be huge is weaker than stated. What is real: 48% of training presentations carry no label semantics at all and 14% have no correct answer, while EVAL always gives real names and always includes the true label. That asymmetry is by design, identical in both arms, and still unquantified as a source of run-to-run variance.

CONSEQUENCE: the -0.1492 is UNATTRIBUTED, and the "not shippable" verdict is DOWNGRADED. The +/-0.02 floor this programme quotes was measured on other heads. Classification's own run-to-run floor has NEVER been measured, and 46%-degenerate supervision predicts a large one. -0.1492 may sit inside it.

DO NOT PIVOT on this evidence. "Negatives on events only" has no support: it rests entirely on a classification loss that may be noise. What stands is `event_argument` +0.0322, which is real, targeted, and outside its floor.

NEXT TEST, and it must come before any repair: measure classification's run-to-run floor. Cheapest honest version is to re-run the CONTROL arm twice under different seeds and read the spread on `classification_strict`; a negatives arm at a second `negative_label_seed` answers a different (also useful) question. Until that number exists, no classification delta on this programme is interpretable.

2026-09-18 -- THE NEGATIVES INJECT FALSE NEGATIVES, and it explains the measured trade

The mechanical part is correct: an injected negative carries an EMPTY surface list (0 of 1,416 on docee carried a surface). Absent by construction, which is the point.

The risk is whether the label is absent from the TEXT, not merely from the GOLD. Probed on 1,500 docee documents: build surface -> label from docee's OWN gold, then ask whether a document injected with "absent" label L contains a surface known to carry L elsewhere in the same corpus. Strict matcher (surface >= 5 chars, non-numeric, word-boundary): **313 / 874 = 35.8%.** `Location` injected as absent while the text says *Africa*; `Affected Area` absent while the text says *France*, *Singapore*; `Discover Time` absent while the text says *Saturday*, *March*.

35.8% is an UPPER BOUND, not the rate. The probe over-fires: `Archaeologist Organization` fires on *Spain*, and docee's entity gold is event-argument-derived, so a passing mention of Africa may legitimately not be a labelled `Location` FOR THAT EVENT. A loose matcher gave 62%; the difference between the two is a measure of how much the instrument, not the data, is talking. Do not quote either number as the false-negative rate without a tighter test.

THE GUARD IS TOO WEAK, and this is the actionable part. `_usable_pool` vetoes a dimension unless every candidate corpus ANNOTATES it:

```
if not (spec.get("annotates") or {}).get(dim):
    return set()
```

That tests annotation of a DIMENSION, never exhaustive annotation of a LABEL. docee annotates entities -- but only event-relevant ones, leaving most mentions untagged -- so "in the pool, not in this record's gold" does NOT imply "absent from the text". The within-dimension rule ("absence means not labelled, not not present") is the rule the partial-annotation flag exists to protect, and the negatives mechanism is breaking it from the inside.

IT PREDICTS THE MEASURED SIGNATURE. False negatives teach suppression of correct spans: precision up, recall down, F1 flat. Measured on event_argument at threshold 0.3: precision **0.3797 -> 0.4685**, recall **0.2437 -> 0.2157**, F1 -0.0015. A one-for-one trade is what suppression looks like; better discrimination would have moved F1.

AND THE AUGMENTATION DILUTES THE REST. `synthetic_entity_label_prob: 0.2` renames EVERY entity key including injected negatives, measured at **20.7%** of presentations, and an injected negative has no description to fall back on. So ~1 in 5 negatives teaches "entity 7 is offered and absent" -- true, and non-transferable. Effective dose is ~0.79 per record, not 1. No LEAK, though: 0 of 500 records in every injected corpus carry `entity_descriptions`, so "has no description" does not distinguish a negative from a real label.

NEXT: tighten the probe (require the surface to be tagged L in a document of the SAME event type, or hand-adjudicate a sample of 50) to turn the upper bound into a rate. If it is materially above zero, the fix is in `_usable_pool`: draw negatives only from labels a corpus annotates EXHAUSTIVELY, which is a stronger property than `annotates[dim]` and is not currently recorded by `build_negative_pools.py`.

2026-09-18 (later still) -- the tightened probe REFUTES the false-negative explanation

Tightening the probe on 4,000 docee documents, 2,659 testable injected negatives:

probe	rate
loose substring (as first reported)	35.8%
+ surface UNAMBIGUOUS corpus-wide and seen >= 2	**5.6%**
+ surface carried that label under THIS document's EVENT TYPE	**0.6%**

Hand-adjudicating the 12 survivors: ~5 genuine (`Archaeologist` injected absent while the text says *Archaeologists*; `Date` absent while the text says *in July*; `Location` absent in a Sports Competition naming

Indianapolis Motor Speedway), ~4 clear probe errors (`Historical Sites` firing on *fossils*, `Affected Area` on *world*), ~3 marginal. Probe precision ~40-65%, so the true rate is **~0.25-0.6%**.

REAL, BUT TWO ORDERS OF MAGNITUDE TOO SMALL to explain a precision shift of +0.089. The claim that the negatives break the within-dimension rule "from the inside" does NOT survive its own tightened test. `_usable_pool` is still weak in principle -- `annotates[dim]` is a boolean over the whole corpus -- but it is not causing measurable harm, so building the surface-index guard is NOT justified by evidence.

AND NO DEFECT IS NEEDED. Negatives are SUPPOSED to make the model conservative; precision up and recall down is the designed behaviour of absent-label supervision, not a symptom. Under the gold menu that trade is F1-neutral; under the FULL menu, where suppressing wrong labels is the whole task, `event_argument` F1 rose +0.0322. Three mechanisms were proposed and refuted today (denominator dilution, labels-as-input perturbation, false negatives) before the parsimonious reading was considered. **Measure before proposing a mechanism.**

2026-09-18 -- classification's floor MEASURED from data we already had; verdict REINSTATED

The "-0.1492 may be inside an unmeasured floor" downgrade was WRONG, and it should never have been written: it rested on a speculated large floor, which in turn rested on the indexing bug already retracted above. Two independent estimates existed in runs this programme had already paid for.

source	regime	classification floor
gate3 warm cells, seeds s1/s4/s21	warm-cell fine-tune	sd **0.0178**, range 0.0400
clean re-baseline, seed42 vs seed43	warm-start	**0.0013**

Seed-to-seed sd by head (gate3, same config, same 18,786-record test set):

head	sd	our delta	sigma
classification	0.0178	**-0.1492**	**8.4**
event_argument	0.0009	**+0.0322**	**36**
event_type	0.0395	-0.0154	0.4 (inside)
relation	0.0110	+0.0007	0.1 (inside)
structure	0.0025	-0.0274	11

BOTH EFFECTS ARE REAL. `event_argument` +0.0322 and `classification` -0.1492 are far outside any floor measured for those heads, in either regime. `structure` -0.0274 is also real (11 sd). `event_type` and `relation` are inside.

VERDICT REINSTATED: do not ship the negatives as configured. The mechanism remains unexplained -- denominator dilution, labels-as-input perturbation and false negatives were each proposed and refuted today -- but the EFFECT is not in doubt. An unexplained real effect is a reason to investigate, not a reason to discount.

PROCESS LESSON, the expensive one from this day: look for seed replicates already in hand BEFORE speculating about a floor. Three mechanisms were hypothesised and refuted, and a verdict was downgraded and reinstated, when the number that settled it was sitting in two prior runs.

Buy entity gold for cmnee and duee -- the only thing that lifts option 2 past 35.7%

Priced 2026-09-19, not started. A role-type map can only be derived from, or applied to, documents carrying BOTH entity and event gold. Measured over the eb16 mix: cmnee 25,679 arguments at **0%** entity-bearing, duee 6,757 at **0%**, casie 17,992 at 100% -- so **17,992 of 50,428 (35.7%)** are reachable and casie is the only source. That is option 2's ceiling, and no amount of work on the map moves it.

OPTION 4 DOES NOT SUBSTITUTE. Its labels are `Event<Role>`, derived from the roles, so typing a role by them is circular; the canonical mode (`Date`->Date) is tautological the same way. Only INDEPENDENT entity annotation unlocks it.

Priced from the corpora, Haiku 4.5 Batch API (-50%), entity-only prompt:

plan	input	output	total
cmnee FULL, 13,611 docs @ ~243 tok	$4.38	$8.51	**$12.88**
duee FULL, 13,056 docs @ ~40 tok, 10 per request	$0.52	$3.26	**$3.79**
both, full			**$16.67**
cmnee sample 2,000 -- enough for the MAP alone	$0.64	$1.25	$1.89
duee sample 2,000 -- enough for the MAP alone	$0.08	$0.50	$0.58
both, map-only sample			**$2.47**

The two buys answer different questions. The $2.47 sample only lets a map be DERIVED for cmnee/duee event types -- it does not let the constraint be APPLIED, because applying it needs an entity type on the document being decoded. The $16.67 full buy does both, and its real value is larger than option 2: **it is genuine entity supervision on 26,667 documents that have none**, which is what option 4 is currently manufacturing from roles. If the option 4 A/B is positive, this is the honest version of the same lever; if it is negative, this tests whether derived labels were the problem rather than the idea.

Note duee's shape before writing the prompt: median 32 characters, mean 54 -- these are SENTENCES, not articles. One document per request wastes ~400 prompt tokens on a 40-token input, which is why the amortised row is a third of the naive one. Batch ~10 per request.

2026-09-22 -- OneIE read from SOURCE, and two wrong turns taken getting there

The settled position lives in [[EVENT_ARGUMENT_DIAGNOSIS]] §4c-i. This is the path, because two of the three steps were errors and the errors share one cause.

1. Read from the paper (09-15). §4c concluded the answer had two halves -- architectural and metric -- "of which the second matters more to us". The metric claim was right: OneIE's Arg-C requires offsets + event type + role, and not the specific trigger.

2. Read from the source (09-22), which inverted the emphasis. Prompted by the question "do their arguments have their own loss?". They do: `role_criteria(role_type_scores, batch.role_type_idxs)`, a cross-entropy over (trigger, entity) CANDIDATE PAIRS with a null role, summed unweighted beside four siblings. And `graph.py` represents an argument as an edge to a specific trigger node. So OneIE trains the binding its own scorer then discards. The architectural half, not the metric half, is the live one.

3. Over-corrected within the hour, and that was the real error. From (2) I wrote that "NOTHING in our boundary loss optimises that edge" -- reasoning from OneIE's loss and our `candidate_pair_loss`, which pairs a span's start with its end. It went into three working papers, a TODO row, a catalog row and a commit message before anyone read our OWN record loss. It is false: `record_loss_weight` defaults to 1.0 and `compute_group_loss` seeds an instance FROM THE ANCHOR, supervising field losses into it. We train the binding too, in a different form -- anchor-seeded instance assignment rather than pairwise edge classification.

What the correction bought, which is more than the correction cost. Once the binding was known to be trained, the interesting question moved: the 0.1178 strict / 0.5783 relaxed spread everyone quotes was measured WITHOUT `event_records`, on a record head that had never seen an event and therefore had no event binding objective at all. That is a sufficient explanation for the spread and says nothing about the architecture eb17 trains. TODO 16 was rewritten from "add an edge objective" to "measure whether the objective we already have works", which is a much cheaper question and one eb17 was already bought to answer.

The standing lesson. Both wrong turns came from reasoning about one codebase while reading another. The tidy formulation -- "we measure a binding we never train" -- is exactly the kind that should be checked against the code before it is written down, and it was written down three times first. It joins the rerank double-count, the hardcoded `ABSENT queries: 0`, and the menu gate that read the wrong tree: a claim that sounds explanatory is the one most worth verifying.

2026-10-08 to 2026-10-14 -- three supervision bugs found, the v2.1.0 merge, and eb21

Our own defects, found by tracing: - Gold entity labels were always first in training, so models learned position, not labels (entity F1 0.000 vs 0.646 by order alone; LABEL_SPACE_COLLAPSE 3d). - Injected negatives never resampled across epochs, because persistent forked workers never saw `set_epoch` (LABEL_NEGATIVES_PLAN correction).

Upstream's fixes we carried: v2.1.0 fixed the classification augmentation that trained the gold label as a negative. On eb21's corpora that was 22.6% of classification views, in every model trained so far.

The merge: - 13 conflicted files. - Three merge defects found only by tracing real outputs: - span models lost every event, because upstream's new batched path did not know our `[V]` marker; - `Schema.from_dict` refused the `open_vocab` key every shipped schema carries; - our engine was missing upstream's relation-confidence change. - The decoder-sync speedup is deferred: its candidate-grouping shortcut would empty our event decode. - Validated locally (3,475 tests) and on a CUDA A10.

Dose experiments (MENU_SPEC s9): - A final dose-20 stage buys news-menu precision. - Gold-menu recall must be learned during the main run, at a dose above 1. - eb21 keeps dose 20 throughout.

eb19 was judged broken; eb20 was skipped. eb21, from scratch on the merge, is the base going forward.

2026-10-15 to 2026-10-17 -- coref that learns news, a split leak, and a poison found in our own negatives

Coreference (COREF_DATA_SOURCES, "What the data buys, trained"): - The bought LLM news coref taught the entity-coref link news pronouns: corefC WikiEvents pronoun AUC 0.811 vs 0.685, gain +0.127 with CI [+0.088, +0.179]. - GUM/LitBank/GAP were dropped. - On the blind test the link beats string match (pair F1 0.598 vs 0.437). - End to end, mention detection caps it: 30% of gold coreferent mentions are never proposed.

A split leak, found by the coref eval purchase: - WikiEvents test #10 is 92% inside a train document. - `dedupe_splits.py --near-dup` now drops the train copy. - eb21 and corefC trained before the fix.

Production menus (MENU_SPEC s10): - `build_schema.py` makes one shared menu per data domain. - The checkpoint's full menu OOMs a 40 GB GPU in the relation decode. - Under CASIE's own menu, every model fired no CASIE events.

The poison was ours. One rarely argued role, Damage_Amount, switched off its event type's trigger query. - Deep tracing found the cause in the label-negatives injector, not in the label space (LABEL_NEGATIVES_PLAN s5g): absent types listed full role lists, gold types only argued roles. - The dose that cured over-firing (MENU_SPEC s8) amplified it. - A first explanation, a cross-corpus "Damage" word collision, was string matching. The tokenizer refuted it: `_Damage` occurs only in CASIE's role. It was corrected in every place it had been written down. - The fix, `negative_gold_full_roles`, is built and traced. The rolesym A/B tests it, with pre-registered VAL reads. Result (2026-10-18): news55 (test, once per arm) event_type F1 0.153 -> 0.541 with precision UP (0.414 -> 0.460), trigger 0.069 -> 0.397, argument 0.007 -> 0.115; CASIE val full menu 2 -> 72 hits (gold menu 60 -> 47); WikiEvents dev full menu 48 -> 97 hits but 3.5x the firing; blind-test heads all inside the floor.

The standing lesson. The gold-menu eval could not fail on this defect, because it shows a gold type exactly as training does. A menu shaped like production surfaced it in one run.

2026-10-18 to 2026-10-24 -- the eb22 series, a trace era, and a per-head readout

Rows for every run are in [[EXPERIMENT_CATALOG]] (2026-10-18 .. 2026-10-24); this is the thread.

- **rolesym A/B POSITIVE (10-18):** `negative_gold_full_roles` removes the menu poison; news55 event_type F1 0.153 -> 0.541 with precision up. eb22 was to be eb21 + rolesym + corefC.
- **eb22 CRASHED (10-19)** at step 5,779: coref clusters matched by text instead of position (fixed 590d038). Not restarted; re-specified as **option B** from scratch with fastino's structural flags (`eb22b`, smoke clean), then **eb22c** = eb22b + offset-anchored event gold + unified-full-v4.
- **eb22c's events swing (10-20..21).** Two launches lost to infrastructure (a dead startup, then cuDNN SDPA recompiling per sequence length on H100, 2.9 samples/s); once running, events fell after the LR peak. The autopsy ([[EB22C_SCORE_LEVEL_AUTOPSY]]) located it to the score LEVEL: gold triggers stay top-ranked while every score slides across the gate; gradient conflict between entity and event supervision was measured and NOT found; the control run to 30k shows the events oscillate

~15k steps and then climb. Along the way: DDP had never averaged gradients (pytree vs the output dataclass; fixed 0b21629), and the warm-start resume handed Adam moments to the wrong parameters (fixed by name, 21088c7).

- **The trace era (10-20..24):** one-change arms on the bce trace base, each stopped at the epoch boundary and read against a three-control floor at matched steps: seed43, sep, encfix, colfix, geomgelu (NULL), tasklr4e5, corefdist (the distance MLP; inside the floor on CASIE, +0.110 CMNEE at 16k; **resumed to the end of epoch 2 and QUALIFIES**, 4/4 CASIE and 3/4 CMNEE matched steps above the band), relfix (the two relation leaks fixed; **NEGATIVE**), clsfix (classification sampling off; **NULL**). The code review that fed the arms is [[20261022_CODE_REVIEW]] (Issues 1-10).
- **fg-GEGLU** ([[FG_GEGLU]]): the user's activation, built, tested, swept on a toy, null on the classifier head; set aside.
- **eb22d proposed (10-24, not launched):** eb22c + tier-1 correctness fixes + `coref_link_distance: mlp`; two decisions with the user. Before the launch, the **per-head VAL threshold readout** was built: one threshold per decode head from the sweep already run, the blind test scored beside the global pick, and `inference_defaults` now records the swept threshold (METRICS.md "Per-head thresholds").

2026-10-26 to 2026-11-06 -- from `eb22d` to `eb22g`: solving role-assignment and proposal starvation (`PAPER_3_DOCUMENT_EVENTS_AND_COREF.md`)

Rows for every run are in [[EXPERIMENT_CATALOG]] (2026-10-26 .. 2026-11-06); the full synthesis is in [[PAPER_3_DOCUMENT_EVENTS_AND_COREF]].

- **`eb22d` Epochs 1 & 2 (`10-26..27`):** `eb22d-trace-epoch` (1x A100) and `eb22d-trace-h100` (1x H100) trained Epochs 1 and 2 of the `eb22d` recipe (`boundary_marginal_loss: bce`, `coref_link_distance: mlp` width 64). On full validation (`30,320` records) at threshold `0.3` (`eb22d-epoch2-read`), `event_trigger` strict micro-F1 rose from **`0.1334 -> 0.3645`** and `event_cluster` rose from **`0.1300 -> 0.3660`**, while `event_argument` moved only **`0.0442 -> 0.0627`** (inside the `±0.02` single-run floor, with strict recall pinned at `3.5%`). Trigger-coref merge calibration failed its precision/fusion gate (`0.640-0.675` precision vs `>= 0.80` required) and stayed off.
- **`life-mdeberta-wikievents` (`10-28`):** Trained the `eb22d` boundary/record head on `microsoft/mdeberta-v3-base` (`512` window, offset gold) for the full 30-epoch cosine (`4,410` steps) on WikiEvents. Loss descended cleanly (`5.0003 -> 1.6020`), yet WikiEvents dev `event_argument` strict micro-F1 at `@0.1` reached only **`0.0107`** (`entity 0.3746`, `event_trigger 0.1487`), confirming the bottleneck was in the head/loss mechanics rather than the `mmBERT` backbone or training length.
- **`eb22e-epoch` and `eb22e-wikievents-eval` (`11-02`):** Tested zero-initializing the post-`eb21` span residual terms (`span_residual_zero_init: true`) and disabling rotary (`enable_rotary_endpoints: false`) for 1 epoch on 1x H100 (`whr778/gliner2-eb22e-epoch`). At swept threshold `0.1`, `entity` rose **`0.3617 -> 0.4262`** (`+0.0645`) and `structure` rose **`0.1691 -> 0.2231`** (`+0.0540`) over `eb22d` Epoch 1, while `event_trigger` stayed at `0.1608` and

`event_argument` dropped to `0.0755`. At flat threshold `0.5` on WikiEvents test, every event head scored `0.0000`.

- **The paired `eb22e` role-candidate probe (`11-03`):** `probe_role_candidates.py` collated each validation batch **once** (`3,568` gold role fillers across `casie`, `cmnee`, `duee`, `wikievents.dev`) and ran two forwards on `eb22e:checkpoint-epoch-1` — first under Epoch 1's scheduled gold injection (`0.9559`), then with gold injection off (`0.0`, inference). The probe exposed **two simultaneous starvations**: 1. Under `0.9559` injection, **`97.1%` of gold role fillers were in the candidate pool (`3,466 / 3,568`), yet their median assignment probability was `0.0155`** (only `2.1% >= 0.5`), because `_list_field_bce` averaged 1 positive with `K=8` hard negatives in one `(1+K)` denominator (equilibrium `P = 1/9 = 0.111`). 2. With injection off (`0.0`), **`61.7%` of gold role fillers were missing** (`28.8%` span not in pool, `32.9%` trigger not proposed — concentrated in English multi-paragraph `casie` `34.8%` candidate and `wikievents` `30.0%` candidate).
- **Solving role-assignment starvation (`overfit-events-eb22e -> eb22f-epoch`, `11-03`):** Splitting the positive and hard-negative BCE means in `_list_field_bce` (`pos_loss.mean() + neg_loss.mean()`), withholding unresolved non-empty gold roles instead of training them toward `0.0`, masking candidates overlapping unresolved gold spans on partial matches (`_unresolved_overlap_cols`), dropping the trigger anchor self-row (`record_drop_anchor_field_loss: true`), and storing assignment probability as field confidence in `engine.py` lifted 64-doc × 400-step overfit (`overfit-events-eb22e`, 1x A10) median gold role assignment probability from **`0.0316 -> 0.5545`** on CASIE and **`0.5078 -> 0.9196`** on CMNEE, and lifted `@0.5` `event_argument` strict micro-F1 by **`+0.055`** on CASIE (`0.1389 -> 0.1938`) and **`+0.266`** on CMNEE (`0.3119 -> 0.5778`). Running 1 full epoch on 4x H100 SXM5 (`eb22f-epoch`, auto-scaled in `trace_job.sh` to `batch_size=4, grad_accum=1` per GPU, `[ddp-sync]` verified identical across all 4 ranks) lifted `entity` to **`0.4679`** (`+0.042` over `eb22e`) and `relation` to **`0.1665`** (`+0.034`), while Epoch-1 `event_argument` stayed at `0.0767` (vs `0.0755`) — confirming the probe's prediction that proposal starvation gated Epoch 1.
- **The 7-finding events pipeline audit (`0607b29`, `11-04`):** Tracing `train.py`, `eval_metrics.py`, `processor.py`, `boundary_preprocessing.py`, `argument_coref.py`, and `losses.py` uncovered why the boundary proposer was starving: 1. **Absent-query negative dominance in `candidate_pair_loss` (`losses.py`):** Under `k=20` negative sampling, 85–90% of queries have zero positives; `hard_negative_keep_all_when_absent: true` kept all 768 candidates on every absent query in one global denominator `total_mask.sum()`, diluting positive `pair_loss` gradients by 7,700× (`0.0492 -> 5.0553` and positive logit gradient `-0.000129 -> -0.993307` once fixed with `pair_split_reduction: true` + `hard_negative_keep_all_when_absent: false`). **This also solved the `eb22c` score-level swing mystery (`EB22C_SCORE_LEVEL_AUTOPSY.md` §4):** with pointwise `pair_loss` crushed to `0.005`, the shift-invariant listwise softmax losses (`proposal_loss`, `rerank_listwise_loss`) had left candidate logit levels unanchored! 2. **Coreferent and multi-occurrence target inflation (`argument_coref.py`, `processor.py`, `boundary_preprocessing.py`):** Both `expand_argument_coreferents` and repeated string-gold arguments compiled every mention in a document as a separate required filler (`len(ft.values) = K`) instead of span alternatives of one filler (`len(ft.values) = 1`). 3. **Double event dropout (`processor.py`):** `remove_events_prob: 0.2` dropped 20% of gold event types on top of

`NegativePoolSampler` (and even fired inside the inference dict-schema branch). Set `remove_events_prob: 0.0` in `eb22g.yaml`. 4. **Offset-dict gold in `eval_metrics.py` and standalone `evaluate_config` in `train.py`:** Fixed `_gold_event_trigger_set`, `_gold_event_argument_set`, and `_gold_oneie_sets` to unwrap `{text, start, end}` dicts (`casie_off` perfect-prediction F1 `0.0 -> 1.0`), and routed `evaluate_config` through `_eval_split_records` (`30,320 -> 19,286` val records).

- **Breaking the CASIE ceiling (`overfit-events-eb22g`) and the 5-epoch `eb22g` base run (`11-04..06`):**
- On the 64-doc × 400-step A100 A/B (`overfit-events-eb22g` on `eb22f:checkpoint-epoch-1`), `eb22g` treatment (`pair_split_reduction: true`, `hard_negative_keep_all_when_absent: false`, `remove_events_prob: 0.0`) actively trained `pair_loss` (`1.6638 -> 0.4118` vs control's flat `0.0113`), lifting `@0.5` strict micro-F1 on **CASIE** from `0.3619 ->` **`0.5210`** (`0.5984` at step 350) on `event_trigger` and `0.2506 ->` **`0.4319`** (`+0.1813`) on `event_argument`, and on **CMNEE** from `0.6375 ->` **`0.9394`** (`+0.3019`) on `event_trigger` and `0.5470 ->` **`0.8005`** (`+0.2535`) on `event_argument`.
- On the full 5-epoch `eb22g` run (`bcfe500c`, 1x A100 SXM4), validation strict micro-F1 climbed monotonically across Epochs 1–4 (`head_min` `0.0448 -> 0.0562 -> 0.1957 ->` **`0.2145`**), with the swept threshold moving up to **`0.5`** by Epoch 3 and Epoch 4 reaching `event_type` **`0.9524`**, `event_trigger` **`0.5656`** (`R=0.7358`, fair **`0.5916`**), `event_argument` **`0.3046`** (`P=0.3202, R=0.2905`, relaxed **`0.5223`**, fair **`0.5080`** — nearly 5× `eb22d` Epoch 2's `0.0627`), `event` **`0.6317`** (relaxed **`0.6818`**), `event_cluster` **`0.5681`**, `event_cluster_argument` **`0.2655`**, `entity` **`0.3689`**, `relation` **`0.2145`**, `structure` **`0.2635`**, `classification` **`0.7166`**, and `overall` **`0.4149`**.